

Программирование дронов и создание автономных беспилотных миссий

Курс «Программные средства БПЛА»



Автор: Корягин Сергей Андреевич
асп. СПбГЭТУ «ЛЭТИ», педагог ЦО «Кудрово»

г. Кудрово, 2026

Версия документа: 1.1.0 (Дата сборки: 14.09.2026 23:0-20)

Содержание

1	Введение и структура курса	8
2	Подготовка к работе	11
2.1	Техника безопасности	11
2.2	Инструментарий разработчика	12
2.2.1	Работа с квадрокоптером: Pioneer Station	12
2.2.2	Изучение синтаксиса Lua	13
2.3	Подключение квадрокоптера	13
2.4	Ваша первая программа	14
3	Аппаратура управления	15
3.1	Настройка пульта	15
3.1.1	Настройки вкладки FUNCTION	16
3.1.2	Настройка вкладки SYSTEM	17
3.2	Связь пульта с приемником	18
3.2.1	Процедура привязки	18
3.2.2	Возможные проблемы и их решение	19
3.3	Стики и тумблеры	20
3.3.1	Режимы управления (Тумблеры)	20
3.4	Управление	22
3.4.1	Управление высотой (Газ / Throttle)	22
3.4.2	Вращение по курсу (Рыскание / Yaw)	22
3.4.3	Тангаж (Pitch) — Движение вперед/назад	23
3.4.4	Крен (Roll) — Движение влево/вправо	23
I	Программирование на LUA	25
4	Язык Lua	25
4.1	Введение	25
4.2	Толковый словарь	25
4.3	История и назначение	26
4.4	Lua в нашем курсе	26
4.5	Базовые конструкции Lua	26

4.6	Переменные	27
4.7	Арифметические операции	29
4.8	Логические операции	31
4.9	Условия	32
4.10	Циклы	33
4.11	Работа с модулями платформы	35
4.12	Типичные ошибки новичка	35
4.13	Примеры, используемые далее	36
5	Программирование светодиодов	37
5.1	Введение	37
5.2	Толковый словарь	37
5.3	Аппаратная часть и модуль Ledbar	38
5.3.1	Физическая реализация	38
5.4	Теория и интерфейсы API	39
5.5	Особенности адресации (Буфер драйвера)	40
5.6	RGB-модель и диапазоны	40
5.7	Адресация: где какой светодиод?	41
5.8	Модуль Ledbar	43
5.9	Таймеры	44
5.10	Событийная модель	45
5.11	Анализ подходов к программированию	46
5.11.1	1. Блокирующее выполнение (Sleep)	47
5.11.2	2. Асинхронные таймеры	47
5.11.3	3. Конечный автомат (State Machine)	47
5.12	Примеры комплексных сценариев	48
5.13	Проверка знаний	52
5.14	Практические задания: базовые операции	55
5.15	Практические задания: таймеры и алгоритмы	56
5.16	Практические задания: реакция на события	58
6	Автономные беспилотные миссии	60
6.1	Введение	60
6.2	Толковый словарь	60
6.3	Событийная модель и интерфейсы управления	61

6.4	Светодиоды для отладки миссии	62
6.5	Справочник функций полёта	62
6.5.1	Таймеры и события	63
6.6	Учебные ступени полёта	63
6.7	Геометрия траекторий: окружность	68
6.8	Архитектурные подходы	69
6.8.1	Пример структуры программы	70
6.9	Разбор примеров миссий	70
6.10	Рекомендации по разработке миссий	76
6.11	Проверка знаний	76
6.12	Задания для лабораторной работы	79
6.12.1	Базовые миссии	79
6.12.2	Продвинутое задания	82

II Программирование на Python 85

7 Программирование на Python 85

7.1	Что такое Python и зачем он нужен?	85
7.2	Переменные и типы данных	87
7.3	Арифметические операторы	88
7.4	Логические операции	89
7.5	Ветвление (Условные операторы)	91
7.6	Циклы	92
7.7	Функции	94
7.8	Модуль «Квадрокоптер»: Pioneer SDK	96
7.9	Светодиоды и диагностическая индикация	97
7.10	Первый полет: Взлет и посадка	100
7.11	Маневры и автоматический полет	101
7.12	Система контрольных упражнений	106

8 Задания к главе 110

8.1	Компьютерное зрение: ArUco и AprilTag	111
8.2	Визуальная навигация	120
8.3	Продвинутое работа с камерой	125

8.4	Телеметрия и работа с датчиками	126
8.5	Трекинг человека и жестов	127
8.6	Групповой полет (Рой дронов)	129
8.7	Итоговые проекты курса	130

III Профессиональные навигационные системы 132

9 Теоретические основы визуальной навигации 132

9.1	Введение в ArUco и AprilTag	132
9.2	Принцип работы алгоритма обнаружения	132
9.2.1	1. Предварительная обработка (Preprocessing)	132
9.2.2	2. Поиск контуров (Contour Extraction)	133
9.2.3	3. Исправление перспективы (Perspective Warping)	133
9.2.4	4. Декодирование (Decoding)	133
9.3	Коррекция ошибок и Расстояние Хэмминга	134
9.4	Сравнение типов словарей (Dictionaries)	134
9.5	Математика позиционирования (PnP Problem)	135
9.6	Практические задания	136

10 Создание навигационных карт 138

10.1	Калибровка камеры	138
10.1.1	Процесс калибровки	138
10.2	Генерация и печать маркеров	142
10.2.1	Векторные маркеры	145
10.2.2	Требования к монтажу	150
10.3	Форматы карт (JSON, XML, KML)	150
10.3.1	JSON (JavaScript Object Notation)	158
10.3.2	XML (eXtensible Markup Language)	159
10.3.3	KML (Keyhole Markup Language)	159
10.4	Визуализация и планирование	159

11 Алгоритмы комплексной навигации 161

11.1	Проблема шума и фильтрация	161
11.1.1	Фильтр Калмана (Kalman Filter)	161
11.2	Системы координат	162

11.3 Точная посадка с PID-регулятором	162
12 Техническая реализация программных модулей	164
12.1 Генератор карт (Marker Map Builder)	164
12.1.1 Структура класса Marker	164
12.1.2 Экспорт в XML	164
12.1.3 Экспорт в KML для Google Earth	165
12.2 Модуль визуализации	165
13 Банк практических заданий	167
13.1 Блок 1: Работа с камерой и ArUco (aruco_examples, camera_examples)	167
13.2 Блок 2: Телеметрия (get_telemetry)	167
13.3 Блок 3: Миссии и полет по точкам (mission_examples)	168
13.4 Блок 4: Групповой полет (group_flight_examples)	168
13.5 Блок 5: Трекинг объектов (PioneerHumanTracking)	169
13.6 Критерии оценки выполнения	169
14 Типичные ошибки и их устранение	170
14.1 Проблемы с камерой и маркерами	170
14.1.1 Дрон не видит маркер	170
14.1.2 Дрон летит не туда	170
14.2 Проблемы с подключением и Wi-Fi	171
14.2.1 Connection Refused / Timeout	171
14.3 Ошибки в коде Python	171
14.3.1 IndentationError	171
14.3.2 OpenCV Error: Assertion failed	171
14.4 Аварийные ситуации	171
14.4.1 Дрон не реагирует на Land	171
A Методика тестирования и безопасность	173
A.1 Протокол безопасности полетов	173
A.1.1 1. Подготовка зоны (Pre-flight)	173
A.1.2 2. Проверка дрона (Checklist)	173
A.1.3 3. Экстренная ситуация	173
A.2 Тестирование в разных условиях	173

A.2.1	Погодные условия (Ветер)	173
A.2.2	Освещение (Lux)	174
В	Готовые конфигурации (Templates)	174
B.1	Файл конфигурации camera_params.yaml	174
B.2	Шаблон скрипта полета (main.py)	174
	Список литературы	176

1. Введение и структура курса

О данном учебном пособии

Данное учебное пособие разработано для систематического освоения навыков программирования беспилотных летательных аппаратов на примере квадрокоптера «Геоскан Пионер». Курс построен таким образом, чтобы обеспечить плавное погружение в предметную область: от изучения синтаксических основ языка программирования до реализации сложных автономных полетных заданий.

Материал структурирован согласно классическому дидактическому принципу «от простого к сложному». На начальных этапах основное внимание уделяется фундаментальным конструкциям языка Lua, необходимым для понимания логики работы скриптов. Впоследствии, по мере накопления теоретической базы, фокус смещается на прикладные аспекты: работу с периферийным оборудованием (светодиодная индикация), обработку событий и алгоритмизацию автономного поведения дрона.

Такой подход позволяет сформировать у обучающегося целостное представление о взаимодействии программного кода и аппаратной части робототехнического комплекса.

Цели и задачи обучения

Основной целью курса является формирование устойчивых навыков разработки программного обеспечения для управления БПЛА. В процессе обучения решаются следующие задачи:

- **Освоение инструментария:** Изучение синтаксиса и семантики языка Lua, а также специфики его применения во встраиваемых системах.
- **Аппаратное взаимодействие:** Получение навыков управления бортовыми системами квадрокоптера, включая систему световой индикации и таймеры.
- **Событийно-ориентированное программирование:** Понимание принципов неблокирующего кода и обработки асинхронных событий, что является критически важным для робототехники.
- **Алгоритмическое мышление:** Развитие навыка разбиения сложных по-

летных задач на последовательность элементарных команд и состояний.

Структура учебного курса (Дорожная карта)

Учебный курс разделен на логически завершенные модули, каждый из которых является фундаментом для последующего:

- 1. Модуль 1: Подготовка и безопасность.** Вводный раздел, посвященный технике безопасности при эксплуатации БПЛА, настройке программного окружения и подготовке оборудования к работе.
- 2. Модуль 2: Основы языка Lua.** Базовый курс программирования. Здесь рассматриваются типы данных, переменные, условные операторы, циклы и таблицы. Этот модуль необходим тем, кто ранее не сталкивался с Lua.
- 3. Модуль 3: Периферия и таймеры.** Переход к прикладным задачам. Изучение методов управления RGB-светодиодами (модуль Ledbar) и работы с системным временем. Вводится понятие таймеров как основы для создания динамических сценариев без блокировки основного потока выполнения.
- 4. Модуль 4: Автономные миссии и события.** Продвинутый уровень. Разбор архитектуры автопилота, работа с событийной моделью (Events) и программирование полетных заданий. Обучающиеся научатся создавать скрипты для автоматического взлета, навигации по точкам и выполнения фигур пилотажа.
- 5. Приложение.** Справочные материалы, содержащие описание API функций, список системных событий и полезные фрагменты кода (сниппеты) для часто встречающихся задач.

Планируемые результаты обучения

По завершении курса обучающийся приобретет компетенции, необходимые для самостоятельной разработки полетных сценариев. Вы научитесь:

- Создавать эффективный и читаемый код на языке Lua, соответствующий стандартам разработки.
- Использовать возможности бортовой периферии для индикации статусов и режимов работы дрона.
- Проектировать архитектуру программ, устойчивую к ошибкам и способ-

ную корректно обрабатывать штатные и нештатные ситуации.

- Реализовывать автономные миссии различной сложности, от простого зависания до пролета по сложному маршруту с выполнением полезной нагрузки.

2. Подготовка к работе

Перед тем как приступить к написанию кода и практическим полетам, необходимо тщательно подготовить рабочее место и изучить особенности эксплуатации оборудования. В этом разделе мы подробно разберем правила техники безопасности, установку необходимого программного обеспечения и процесс загрузки программ на борт квадрокоптера.

2.1. Техника безопасности

Критически важно!

Соблюдение правил безопасности — это фундаментальный навык любого инженера и пилота БПЛА. Пренебрежение этими правилами может привести к травмам или поломке дорогостоящего оборудования.

Квадрокоптер «Геоскан Пионер» является сложным электромеханическим устройством. При работе с ним необходимо неукоснительно соблюдать следующие требования:

- **Работа с механикой только без питания:** Любые монтажные работы, замена деталей или подключение модулей расширения должны производиться на полностью обесточенном аппарате (аккумулятор отключен).
- **Снятие пропеллеров при отладке:** Это «золотое правило» программиста дронов.
 - При отладке скриптов, тестировании моторов или проверке реакции на команды пульта **пропеллеры должны быть сняты**.
 - Ремонт, сборка и разборка квадрокоптера всегда осуществляются со снятыми пропеллерами.
 - Случайный запуск двигателей (например, из-за ошибки в коде или случайного касания стика газа) при установленных пропеллерах неминуемо приведет к неконтролируемому взлету, травмам пальцев или повреждению имущества.
- **Обращение с аккумуляторами:** Используйте только исправные литий-полимерные (LiPo) батареи. Не оставляйте их на зарядке без присмотра. Если аккумулятор вздулся или имеет механические повреждения оболочки — его эксплуатация строго запрещена.

- **Порядок включения и выключения:**
 - **Включение:** Сначала всегда включается пульт управления (передатчик), и только затем подается питание на борт дрона. Это гарантирует, что дрон сразу увидит управляющий сигнал.
 - **Выключение:** В обратном порядке — сначала отключаем питание дрона, затем выключаем пульт.
- **Организация рабочего пространства:** Уберите со стола лишние предметы, провода и инструменты. Дрон, даже без пропеллеров, может создавать вибрацию, а с пропеллерами — сильный воздушный поток, который может сдуть мелкие детали или затянуть легкие предметы в винты.

2.2. Инструментарий разработчика

Для успешного освоения курса вам потребуется специализированное программное обеспечение. Мы разделяем инструменты на те, что необходимы для работы непосредственно с «Пионером», и те, что удобны для изучения чистого языка Lua.

2.2.1. Работа с квадрокоптером: Pioneer Station

Основной средой разработки для квадрокоптера «Геоскан Пионер» является **Pioneer Station**. Это интегрированная среда (IDE), предоставляющая полный цикл инструментов для взаимодействия с автопилотом:

- **Редактор кода:** Позволяет писать скрипты на языке Lua с подсветкой синтаксиса.
- **Загрузчик:** Обеспечивает компиляцию и передачу программ в память дрона.
- **Терминал и отладка:** Выводит сообщения из функции `print()` и позволяет следить за состоянием датчиков (гироскопа, акселерометра, дальномера) в реальном времени.
- **Менеджер прошивок:** Позволяет обновлять встроенное ПО автопилота до актуальной версии.

i Установка

Убедитесь, что на вашем компьютере установлена последняя версия Pioneer Station. Дистрибутив можно бесплатно скачать с официального сайта поддержки Геоскан.

2.2.2. Изучение синтаксиса Lua

Язык Lua является стандартным, и его синтаксис (циклы, условия, функции) можно изучать отдельно от квадрокоптера. Это часто бывает удобнее, так как не требует подключения аккумулятора и постоянной перезагрузки дрона.

Для тренировки в написании чистого кода на Lua (без специфичных функций дрона, таких как `ap.goToLocalPoint`) мы рекомендуем использовать:

- **ZeroBrane Studio:** Легковесная и быстрая IDE, созданная специально для Lua. Отлично подходит для новичков.
- **Visual Studio Code:** Популярный редактор кода. Для работы с Lua требуется установить расширение (например, *Lua by sumneko*).
- **Онлайн-компиляторы:** Для быстрой проверки коротких фрагментов кода можно использовать веб-сервисы, такие как *Lua Demo* или *Replit*.

Примечание: Код, написанный в сторонних редакторах, для запуска на дроне все равно необходимо будет перенести в Pioneer Station.

2.3. Подключение квадрокоптера

Процесс соединения дрона с компьютером состоит из нескольких простых шагов:

1. Убедитесь, что пропеллеры сняты (если вы не планируете немедленный вылет на полигоне).
2. Подключите аккумулятор к разъему питания квадрокоптера. Вы услышите характерную мелодию инициализации регуляторов оборотов, и светодиодная индикация начнет мигать.
3. Подключите квадрокоптер к USB-порту компьютера с помощью кабеля Micro-USB.
4. Запустите приложение Pioneer Station.

5. В интерфейсе программы выберите соответствующий COM-порт (обычно он определяется автоматически при подключении кабеля).

2.4. Ваша первая программа

Чтобы убедиться, что все системы работают исправно, напомним и запустим простейшую программу. Традиционно в программировании первой программой является вывод приветствия.

Алгоритм запуска скрипта:

1. **Написание:** Откройте редактор в Pioneer Station и введите код программы.
2. **Загрузка:** Нажмите кнопку «Загрузить» (иконка Play или стрелка). Скрипт будет передан в память автопилота.
3. **Исполнение:** После загрузки дрон автоматически перезагрузит виртуальную машину Lua и начнет выполнение скрипта. Результат работы вы увидите в окне «Консоль» (Log).

Пример: Hello World

```
lua/examples/02_sensors/getting_started_hello.lua
-- Это комментарий, он не выполняется
-- Вывод сообщения в консоль Pioneer Station
print("Системы в норме. Привет, Пионер!")
```

Если в логе появилось ваше сообщение — поздравляем! Вы успешно настроили рабочее место и готовы погрузиться в мир программирования полетных контроллеров.

3. Аппаратура управления

В этой главе мы подробно рассмотрим устройство и настройку аппаратуры радиуправления FlySky FS-i6S, которая входит в стандартный комплект образовательного квадрокоптера «Геоскан Пионер». Пульт дистанционного управления (аппаратура) — это основной инструмент внешнего пилота, позволяющий контролировать полет, переключать режимы работы и выполнять настройку параметров дрона на расстоянии.

Аппаратура работает в паре с приемником, установленным на борту квадрокоптера, передавая команды по радиоканалу. Для корректной работы системы необходимо правильно настроить пульт и установить связь с приемником.

3.1. Настройка пульта

Перед первым использованием или после сброса настроек необходимо сконфигурировать пульт управления. Это обеспечит правильную интерпретацию команд полетным контроллером.

Для настройки пульта выполните следующие действия шаг за шагом:

1. Вставьте батарейки (4 шт. типа АА) в батарейный отсек и включите пульт дистанционного управления. Для включения необходимо нажать одновременно две кнопки питания (расположены в центре лицевой панели) и удерживать их до включения экрана.
2. После загрузки на дисплее появится главный экран с отображением текущего состояния, напряжения батарей и положения стиков.

Подсказка

Подсказка: В целях безопасности пульт проверяет положение переключателей при включении. Если на экране отображается предупреждение «Warning!!! Place all switches in their up position», это означает, что один из тумблеров находится в активном состоянии или стик газа не в нуле. Приведите все переключатели в верхнее положение (от себя), а левый стик (газ) опустите в самое нижнее положение.

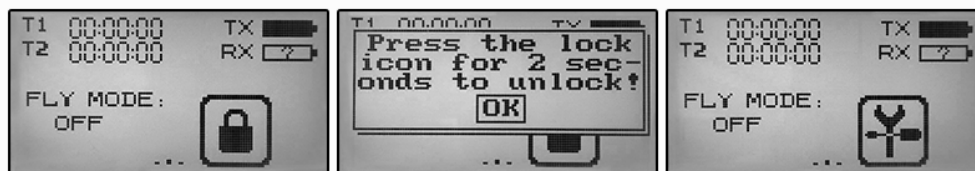


Рис. 1: Главный экран пульта управления FS-i6S

3. Изначально сенсорный экран заблокирован от случайных нажатий, о чём сигнализирует иконка замка в нижнем правом углу. Для разблокировки нажмите на иконку и удерживайте её в течении 2-4 секунд. После разблокировки иконка сменится на изображение гаечного ключа.
4. Нажмите на иконку с гаечным ключом, чтобы попасть в меню настроек. Меню разделено на две основные вкладки:
 - **FUNCTION** — здесь производится настройка функций самого пульта (каналы, реверсы и т.д.).
 - **SYSTEM** — отвечает за системные настройки связи и приемника.

3.1.1. Настройки вкладки FUNCTION

В этой вкладке мы настроим назначение каналов и направление их работы. Это критически важно для того, чтобы движения стиков и переключения тумблеров вызывали ожидаемую реакцию дрона.

Для работы с «Пионером» выполните следующие настройки:

1. **Настройка реверса каналов (Reverse):**
 - Перейдите в меню **REVERSE**.
 - Установите каналы **Ch2** (Тангаж) и **Ch4** (Рыскание) в положение **Rev** (Реверс). Остальные каналы оставьте в **Norm**.
2. **Настройка вспомогательных каналов (Aux. Channels):** Для управления режимами полета и дополнительными функциями необходимо назначить физические тумблеры на логические каналы пульта. Перейдите в меню **AUX. CHANNELS** и настройте каналы следующим образом:
 - **Channel 5:** Нажмите на кнопку выбора канала. В окне **CH TYPE** выберите **SWx** (переключатель). Затем нажмите на надпись **SwA** (по умолчанию) и в открывшемся списке выберите тумблер **SwC** (трехпозиционный переключатель справа). Этот канал обычно используется для смены режимов полета.

- **Channel 6:** Аналогично настройте на тумблер **SwD** (крайний правый переключатель).
- **Channel 7:** Настройте на тумблер **SwB** (трехпозиционный переключатель слева).
- **Channel 8:** Настройте на тумблер **SwA** (крайний левый переключатель).



Рис. 2: Меню настройки вспомогательных каналов

3.1.2. Настройка вкладки **SYSTEM**

В этой вкладке задаются параметры протокола передачи данных и режим работы стиков.

1. Выбор протокола выхода (**Output Mode**):

- Перейдите в меню **OUTPUT MODE**.
- Убедитесь, что в поле **Output** выбран протокол **PPM**. Полетный контроллер «Пионера» настроен на прием сигналов именно в этом формате. Также должен быть выбран режим **S.BUS** в поле **Serial**, если используется соответствующее подключение, но для базовой настройки PPM является основным.

2. Режим стиков (**Sticks Mode**):

- Перейдите в меню **STICKS MODE**.
- Выберите режим **M2 (Mode 2)**. В этом режиме левый стик отвечает за Газ (вверх/вниз) и Рыскание (влево/вправо), а правый — за Тангаж (вперед/назад) и Крен (влево/вправо). Это наиболее распространенная схема управления для квадрокоптеров.

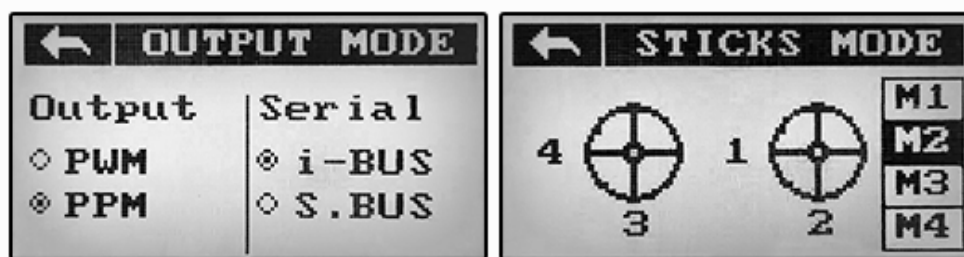


Рис. 3: Системные настройки: выбор режима PPM и Mode 2

3.2. Связь пульта с приемником

Процедура привязки (Binding) необходима для того, чтобы приемник, установленный на квадрокоптере, «узнал» ваш пульт и реагировал только на его команды. Это исключает пересечение сигналов при полетах нескольких дронов одновременно.

⚠ Внимание

Внимание: Для использования квадрокоптера «Пионер Мини» с пультом радиуправления, в автопилот должны быть загружены соответствующие параметры! Без этого дрон может не реагировать на команды.

3.2.1. Процедура привязки

Выполните следующие действия для сопряжения устройств:

1. Убедитесь, что приемник (FS-A8S или аналогичный) подключен к плате автопилота.
2. Найдите на корпусе приемника маленькую кнопку с подписью «**BIND**». Нажмите её и **удерживайте**.
3. **Не отпуская** кнопку BIND, подайте питание на квадрокоптер (подключите аккумулятор).
4. После подачи питания светодиод на приемнике должен начать **быстро мигать**. Это означает, что приемник перешел в режим привязки. Теперь кнопку можно отпустить.
5. Включите пульт управления. Зайдите в меню настроек (**SYSTEM**) и выберите пункт **RX Bind**.
6. На экране пульта появится надпись **Binding to RX....** В этот момент пульт

- ищет приемник в режиме привязки.
7. Как только связь установится, светодиод на приемнике начнет **мигать медленно**.
 8. Для завершения процедуры нажмите иконку «Назад» (←) на пульте, чтобы выйти из меню RX Bind.
 9. После выхода из режима привязки светодиод на приемнике должен загореться **постоянным светом** (красным). Это подтверждает успешное сопряжение и наличие стабильной связи.

⚠ Внимание

Важно: Чтобы избежать конфликтов управления, связывайте только один пульт с одним приемником. Если вы меняете пульт или приемник, процедуру привязки необходимо повторить.



Рис. 4: Приемник FS-A8S: расположение кнопки BIND

3.2.2. Возможные проблемы и их решение

Если после выполнения процедуры квадрокоптер не реагирует на команды:

- **Проверьте протокол:** Зайдите в настройки пульта **OUTPUT MODE** и убедитесь, что выбран **PPM**. Другие протоколы (например, PWM или i-BUS) могут не восприниматься текущей прошивкой автопилота без дополнительной настройки.
- **Повторите привязку:** Возможно, вы отпустили кнопку BIND слишком ра-

но или не успели включить режим привязки на пульте. Попробуйте снова.

- **Проверьте параметры:** Если вы летаете на Pioneer Mini, убедитесь через станцию наземного управления, что загружены корректные параметры для работы с RC-пультом.

В случае, если выполнить привязку не получается, вы всегда можете обратиться в техподдержку: support@geoscan.aero или в telegram-канал [@geoscan_edu](https://t.me/geoscan_edu).

3.3. Стиki и тумблеры

Пульт управления предоставляет пилоту гибкие возможности контроля. Основные органы управления — это два джойстика (стика) и набор переключателей (тумблеров).

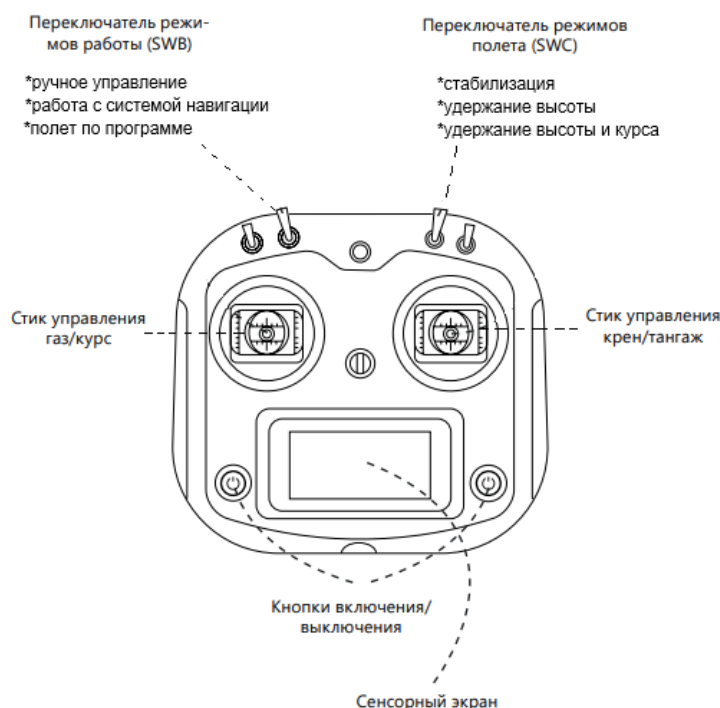


Рис. 5: Обзор органов управления пульта FS-i6S

3.3.1. Режимы управления (Тумблеры)

Тумблеры используются для переключения полетных режимов и активации функций. Ранее мы настроили тумблер **SwC** (трехпозиционный справа) на 5-й канал.

Изменение режима полета (переключатель SwC):

- **Верхнее положение (Stabilize):** Режим стабилизации. Стик газа напрямую управляет мощностью моторов. Это наиболее «ручной» режим, требующий от пилота постоянного контроля высоты. Подходит для опытных пилотов.
- **Среднее положение (AltHold):** Режим удержания высоты. Квадрокоптер использует барометр для автоматического поддержания заданной высоты. Стик газа в центре — высота не меняется. Стик вверх — набор высоты, вниз — снижение. Это рекомендуемый режим для обучения.
- **Нижнее положение (Headless):** Режим удержания высоты и курса. В дополнение к удержанию высоты, упрощается навигация. Квадрокоптер «запоминает» направление, в котором он был ориентирован при включении моторов (обычно «от пилота»). В полете, независимо от того, куда повернут нос дрона, нажатие стика «вправо» заставит его лететь вправо относительно пилота.

Внимание

Физика полета: Для определения высоты полетный контроллер использует барометр, измеряющий атмосферное давление. Помните, что резкие изменения давления (например, открытие окна в помещении или сквозняк) могут быть восприняты дроном как изменение высоты, что приведет к коррекции оборотов моторов.

Режимы работы пульта:

- **Ручное управление:** Вы полностью контролируете дрон с помощью стиков. Режим полета выбирается переключателем SwC.
- **Навигация и GPS:** В режимах с использованием системы навигации или GPS квадрокоптер управляется с пульта, но автоматически удерживает высоту и позицию, если отпустить стики.
- **Полет по программе:** Пионер выполняет загруженную миссию. Вы всегда можете переключить тумблер режимов (например, пощелкать SwC), чтобы перехватить управление и перевести дрон в ручной режим.

3.4. Управление

Пилотирование квадрокоптера требует координации и понимания того, как каждое движение стика влияет на движение аппарата в пространстве. Рассмотрим управление в режиме **Mode 2**.

3.4.1. Управление высотой (Газ / Throttle)

Левый стик, движение вверх-вниз.

- **Стик вверх:** Обороты моторов увеличиваются, подъемная сила растет, квадрокоптер набирает высоту.
- **Стик вниз:** Обороты уменьшаются, квадрокоптер снижается.

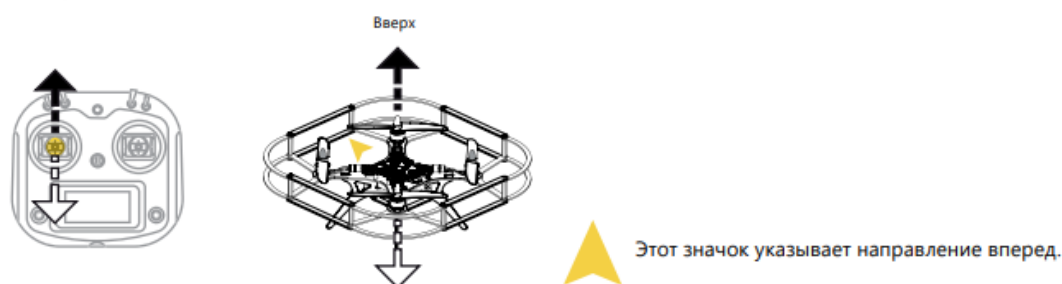


Рис. 6: Управление газом: набор высоты и снижение

⚠ Внимание

Важно: В режиме AltHold (удержание высоты) центр стика соответствует зависанию. Для взлета необходимо поднять стик выше середины.

3.4.2. Вращение по курсу (Рыскание / Yaw)

Левый стик, движение влево-вправо.

Отвечает за поворот квадрокоптера вокруг своей вертикальной оси. Это позволяет изменить направление «взгляда» дрона.

- **Стик влево:** Нос квадрокоптера поворачивается влево (против часовой стрелки).
- **Стик вправо:** Нос квадрокоптера поворачивается вправо (по часовой стрелке).

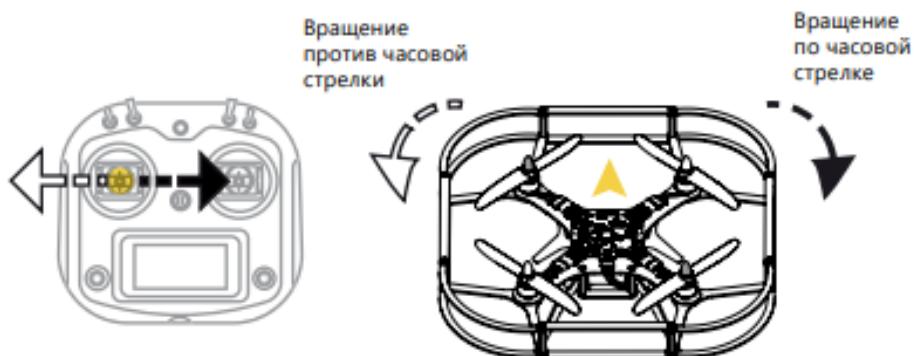


Рис. 7: Рыскание: вращение вокруг оси

3.4.3. Тангаж (Pitch) — Движение вперед/назад

Правый стик, движение вверх-вниз.

Изменяет угол наклона квадрокоптера в продольной плоскости, заставляя его двигаться горизонтально.

- **Стик вверх (от себя):** Нос опускается, дрон летит вперед.
- **Стик вниз (на себя):** Нос поднимается, дрон летит назад.

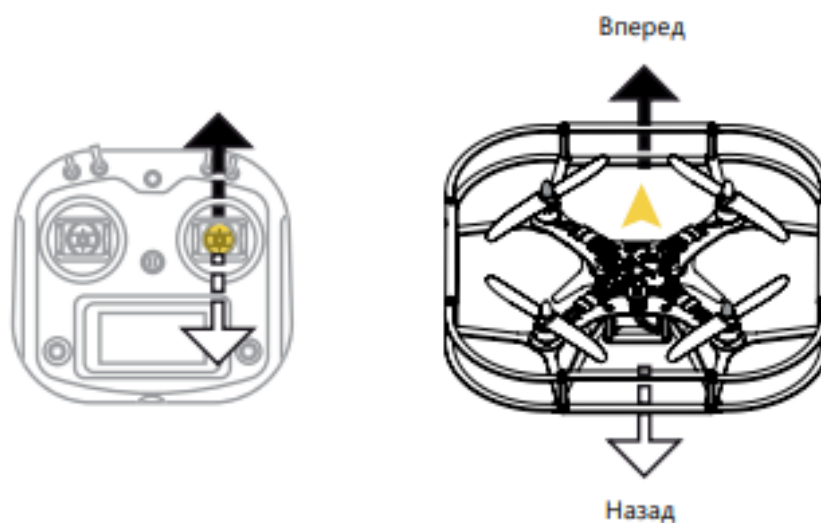


Рис. 8: Тангаж: движение вперед и назад

3.4.4. Крен (Roll) — Движение влево/вправо

Правый стик, движение влево-вправо.

Изменяет угол наклона квадрокоптера в поперечной плоскости, заставляя его смещаться вбок.

- **Стик влево:** Дрон наклоняется влево и летит боком влево.

- **Стик вправо:** Дрон наклоняется вправо и летит боком вправо.

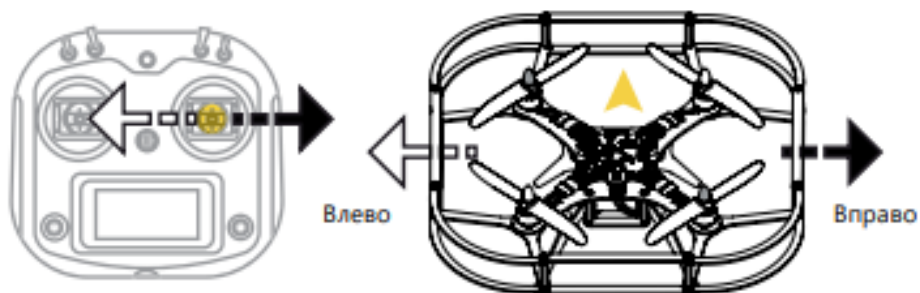


Рис. 9: Крен: движение боком

💡 Подсказка

Совет: При обучении полетам держите хвост квадрокоптера (где установлен аккумулятор или светодиод) к себе. Так направления управления будут совпадать: стик вправо — дрон вправо. Если дрон развернется к вам носом, управление станет «зеркальным» (стик вправо — дрон влево с вашей точки зрения).

Часть I.

Программирование на LUA

4. Язык Lua

Цели раздела

- Освоить типы данных, области видимости и таблицы.
- Научиться работать с арифметикой, логикой, условиями и циклами.
- Использовать идиомы Lua для краткости и надёжности кода.
- Подготовить основу для программирования индикации и миссий.

4.1. Введение

Lua — лёгкий и встраиваемый язык программирования, созданный в начале 1990-х для расширения приложений скриптами. Он компактен, прост и быстро работает, поэтому широко используется в встраиваемых системах, играх и робототехнике.

4.2. Толковый словарь

Нажмите на термин в тексте, чтобы перейти к его определению.

Число вещественное значение (обычно двойной точности), например 3.14 или 10; доступна математика через `math`.

Строка последовательность символов в кавычках, например "Lua"; операции: конкатенация `..`, длина `#s`.

Булево логическое значение `true` или `false`; используется в условиях.

nil «пусто»/«нет значения»; удаляет поля таблицы, означает отсутствие данных.

Таблица универсальная структура (массив и словарь одновременно); хранит позиции `t[1]` и поля `t.x`.

Функция блок кода, вызываемый по имени, принимает аргументы и возвращает значения.

type() функция, возвращающая тип значения: `type(10) -> "number"`.

Область видимости `local` делает переменную видимой только в текущем блоке; без `local` — глобальная.

Оператор символ или слово, выполняющее действие:
`+ - * / % ^ and or not`.

Выражение комбинация значений и операторов, дающая новое значение.

Условие логическое выражение, определяющее выбор ветки выполнения.

Цикл повторение действий: `for`, `while`, `repeat until`.

ipairs/pairs итераторы по таблицам: `ipairs` для массивов; `pairs` для словарей (порядок не гарантирован).

Истинность в Lua ложными считаются только `false` и `nil`.

table.unpack распаковывает таблицу в список аргументов:
`table.unpack({1,0,0}) -> 1,0,0`. Это функция из библиотеки `table`, а не глобальная функция: вызывать её нужно как `table.unpack`. Для краткости в начале скрипта часто пишут `local unpack = table.unpack`.

4.3. История и назначение

- Появился как средство «подключаемой логики» поверх приложений на C/C++. - Минимальные требования к памяти, переносимость и простой синтаксис. - Табличные структуры данных (`table`) и функции первого класса делают язык выразительным при небольшом ядре.

4.4. Lua в нашем курсе

Ниже — краткий анонс того, для чего нам понадобится Lua; каждый из этих инструментов подробно разбирается в своей главе далее, здесь их не нужно запоминать. - Мы используем Lua, чтобы программировать поведение квадрокоптера «Геоскан Пионер». - Через модуль `Ledbar` управляем светодиодной индикацией. - Через `Timer` создаём неблокирующие анимации. - Через `callback(event)` реагируем на системные события (посадка, напряжение, достижение точки).

4.5. Базовые конструкции Lua

Ниже — краткий пример базовых элементов: переменная, строка и таблица. Циклы и условия разбираются далее.

Практика

Этот и последующие примеры кода вы можете копировать в редактор Pioneer Station и запускать, чтобы увидеть результат в консоли.

 lua/examples/04_language/lua_intro_basics.lua

```
-- числовая переменная
local x = 10
-- строковая переменная
local s = "Lua"
-- таблица с позиционными элементами и именованным полем
local t = {1, 2, 3}
t.mode = "auto"
-- выводим значения и длину массива
print(x, s, #t) -- 10 Lua 3
```

Итоги раздела

Вы научились объявлять и различать типы данных, писать выражения, условия и циклы, работать с таблицами как с массивами и словарями, применять стандартные математические функции. Эти навыки будут использованы в главах о светодиодах и автономных миссиях.

4.6. Переменные

Переменная — это имя, за которым «закреплено» значение. В Lua есть базовые типы: **число**, **строка**, **булево**, **таблица**, **функция**, **nil**. Узнать тип можно через `type()`. **Локальная** переменная объявляется через `local` и видна внутри текущего блока; **глобальная** — без `local` (видна во всём скрипте).

Число — это величина для арифметики (например, 3.14 или 10). Строка — текст в кавычках (например, "Pioneer"), над ней выполняют конкатенацию `..` и получают длину `#s`. Булево — `true/false`, используется в **условиях**. Таблица — универсальная структура: массив `t[1]` и словарь `t.x` одновременно. Функция — именуемый блок кода, например `local function f(x) return x*2 end`.

 lua/examples/04_language/variables_intro.lua

```
-- локальная числовая переменная
local x = 10
-- глобальная строковая переменная
name = "Pioneer"
-- локальная булева переменная
local ready = true
-- таблица с позиционными элементами и именованным полем
local t = {1, 2, 3, speed = 3.5}
-- множественное присваивание
x, y = 1, 2
-- обмен значениями
x, y = y, x
-- выясняем типы значений
print(type(x), type(name), type(ready), type(t)) -- number string
-- boolean table
```

 lua/examples/04_language/lua_variables_tables.lua

```
-- словарь (именованные поля)
local p = {x = 1, y = 2}
-- массив (позиционные элементы)
local a = {10, 20, 30}
-- добавление нового поля
p.z = 3
-- добавление нового элемента массива
a[4] = 40
```

Пояснения. - Таблица — единственная сложная структура в Lua; через неё строят списки, словари и объекты. - nil при присваивании полю таблицы удаляет это поле: `t.speed = nil`. - Глобальные переменные удобно использовать только для настроек и состояний, иначе код становится хрупким; предпочитайте local.

Задачи для самостоятельного решения

1. Объявите локальную переменную `x` со значением 42.
2. Присвойте строку "Pioneer" глобальной переменной `name`.
3. Поменяйте местами значения `a` и `b` одной строкой.
4. Создайте таблицу `t` с полем `speed=3.5` и элементами 1, 2, 3.
5. Добавьте в таблицу `t` поле `mode` со значением "auto".
6. Объявите `local flag = false` и затем присвойте глобальной `flag` значение `true`.

7. Создайте пустую таблицу `buf` и добавьте в неё числа 5 и 7.
8. Удалите поле `speed` из таблицы `t`, присвоив `nil`.
9. Присвойте одновременно значения `r=1`, `g=0`, `b=0`.
10. Создайте таблицу `color` вида `{r=1,g=0,b=0}`.
11. Извлеките третий элемент массива `a` в переменную `third`.
12. Присвойте `local s = "Lua"` и `local n = 3.14`.
13. Создайте локальную функцию `id`, возвращающую свой аргумент.
14. Объявите `local config = nil` и затем присвойте `config = {enabled=true}`.
15. Создайте таблицу `point` и добавьте в неё `x=10` и `y=20`.

4.7. Арифметические операции

Операции: `+` `-` `*` `/` `%` `^` (плюс, минус, умножение, деление, остаток, возведение в степень). Сравнения: `<` `<=` `>` `>=` `==` `~=`. Оператора `**` в Lua нет ни в одной версии языка — это оператор из Python; для возведения в степень всегда используйте `a ^ b`. Оператор целочисленного деления `//` в симуляторе (Lua 5.3) доступен и работает, однако на других сборках Lua он может отсутствовать, поэтому для переносимости кода надёжнее писать `math.floor(a/b)`. В Lua 5.3 у **чисел** есть два подвида — целые (`integer`) и вещественные (`float`), — но для задач этого курса это различие почти не важно: арифметика работает с ними единообразно. Единственное, что стоит запомнить: оператор `/` всегда возвращает вещественное число, даже если оба операнда целые (`10 / 2` даёт `5.0`, а не `5`); поэтому для округлений применяются `math.floor`, `math.ceil` и `math.abs`.

Приоритет: сначала выполняются `^`, затем `*` `/` `%`, далее `+` `-`. Скобки меняют порядок вычислений. Остаток `%` определён как «остаток от деления», полезен для задач чётности и циклических индексов.

lua/examples/04_language/lua_arithmetic_ops.lua

```
-- приоритет: умножение выполняется раньше сложения
local a = 2 + 3 * 4
-- остаток от деления (modulo)
local r = 5 % 2
-- возведение 2 в степень 3
local p = 2 ^ 3
-- сравнение: равно?
```

```
local eq = (a == 14)
-- сравнение: больше?
local gt = (a > 10)
-- безопасное «целочисленное» деление с округлением вниз
local q = math.floor(17 / 4) -- 4
-- евклидова длина вектора
local len = math.sqrt(3*3 + 4*4) -- 5
```

Пояснения. - `math.sqrt` возвращает корень квадратный; `math.pi` — число π . - `math.max` и `math.min` ограничивают значения: `v = math.max(0, math.min(1, v))`. - Деление на ноль следует избегать: проверяйте знаменатель перед делением. - Перевод градусов в радианы и обратно можно писать вручную (`deg * math.pi / 180`) или короче — через готовые функции `math.rad(deg)` и `math.deg(rad)`. Дальше в курсе (полёт по окружности, курс уав) встречаются оба варианта записи.

Задачи для самостоятельного решения

1. Вычислите выражение $3 + 4 * 2 - 5$.
2. Найдите остаток $17 \% 4$.
3. Запишите $3 ^ 4$.
4. Вычислите среднее $(x + y) / 2$.
5. Запишите `math.abs(x2 - x1)`.
6. Вычислите `math.floor(n / 3)`.
7. Запишите условие «n чётное» с %.
8. Вычислите `math.sqrt(x*x + y*y)` (евклидова длина).
9. Ограничьте `v` в диапазоне `[0,1]`: `v = math.max(0, math.min(1, v))`.
10. Переведите градусы в радианы: `rad = deg * math.pi / 180`.
11. Вычислите площадь прямоугольника $S = w * h$.
12. Запишите среднее трёх чисел $(a+b+c)/3$.
13. Реализуйте линейную интерполяцию `lerp = a + t*(b-a)`.
14. Посчитайте сумму массива `a`: примените цикл `for`.
15. Найдите максимум массива `a`: пройдитеесь и сравнивайте.

4.8. Логические операции

Логика: and (И), or (ИЛИ), not (НЕ). В Lua ложными считаются только false и nil (**истинность**). Операторы and и or обладают **коротким замыканием**: правая часть вычисляется только при необходимости. Сравнения работают для чисел и строк (лексикографически).

 lua/examples/04_language/lua_logic_ops.lua

```
-- логическое И и НЕ
local ok = true and not false
-- «тернарный» идиом в Lua через and/or
local sel = (gt and 1 or 0)
-- композиция условий «в диапазоне»
local within = (a >= 0) and (a <= 100)
-- проверка непустой строки
local nonempty = (name ~= nil) and (name ~= "")
-- исключающее ИЛИ для булевых
local xor = (p and not q) or (q and not p)
```

Пояснения. - Идиома (cond and A or B) выбирает A при истинном cond и B иначе. - nil часто означает «нет данных», поэтому проверяйте значение перед доступом к полям. - Строки сравниваются лексикографически: "a" < "b" истинно.

Задачи для самостоятельного решения

1. Составьте условие «x в диапазоне [0,1]».
2. Верните 1 если b==true, иначе 0 с and/or.
3. Составьте проверку «строка s не пустая» (s~="").
4. Сравните "a" и "b": проверка «меньше» (<).
5. Составьте выражение «x>0 и y~=0».
6. Запишите де Моргана: not(a and b) эквивалент (not a) or (not b).
7. Реализуйте «исключающее ИЛИ» для p и q.
8. Преобразуйте число n в булево: n~=0.
9. Составьте логическое выражение: t.enabled == true or t.mode=="auto".
10. Верните «сигнал опасности», если voltage<3.5 или temp>80.
11. Напишите предикат «индекс чётный»: i % 2 == 0.
12. Составьте выражение, выбирающее максимум из a и b через and/or.
13. Используя not, инвертируйте булеву переменную on.

14. Составьте условие «t не nil и t.value существует».
15. Напишите логическую функцию «входит ли символ c в набор {"r", "g", "b"}».

4.9. Условия

Ветвления выполняются через ключевые слова `if`, `then`, `elseif`, `else`, `end`. Истинными считаются любые значения, кроме `false` и `nil`. Шаблоны: «охранные условия» (ранний выход), таблица переходов (эмуляция конвейера состояний), каскад `elseif` для диапазонов.

 lua/examples/04_language/lua_conditions_if.lua

```
-- классификация числа x
if x > 0 then
    state = "positive"
elseif x == 0 then
    state = "zero"
else
    state = "negative"
end
```

 lua/examples/04_language/lua_conditions_check.lua

```
-- безопасно работаем с полями, если таблица и поле существуют
if t and t.value then
    result = t.value * 2
end
```

Пояснения. - «Охранное» условие позволяет быстро завершить ветку при неверных входных данных. - Вложенные условия стоит разбивать на логические блоки для читаемости. - Для выбора из фиксированного набора значений удобно использовать таблицу функций: `actions[state]()`.

Задачи для самостоятельного решения

1. Напишите условие, присваивающее `sign="+"` если `x>0`, иначе `sign="-"`.
2. Присвойте `parity="even"` при чётном `n`, иначе `"odd"`.
3. Составьте ветвление: `temp<0` → `"cold"`, `0<=temp<20` → `"cool"`, иначе `"warm"`.
4. Присвойте `level="low"` если `v<3.7`, `"ok"` если `v>=3.7`.

5. Напишите условие: если `config.enabled` истинно, то `mode="run"`, иначе `"stop"`.
6. Реализуйте проверку: если `t` не `nil` и `t.x` существует, удвойте `t.x`.
7. Напишите ветвление, присваивающее цвет: `r=1,g=0,b=0` при `state=="RED"`, `r=1,g=1,b=0` при `"YELLOW"`, иначе `r=0,g=1,b=0`.
8. Составьте условие: если `s==""`, то `empty=true`, иначе `false`.
9. Реализуйте защиту от деления на ноль: если `b~=0`, то `q=a/b`.
10. Напишите проверку «строка `name` начинается с "P"» (используйте `string.sub`).
11. Если `voltage<3.5`, установите `alarm=true`.
12. Составьте ветвление: если `t.mode=="manual"` и `t.enabled`, то `run=true`.
13. Если `count==0`, присвойте `count=1`, иначе увеличьте на 1.
14. Напишите условие, которое инвертирует булеву переменную `on`.
15. Реализуйте тройное ветвление по `speed`: `slow`, `medium`, `fast`.

4.10. Циклы

Циклы: числовой `for` (счётчик от `start` до `finish` с шагом), `while` (пока условие истинно), `repeat ... until` (выполнить хотя бы один раз). Для таблиц: `ipairs` (массив) и `pairs` (словарь). `break` прерывает цикл; `continue` отсутствует (можно имитировать через `goto`, но лучше структурировать условие).

 `lua/examples/04_language/loops_sum.lua`

```
-- суммирование 1..10
local sum = 0
for i = 1, 10 do
    sum = sum + i
end
```

 `lua/examples/04_language/loops_while.lua`

```
-- увеличиваем x до 5
local x = 0
while x < 5 do
    x = x + 1
end
-- уменьшаем x до 0 минимум один раз
repeat
    x = x - 1
```

```
until x == 0
```

 lua/examples/04_language/loops_iterators.lua

```
-- умножаем элементы массива на 2
local a = {10, 20, 30}
for i, v in ipairs(a) do
    a[i] = v * 2
end
-- увеличиваем значения словаря на 1
local t = {x=1, y=2}
for k, v in pairs(t) do
    t[k] = v + 1
end
```

Пояснения. - `ipairs` проходит по индексам `1..N` без пропусков; `pairs` проходит по всем полям словаря в произвольном порядке. - В числовом `for` границы включительно: `for i=1,3 do` даёт `1,2,3`. - Для обхода диапазона индексов, используемых в светодиодной линейке, применяйте `for i=0,28 do`.

Задачи для самостоятельного решения

1. Посчитайте сумму чисел от 1 до 100.
2. Найдите максимум в массиве `a`.
3. Подсчитайте количество чётных элементов в массиве `a`.
4. Постройте новый массив `b` из квадратов элементов `a`.
5. Разверните массив `a` (создайте `rev`).
6. Удалите из массива `a` все нули.
7. Сформируйте массив `seq` из чисел `1..20`.
8. Вычислите факториал `n` с помощью `while`.
9. Реализуйте цикл `repeat ... until`, который увеличивает `x` до `>=10`.
10. Скопируйте словарь `t` в `copy`, увеличив каждое значение на 1.
11. Объедините два массива `a` и `b` в `c`.
12. Посчитайте, сколько значений `true` в массиве булевых `flags`.
13. Создайте таблицу `squares` с полями `["1"]=1`, `["2"]=4`, ..., `["10"]=100`.
14. Пройдитесь по индексам `0..28` и заполните таблицу `idx` значениями `i`.
15. Постройте массив `fibs` из первых 10 чисел Фибоначчи.

4.11. Работа с модулями платформы

1. `Ledbar.new(N)` — создание объекта управления (обычно $N=25$).
2. `leds:set(i, r, g, b)` — установка цвета светодиода i . Параметры r, g, b — от 0.0 до 1.0.
3. `Timer.new(period, fn)` — создание периодического таймера.
4. `Timer.callLater(delay, fn)` — отложенный вызов функции.
5. `function callback(event)` — обработчик системных событий.

4.12. Типичные ошибки новичка

1. **Регистр букв:** `Print`, `PRINT` и `print` — это три разных слова. В Lua все ключевые слова и функции пишутся строчными буквами (кроме классов вроде `Ledbar`).
2. **Путаница с индексами:**
 - Обычные таблицы Lua нумеруются с 1: `t[1]`.
 - Аппаратные модули (светодиоды, массивы из C++) часто нумеруются с 0: `leds:set(0, ...)`. Будьте внимательны!
3. **Бесконечные циклы:**

```
lua/examples/04_language/lua_platform_infinite_loop.lua
-- ОПАСНО! Этот код "повесит" выполнение скрипта
while true do
    print("Я завис")
end
```

Вместо вечных циклов используйте `Timer`.

4. **Ошибка "attempt to index a nil value":** Вы пытаетесь обратиться к полю переменной, которая пуста (`nil`). Проверьте, правильно ли вы создали объект или таблицу.

4.13. Примеры, используемые далее

Обратите внимание

В следующих примерах используется библиотека `Ledbar`, которую мы подробно изучим в следующей главе. Сейчас сосредоточьтесь на синтаксисе языка Lua (циклы, функции, таблицы), не вдаваясь в детали работы светодиодов.

lua/examples/04_language/leds_unpack.lua

```
local unpack = table.unpack
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local function changeColor(col)
    for i=0, ledNumber-1 do
        leds:set(i, unpack(col))
    end
end
changeColor({0,1,0})
```

lua/examples/04_language/leds_timer.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
for i=0, ledNumber-1 do leds:set(i, 0, 0, 0) end
Timer.callLater(3, function ()
    for i=0, ledNumber-1 do leds:set(i, 0, 0, 1) end
end)
function callback(event)
end
```

lua/examples/04_language/leds_blink.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
on = false
function updateBlink()
    local r = on and 1 or 0
    for i=0, ledNumber-1 do leds:set(i, r, 0, 0) end
    on = not on
end
blinkTimer = Timer.new(0.5, updateBlink)
blinkTimer:start()

function callback(event)
end
```

5. Программирование светодиодов

Цели раздела

- Понять RGB-модель и адресацию индексов 0..28.
- Научиться работать с Ledbar и таймерами без блокировок.
- Освоить событийную модель `callback(event)` и базовые паттерны индикации.
- Выполнить практикум: от простых эффектов до реакций на события.

5.1. Введение

Программирование световой индикации — это не просто способ украсить полёт, но и важнейший инструмент отладки и взаимодействия с пилотом. Светодиоды позволяют дрону «общаться»: сообщать о текущем режиме работы, уровне заряда батареи, готовности к взлёту или возникновении ошибок.

В этом уроке мы пройдем путь от простого включения лампочки до создания сложных динамических световых эффектов, работающих параллельно с полётной программой.

Цели урока

Мы научимся:

- Управлять цветом и яркостью светодиодов через код на Lua.
- Использовать таймеры для создания анимаций без задержки выполнения программы.
- Проектировать алгоритмы индикации, реагирующие на события дрона.

5.2. Толковый словарь

Нажмите на термин в тексте, чтобы перейти к его определению.

Модуль часть программы, отвечающая за конкретную задачу и предоставляющая функции для работы. Модуль [Ledbar](#) управляет светодиодами.

Драйвер программный компонент, который общается с аппаратурой и переводит команды в сигналы для устройств.

Буфер область памяти, куда мы записываем значения (например, цвета); драйвер считывает буфер и выводит свечение.

Переменная именованная «коробочка» со значением. Пример: $x = 10$.

Локальная/Глобальная локальная переменная объявляется через `local` и видна только внутри текущего блока; глобальная (без `local`) видна во всём скрипте.

Таймер «будильник», который вызывает указанную функцию периодически или один раз с задержкой.

Период время между двумя вызовами таймера.

Тик один вызов функции таймером.

Состояние текущее положение логики (например, «красный», «жёлтый», «зелёный»).

Конечный автомат модель, где программа находится в одном из состояний и переходит в другое при событиях.

RGB-компоненты три числа R, G, B (яркость каналов). Диапазон значений: $[0.0, 1.0]$.

Индекс светодиода номер светодиода в линейке (от 0 до 28).

API набор функций модуля. Часть из них вызывается от самого модуля (`Ledbar.new`, `Timer.new`), а часть — от созданного объекта через двоеточие (`leds:set(...)`); такие функции называют методами объекта.

Ledbar модуль управления RGB-светодиодами квадрокоптера.

5.3. Аппаратная часть и модуль Ledbar

5.3.1. Физическая реализация

Квадрокоптер Геоскан Пионер может быть оснащён модулем светодиодной навигации. Стандартная конфигурация подразумевает использование адресуемой светодиодной ленты или специального модуля расширения.

Устройство пикселя

Каждый светодиод состоит из трёх кристаллов:

- **Red (R)** — красный канал.
- **Green (G)** — зелёный канал.
- **Blue (B)** — синий канал.

Смешивая эти три цвета в разных пропорциях, мы можем получить любой оттенок радуги. Например, включение красного и зелёного даёт жёлтый цвет.

Питание

Светодиоды потребляют энергию аккумулятора. При написании алгоритмов стоит помнить, что включение всех светодиодов на полную яркость (белый цвет) создаёт максимальную нагрузку на систему питания, хотя для современных дронов это редко является критичным фактором.

5.4. Теория и интерфейсы API

Под **интерфейсом API** мы будем понимать набор функций и объектов, через которые скрипт общается с аппаратной частью дрона. Для светодиодной линейки нас интересуют прежде всего модули **Ledbar** и таймеры.

Модуль **Ledbar** связывает абстрактные **RGB-компоненты** с реальными светодиодами по их **индексам**. Скрипту не нужно думать о проводах и регистрах: достаточно выбрать номер светодиода и задать три числа яркости каналов.

Таймеры выступают в роли «двигателя времени» для анимаций. Вместо того чтобы блокировать выполнение с помощью `sleep()`, мы просим систему периодически вызывать нашу функцию (тик) или один раз через заданную задержку. В сочетании с **состояниями** это позволяет строить сложные, но управляемые сценарии индикации.

В следующих подразделах мы:

- разберём, как устроена аппаратная часть и модуль **Ledbar**;
- рассмотрим адресацию светодиодов и связь индексов с реальными лампочками;
- познакомимся с основными функциями **Ledbar** для установки цвета;
- изучим работу таймеров и их применение для динамических эффектов.

5.5. Особенности адресации (Буфер драйвера)

Важно: `Ledbar` работает с **буфером** из **29** элементов. Поэтому инициализация должна задавать размерность 29.

Обратите внимание

Требование: `Ledbar.new(29)`.

Типовой шаблон инициализации выглядит так:

```
lua/examples/05_leds/leds_buffer_init.lua

local unpack = table.unpack
local ledNumber = 29
local leds = Ledbar.new(ledNumber)

local function changeColor(col)
    for i=0, ledNumber-1 do
        leds:set(i, unpack(col))
    end
end

changeColor({0,1,0})
```

-- размерность буфера
-- объект управления лентой
-- проходим по всем индексам 0..28
-- записываем RGB в буфер
-- включаем зелёный

Пояснение. В примере создаётся объект `Ledbar` на 29 светодиодов и функция `changeColor(col)`, которая проходит по индексам `0..28` и вызывает `leds:set(i, ...)`. Основной принцип — один раз инициализировать буфер нужного размера и затем менять цвет сразу всей ленты через обход по индексам.

5.6. RGB-модель и диапазоны

RGB — три управляющих числа: R (красный), G (зелёный), B (синий) в `[0.0, 1.0]`. `0.0` — канал выключен, `1.0` — горит ярко. Смешивая каналы, получаем цвета: `{1,0,0}` (красный), `{0,1,0}` (зелёный), `{0,0,1}` (синий), `{1,1,0}` (жёлтый), `{1,1,1}` (белый), `{0,0,0}` (выключено). Плавная смена значений позволяет создавать анимации.

5.7. Адресация: где какой светодиод?

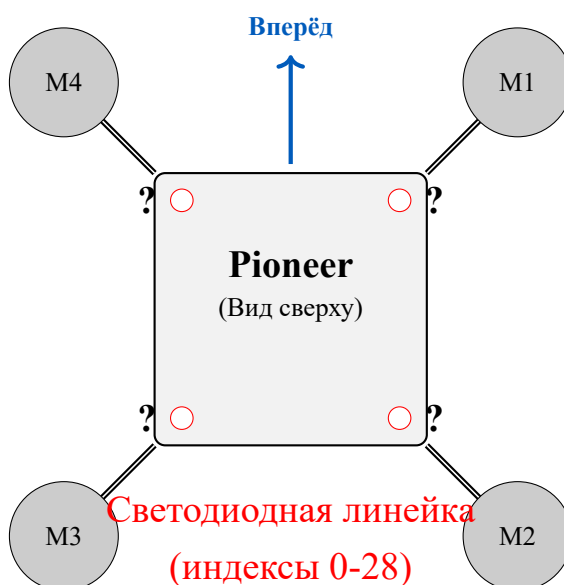
Мы выяснили, что в буфере 29 ячеек (0..28). На плате квадрокоптера вы видите светодиодную линейку из 29 элементов.

Как понять, какой индекс в коде соответствует какой лампочке в реальности?

Задание 1: Исследовательская задача

Напишите программу-исследователь, которая поможет составить «карту» светодиодов.

1. Включите светодиод с индексом 0 красным цветом.
2. Посмотрите на дрон. Какой светодиод загорелся? (Например, «Передний левый»).
3. Повторите для индексов 1, 2, 3, ..., 28.
4. Запишите результаты в тетрадь.



Индекс	Расположение (Например: Передний левый, центр, хвост)
0	
1	
2	
3	
4	
5	
...	
28	

Скрипт-исследователь

```
lua/examples/05_leds/leds_addressing_research.lua

local leds = Ledbar.new(29)

-- Функция для очистки (выключения всех)
local function clear()
    for i=0, 28 do leds:set(i, 0,0,0) end
end

clear()
-- Включаем индекс 0
leds:set(0, 1, 0, 0)
-- Если загорелся не тот, который вы ожидали ☐ поменяйте 0 на
-- другой индекс от 0 до 28
```

Это упражнение учит главному принципу инженера: **если документации недостаточно — проведи эксперимент!** В листинге функция `clear()` по индексу `0..28` выключает все светодиоды, а затем по команде `leds:set(index, ...)` загорается выбранный номер. Основной принцип — сопоставить индекс из кода с реальным светодиодом на линейке.

5.8. Модуль Ledbar

Для управления светодиодами в системе Geoskan Pioneer используется встроенный модуль Ledbar. Он предоставляет простой интерфейс (API) для работы с цветом.

Основные функции

- `Ledbar.new(N)` — конструктор. Создаёт объект для управления лентой из `N` светодиодов.
 - *Пример:* `local leds = Ledbar.new(29)`
- `leds:set(i, r, g, b)` — устанавливает цвет конкретного светодиода.
 - `i` — индекс светодиода (от 0 до `N-1`).
 - `r, g, b` — яркость каналов в диапазоне от 0.0 (выключен) до 1.0 (максимум).

⚠ Индексация

Обратите внимание: нумерация светодиодов начинается с **нуля**. Если вы создали ленту на 29 элементов, допустимые индексы — от 0 до 28. Обращение к индексу вне этого диапазона (например, к индексу 29) **не вызовет ошибку**: вызов `leds:set(...)` будет молча проигнорирован, и светодиод просто не загорится. Это коварнее явной ошибки — программа продолжит работать как ни в чём не бывало, и вы не получите никакой подсказки. Поэтому если светодиод не светится, первым делом проверьте индекс.

Также часто используется вспомогательная функция для заливки всей ленты одним цветом:

Вспомогательная функция

 `lua/examples/05_leds/leds_api_change_color.lua`

```
local function changeColor(color)
    for i = 0, ledNumber - 1 do
        leds:set(i, table.unpack(color))
    end
end
```

Пояснение. Функция `changeColor(color)` принимает таблицу

{r, g, b} и в цикле вызывает `leds:set(i, ...)` для всех индексов `0..N-1`. Основной принцип — вынести повторяющийся код установки одного и того же цвета в отдельную функцию, чтобы проще переиспользовать его в разных сценариях.

5.9. Таймеры

- `Timer.new(period, fn)` — создаёт **периодический таймер**; `period` — **период** в секундах; `fn` вызывается каждый **тик**.
- `timer:start()`, `timer:stop()` — запуск/остановка таймера.
- `Timer.callLater(delay, fn)` — единичный вызов `fn` через `delay` секунд.
- Таймеры и **состояния** делайте **глобальными** (без `local`), иначе сборщик мусора их удалит.

Пример периодического таймера (мигание):

 `lua/examples/05_leds/leds_timers_blink.lua`

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
local on = false
function updateBlink()
    local r,g,b = (on and 1 or 0), 0, 0
    for i=0, ledNumber-1 do leds:set(i, r, g, b) end
    on = not on
end
blinkTimer = Timer.new(0.5, updateBlink) -- 0.5 секунды
blinkTimer:start()

function callback(event)
end
```

Пояснение. В примере функция `updateBlink()` инвертирует флаг `on` и по таймеру `blinkTimer` раз в 0,5 секунды включает или выключает все светодиоды. Основной принцип — периодически вызывать одну и ту же функцию, чтобы реализовать мигание без блокирующих задержек. **Пример отложенного вызова (через 3 секунды):**

 `lua/examples/05_leds/leds_timers_delayed.lua`

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
Timer.callLater(3, function ()
```

```

    for i=0, ledNumber-1 do leds:set(i, 0, 0, 1) end -- включить синий
end)
function callback(event)
end

```

Пояснение. Скрипт создаёт объект `Ledbar` и с помощью `Timer.callLater` один раз через 3 секунды включает синий цвет на всех светодиодах. Основной принцип — выполнить действие с задержкой, не блокируя основной поток выполнения. **Отложенный таймер внутри обработчика событий:**

```

lua/examples/05_leds/leds_timers_callback_delayed.lua

local ledNumber = 29
local leds = Ledbar.new(ledNumber)
function callback(event)
    if (event == Ev.LOW_VOLTAGE2) then
        Timer.callLater(1, function ()
            for i=0, ledNumber-1 do leds:set(i, 1, 0, 0) end
        end)
    end
end

```

Пояснение. Здесь функция `callback(event)` реагирует на событие `Ev.LOW_VOLTAGE2` и внутри неё запускает `Timer.callLater`, который через секунду заливает линейку красным цветом. Основной принцип — сочетать обработку событий и отложенные таймеры, оставаясь в неблокирующей модели.

5.10. Событийная модель

- `function callback(event)` — обработчик системных событий.
- Примеры: `Ev.COPTER_LANDED` (посадка), `Ev.LOW_VOLTAGE2` (низкое напряжение), `Ev.POINT_REACHED`, `Ev.TAKEOFF_COMPLETE`.
- Внутри `callback` быстро реагируйте на событие; не используйте `sleep()`.

Примеры:

 lua/examples/05_leds/leds_events_landed.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
function callback(event)
    if (event == Ev.COPTER_LANDED) then
        for i=0, ledNumber-1 do leds:set(i, 0, 0, 0) end
    end
end
```

Пояснение. В этом листинге функция `callback(event)` отслеживает событие `Ev.COPTER_LANDED` и при посадке выключает все светодиоды, проходя по индексам линейки. Основной принцип — простая реакция на одно конкретное событие автопилота.

 lua/examples/05_leds/leds_events_low_voltage.lua

```
local ledNumber = 29
local leds = Ledbar.new(ledNumber)
mainTimer = Timer.new(0.1, function () end)
mainTimer:start()
function callback(event)
    if (event == Ev.LOW_VOLTAGE2) then
        mainTimer:stop()
        Timer.callLater(1, function ()
            for i=0, ledNumber-1 do leds:set(i, 1, 0, 0) end
        end)
    end
end
```

Пояснение. Во втором примере создаётся периодический таймер `mainTimer`, а `callback(event)` реагирует на `Ev.LOW_VOLTAGE2`: останавливает таймер и через `Timer.callLater` включает красный цвет на всех светодиодах. Основной принцип — объединить событийную модель и таймеры для безопасной индикации аварийного состояния.

5.11. Анализ подходов к программированию

При разработке скриптов для дрона можно использовать разные архитектурные паттерны. Рассмотрим три основных подхода.

5.11.1. 1. Блокирующее выполнение (Sleep)

Самый простой, но **не рекомендуемый** способ. Программа выполняет действие, затем «засыпает» на определенное время с помощью функции `sleep()`, затем выполняет следующее действие.

⚠ Почему `sleep` — это плохо?

На борту квадрокоптера функция `sleep()` останавливает выполнение всего потока скрипта. Это может помешать своевременной реакции на датчики, команды управления или аварийные ситуации. Используйте этот подход только для простейших тестов на земле.

5.11.2. 2. Асинхронные таймеры

Это «золотой стандарт» программирования встраиваемых систем. Вместо того чтобы ждать, мы поручаем системе вызвать нашу функцию через определённый интервал времени.

ℹ Преимущества таймеров

- **Параллельность:** можно запустить несколько независимых анимаций (например, мигание навигационных огней и перелив основной ленты).
- **Отзывчивость:** скрипт не блокируется и готов обрабатывать другие события.

Основной инструмент — `Timer.new(period, fn)`. Функция `fn` должна быть короткой и выполнять только один шаг анимации (тик).

5.11.3. 3. Конечный автомат (State Machine)

Конечный автомат — это подход для организации сложной логики. В каждый момент времени программа находится в одном из состояний (например, `IDLE`, `TAKEOFF`, `ALARM`), и может переходить в другое состояние только по явно заданным правилам.

У любой такой модели есть:

- множество состояний (режимов работы системы);

- входные события (сигналы таймеров, данные датчиков, команды пользователя);
- таблица переходов: «если сейчас состояние А и пришло событие Х — перейти в состояние В и выполнить действие Y».

На практике это обычно реализуется одной глобальной переменной `state` и функцией обработчика, которая по значению `state` и типу события выбирает, что делать дальше. Пример такого подхода — модель светофора в заданиях: цвета (красный, жёлтый, зелёный) являются состояниями, а срабатывание таймера — событием, которое переводит автомат в следующее состояние.

Этот подход особенно удобен для миссий, где поведение дрона и светодиодов зависит от внешних условий (взлёт, посадка, разряд аккумулятора, аварийные ситуации), потому что делает структуру программы прозрачной и легко расширяемой.

5.12. Примеры комплексных сценариев

```
lua/examples/05_leds/leds_basic.lua

-- создание порта управления светодиодом
local ledbar = Ledbar.new(29)

-- переменная текущего состояния
local curr_state = "START"

-- таблица функций, вызываемых в зависимости от состояния
action = {
    ["START"] = function(x)

        ledbar:set(0, 1, 0.0, 0.0)

        ledbar:set(1, 0.0, 1, 0.0)

        ledbar:set(2, 0.0, 0.0, 1)

        ledbar:set(3, 1, 1, 0.0)

        -- выключение двигателей и конец программы
        ap.push(Ev.ENGINES_DISARM)
        curr_state = "NONE"

    end,
}
```

```

-- функция обработки событий, автоматически вызывается автопилотом
function callback(event)
    if (event == Ev.TAKEOFF_COMPLETE) then
        action[curr_state]()
    end

    if (event == Ev.POINT_REACHED) then
        action[curr_state]()
    end

    if (event == Ev.COPTER_LANDED) then
        sleep(2)
        action[curr_state]()
    end
end

-- вызов функции из таблицы состояний, соответствующей первому состоянию
action[curr_state]()

```

Пояснение. В `leds_basic.lua` создаётся объект `Ledbar` и таблица `action` с единственным состоянием "START", где по индексам выставляются разные цвета и в конце подаётся команда `Ev.ENGINES_DISARM`. Функция `callback(event)` по событиям `Ev.TAKEOFF_COMPLETE`, `Ev.POINT_REACHED` и `Ev.COPTER_LANDED` вызывает соответствующее действие из таблицы состояний. Основной принцип — связать индикацию с автоматом состояний и событиями автопилота. Это уже не базовое упражнение, а пример более цельного сценария.

lua/examples/05_leds/leds_sleep.lua

```

-- создание порта управления светодиодом
local ledbar = Ledbar.new(29)

-- переменная текущего состояния
local curr_state = "START"

-- таблица функций, вызываемых в зависимости от состояния
action = {
    ["START"] = function(x)

        ledbar:set(0, 1, 0.0, 0.0)

        sleep(1000 / 1000.0)
        ledbar:set(1, 0.0, 1, 0.0)

        sleep(1000 / 1000.0)
    end
}

```

```

        ledbar:set(2, 0.0, 0.0, 1)

        sleep(1000 / 1000.0)
        ledbar:set(3, 1, 1, 0.0)

        -- выключение двигателей и конец программы
        ap.push(Ev.ENGINES_DISARM)
        curr_state = "NONE"

    end,
}

-- функция обработки событий, автоматически вызывается автопилотом
function callback(event)
    if (event == Ev.TAKEOFF_COMPLETE) then
        action[curr_state]()
    end

    if (event == Ev.POINT_REACHED) then
        action[curr_state]()
    end

    if (event == Ev.COPTER_LANDED) then
        sleep(2)
        action[curr_state]()
    end
end

-- вызов функции из таблицы состояний, соответствующей первому состоянию
action[curr_state]()

```

Пояснение. В `leds_sleep.lua` используется тот же подход с таблицей `action`, но между установками цвета вставлены вызовы `sleep()`. Основной принцип — показать, как последовательные блокирующие задержки влияют на выполнение сценария, и подготовить мотивацию для перехода к неблокирующим таймерам.

lua/examples/05_leds/leds_random_color_example.lua

```

-- https://learnxinyminutes.com/docs/ru-ru/lua-ru/ ссылка для быстрого
→ ознакомления с основами языка LUA

-- Скрипт реализует мигание светодиодов на базовой плате случайными
→ цветами

-- Упрощение вызова функции распаковки таблиц из модуля table
local unpack = table.unpack
-- Количество светодиодов на базовой плате

```

```

local ledNumber = 29
-- Создание порта управления светодиодами
local leds = Ledbar.new(ledNumber)

-- Функция, изменяющая цвет всех RGB светодиодов на базовой плате
local function changeColor(col)
    for i=0, ledNumber - 1, 1 do
        leds:set(i, unpack(col))
    end
end

-- Функция, реализующая выключение таймера и изменение цвета светодиодов
--   ↪ на красный
local function emergency()
    -- Остановка таймера
    timerRandomLED:stop()
    -- Изменение цвета светодиодов (через секунду - т.к. после остановки
    --   ↪ таймера timerRandomLED
    --   его функция выполнится еще раз)
    Timer.callLater(1, function () changeColor({1, 0, 0}) end)
end

-- Функция обработки событий, автоматически вызывается автопилотом
function callback(event)
    -- Вызов функции emergency() при низком напряжении на аккумуляторе
    if (event == Ev.LOW_VOLTAGE2) then
        emergency()
    end
end

color = {0, 0, 0}
step = 0.05
index = 1

-- Создание таймера, каждую секунду меняющего цвета всех светодиодов на
--   ↪ случайные
timerRandomLED = Timer.new(0.05, function ()
    color[index] = color[index] + step
    if color[index] > 0.95 or color[index] < 0.05 then
        if index < 3 then
            index = index + 1
        else
            index = 1
            step = -step
        end
    end
    changeColor(color)
end)

-- Запуск созданного таймера
timerRandomLED:start()

```

Пояснение. В этом примере создаётся объект `Ledbar` на 29 светодиодов и функция `changeColor(col)`, которая заливает всю линейку одним цветом. Периодический таймер `timerRandomLED` раз в 0,05 секунды плавно изменяет компоненты RGB в таблице `color` и передаёт её в `changeColor`, обеспечивая «переливание» цвета. Функция `callback(event)` отслеживает событие `Ev.LOW_VOLTAGE2` и через вспомогательную функцию `emergency()` останавливает таймер и через `Timer.callLater` включает красный цвет. Основной принцип — комбинировать плавную анимацию на таймере и аварийную индикацию по событию.

5.13. Проверка знаний

Перед выполнением практических заданий ответьте на вопросы. Вставьте пропущенную строчку кода или слово. Выберите один верный вариант и сверьте с пояснением.

1. Какое значение размера буфера должно быть передано в инициализацию?

```
leds = Ledbar.new(____)
```

- (a) 29
- (b) 8
- (c) 4
- (d) 1

Ответ: (a). Пояснение: драйвер считывает буфер из 29 элементов.

2. Чем должно быть начальное значение счётчика в цикле?

```
for i = ____, ledNumber-1 do ... end
```

- (a) 0
- (b) 1
- (c) -1
- (d) `ledNumber`

Ответ: (a). Пояснение: индексы линейки начинаются с 0 и заканчиваются 28.

3. Каким вызовом передать RGB-таблицу в `leds:set` при объявленном `local unpack = table.unpack`?

- (a) `leds:set(i, col)`
- (b) `leds:set(i, unpack(col))`
- (c) `leds:set(i, toRGB(col))`

(d) `leds:set(i, rgb(col))`

Ответ: (b). Пояснение: `unpack` распаковывает таблицу `{r,g,b}` в три аргумента.

4. Как корректно единожды включить зелёный цвет на всей линейке?

(a) `changeColor({0,1,0})`

(b) `changeColor({1,0,0})`

(c) `changeColor({0,0,1})`

(d) `changeColor({1,1,1})`

Ответ: (a). Пояснение: компоненты RGB для зелёного — `{0,1,0}`.

5. Как создать отложенный запуск синего через 3 секунды?

(a) `sleep(3); changeColor({0,0,1})`

(b) `Timer.new(3, changeColor)`

(c) `Timer.callLater(3, function() changeColor({0,0,1}) end)`

(d) `ap.push(Ev.COPTER_LANDED)`

Ответ: (c). Пояснение: `Timer.callLater` планирует единичный вызов без блокировки.

6. Какой период задать для мигания раз в полсекунды? `Timer.new(____, fn)`

(a) 0.5

(b) 500

(c) 50

(d) 1.5

Ответ: (a). Пояснение: период указывается в секундах.

7. Как инициализировать булево состояние для мигания?

(a) `on = false`

(b) `on = 0`

(c) `on = "false"`

(d) `on = nil`

Ответ: (a). Пояснение: булев тип Lua использует `true/false`.

8. Как проверить чётность индекса `i`?

(a) `i / 2 == 0`

(b) `i % 2 == 0`

(c) `i == even`

(d) `isEven(i)`

Ответ: (b). Пояснение: оператор остатка `%` определяет чётность.

9. Как остановить запущенный таймер `t`?

- (a) `t:stop()`
- (b) `t:pause()`
- (c) `stop(t)`
- (d) `t.cancel()`

Ответ: (a). Пояснение: API таймера предоставляет методы `start/stop`.

10. Как запустить таймер `t` после создания?

- (a) `t:run()`
- (b) `t:start()`
- (c) `start(t)`
- (d) `t:on()`

Ответ: (b). Пояснение: корректный метод — `start`.

11. Как называется функция-обработчик событий автопилота?

- (a) `function events(e)`
- (b) `function callback(event)`
- (c) `function onEvent(e)`
- (d) `function handler(ev)`

Ответ: (b). Пояснение: платформа вызывает `callback` и передаёт `event`.

12. Какое событие соответствует посадке?

- (a) `Ev.POINT_REACHED`
- (b) `Ev.ALTITUDE_REACHED`
- (c) `Ev.COPTER_LANDED`
- (d) `Ev.LOW_VOLTAGE2`

Ответ: (c). Пояснение: `COPTER_LANDED` — системное событие посадки.

13. Какое событие соответствует низкому напряжению?

- (a) `Ev.LOW_VOLTAGE2`
- (b) `Ev.BATTERY_OK`
- (c) `Ev.POWER_ON`
- (d) `Ev.MOTOR_ARMED`

Ответ: (a). Пояснение: используйте `LOW_VOLTAGE2` для аварийной индикации.

14. Как выглядит условие для «полосы загрузки»: включить все индексы $\leq k$?

- (a) `if i < k then ...`

(b) if $i \leq k$ then ...

(c) if $k \leq i$ then ...

(d) if $i == k$ then ...

Ответ: (b). Пояснение: включёнными остаются все индексы не выше текущего k .

15. Какой допустимый диапазон значений r, g, b в модели RGB?

(a) $0..255$

(b) $0..100$

(c) $0.0..1.0$

(d) $-1..1$

Ответ: (c). Пояснение: платформа принимает вещественные значения от 0.0 до 1.0 .

5.14. Практические задания: базовые операции

Этот блок опирается только на уже изученные основы: создание объекта `Ledbar`, модель RGB, индексацию $0..28$ и вызов `leds:set(...)`. Здесь не нужны таймеры, обработчик `callback(event)` и конечные автоматы.

Задание 2: Одновременное включение всей светодиодной линейки зелёным цветом

Требуется реализовать алгоритм, обеспечивающий однократную установку зелёного цвета с компонентами $\{0, 1, 0\}$ на всех светодиодах линейки с индексами от 0 до 28 . Для этого необходимо создать объект светодиодной линейки, реализовать функцию массовой установки цвета и выполнить её вызов.

Ожидаемый результат: вся линейка равномерно светится зелёным цветом.

Задание 3: Чередование цветов по чётности индекса

Требуется реализовать статический шаблон индикации, при котором светодиоды с чётными индексами светятся красным цветом $\{1, 0, 0\}$, а с нечётными — синим цветом $\{0, 0, 1\}$. Изображение должно устанавливаться однократно, без анимации и таймеров.

Ожидаемый результат: на линейке виден устойчивый чередующийся цветовой узор.

Задание 4: Индикатор заполнения

Необходимо реализовать однократное заполнение линейки от младших индексов к старшим: светодиоды с индексами от 0 до некоторого значения k включаются зелёным цветом $\{0, 1, 0\}$, а остальные остаются выключенными. Значение k можно задать прямо в программе.

Ожидаемый результат: на линейке отображается статическая «полоса загрузки» длиной $k+1$ светодиодов.

5.15. Практические задания: таймеры и алгоритмы

Этот блок следует выполнять после разделов про таймеры, неблокирующее выполнение и конечные автоматы. Во всех задачах ниже допустимо использовать `Timer.new(...)`, `Timer.callLater(...)` и глобальные переменные состояния.

Задание 5: Отложенное включение синего цвета

Необходимо реализовать алгоритм, при котором светодиодная линейка остаётся выключенной в течение первых трёх секунд работы программы, после чего все светодиоды одновременно загораются синим цветом $\{0, 0, 1\}$. Реализация должна использовать механизм отложенного вызова без применения блокирующих задержек.

Ожидаемый результат: через 3 секунды после запуска программы линейка полностью загорается синим цветом.

Задание 6: Периодическое мигание красным цветом

Требуется реализовать периодическое переключение всей светодиодной линейки между состояниями «включено красным цветом $\{1, 0, 0\}$ » и «выключено $\{0, 0, 0\}$ » с периодом 0,5 с. Алгоритм должен быть основан на использовании асинхронного таймера и логической переменной состояния.

Ожидаемый результат: равномерное мигание линейки с периодом 0,5 с.

Задание 7: Эффект «бегущего огня» (один активный светодиод)

Необходимо реализовать алгоритм, при котором в каждый момент времени включён только один светодиод зелёного цвета $\{0, 1, 0\}$, а остальные

выключены. Активный светодиод должен последовательно перемещаться от индекса 0 к 28 с циклическим повторением.

Ожидаемый результат: визуальный эффект перемещающегося «огонька» по всей линейке.

Задание 8: Модель светофора на основе конечного автомата

Требуется реализовать конечный автомат с тремя состояниями: красный {1,0,0}, жёлтый {1,1,0} и зелёный {0,1,0}. Состояния должны циклически сменять друг друга с заданным временным интервалом.

Ожидаемый результат: периодическая и стабильная смена цветов по схеме «красный – жёлтый – зелёный».

Задание 9: Плавная пульсация яркости красного канала (PWM-модуляция)

Необходимо реализовать алгоритм плавного увеличения и уменьшения яркости красного канала от значения 0.0 до 1.0 и обратно. Изменение яркости должно происходить малыми приращениями с использованием таймера.

Ожидаемый результат: вся светодиодная линейка плавно пульсирует красным цветом.

Задание 10: Стробоскопический режим (белые вспышки)

Требуется реализовать быстрое периодическое включение и выключение всей светодиодной линейки белым цветом {1,1,1} с малым временным интервалом.

Ожидаемый результат: частые и чёткие белые вспышки, создающие эффект стробоскопа.

Задание 11: Инвертируемая полицейская мигалка

Требуется реализовать алгоритм, при котором цвета чётных и нечётных светодиодов периодически меняются местами с заданным интервалом времени.

Ожидаемый результат: регулярное чередование цветовых паттернов, имитирующее работу полицейской световой сигнализации.

Задание 12: Генерация цветового шума

Необходимо реализовать алгоритм, который с заданной периодичностью устанавливает для каждого светодиода случайные значения компонент RGB в диапазоне от 0 до 1.

Ожидаемый результат: динамично изменяющаяся цветовая картина без видимой закономерности.

Задание 13: Индикатор загрузки (loading bar)

Требуется реализовать алгоритм постепенного заполнения светодиодной линейки от младших индексов к старшим, при котором включённые светодиоды сохраняют своё состояние. Для анимации используйте таймер, который на каждом тике увеличивает индекс последнего включённого светодиода.

Ожидаемый результат: визуальный эффект «заполняющейся» полосы.

Задание 14: Сигнал SOS по азбуке Морзе

Необходимо реализовать световую передачу сигнала «SOS» в соответствии с азбукой Морзе: три коротких импульса, три длинных импульса и снова три коротких. Длительности импульсов должны соответствовать заданным временным параметрам.

Ожидаемый результат: корректно воспроизводимый световой сигнал SOS.

Задание 15: Двоичный счётчик секунд

Требуется реализовать алгоритм отображения текущего значения секунд в двоичном виде, используя светодиоды линейки как индикаторы отдельных битов. Обновление состояния должно происходить один раз в секунду.

Ожидаемый результат: на линейке отображается двоичное представление времени в секундах.

5.16. Практические задания: реакция на события

Этот блок выполняется после раздела про `callback(event)`. Здесь важно не просто менять цвет светодиодов, а корректно реагировать на события

автопилота и не использовать блокирующие задержки.

Задание 16: Реакция на событие посадки

Требуется реализовать обработчик событий, который при получении события `Ev.COPTER_LANDED` полностью отключает светодиодную индикацию. Реализация не должна использовать блокирующие задержки.

Ожидаемый результат: после посадки все светодиоды выключены.

Задание 17: Аварийная индикация при низком напряжении

Необходимо реализовать обработку события `Ev.LOW_VOLTAGE2`, при котором все активные таймеры останавливаются, а светодиодная линейка переводится в устойчивое состояние красной индикации.

Ожидаемый результат: линейка постоянно светится красным цветом, сигнализируя об аварийном режиме.

Итоги раздела

Вы закрепили работу с буфером 29 индексов, научились строить анимации на таймерах, корректно реагировать на системные события, составлять проверяемые алгоритмы индикации. Эти приёмы будут применяться при проектировании автономных миссий.

6. Автономные беспилотные миссии

Цели раздела

- Освоить события платформы и интерфейсы автопилота.
- Построить неблокирующую архитектуру миссий (таймеры, конечный автомат).
- Реализовать маршруты по точкам и параметрические траектории.
- Настроить реакции на аварии и низкое напряжение, добавить индикацию этапов.

6.1. Введение

Автономная миссия — заранее запрограммированная последовательность действий квадрокоптера, выполняемая без участия оператора. В рамках этого раздела мы рассмотрим модель событий платформы «Геоскан Пионер», управление полётом через функции автопилота, реакцию на внешние события и построение базовых траекторий. Документ ориентирован на академический стиль и детально объясняет каждый термин, опираясь на толковый словарь и развёрнутые примеры.

6.2. Толковый словарь

Автопилот (ap) программный интерфейс управления полётом: команды взлёта, посадки, полёта к точке, обновления курса.

События (Ev.*) сигналы от платформы: TAKEOFF_COMPLETE (взлёт завершён), POINT_REACHED (точка достигнута), COPTER_LANDED (посадка), SHOCK (удар/столкновение), LOW_VOLTAGE2 (низкое напряжение), ENGINES_DISARM (выключение двигателей), MCE_* (команды режима).

Обработчик событий (callback(event)) функция, которую платформа вызывает при наступлении системного события.

Таймер (Timer) механизм неблокирующих вызовов: `Timer.new(period, fn)` — периодический, `Timer.callLater(delay, fn)` — отложенный.

Локальная система координат плоскость X–Y и высота Z относительно текущей базы; единицы — метры.

Курс (yaw) угловое направление, выражаемое в радианах; конвертация градусов: $\text{rad} = \text{deg} * \text{math.pi} / 180$.

Траектория набор точек или правило формирования положения во времени (например, окружность).

Конечный автомат разбиение логики на состояния с переходами по событиям; состояние хранится в переменной.

RC-каналы (Sensors.rc) чтение тумблеров/органов управления для запуска миссии или изменения режима.

Индикаторы (Ledbar) световая индикация состояния миссии, аварий и этапов.

6.3. Событийная модель и интерфейсы управления

- Предстартовая подготовка: `ap.push(Ev.MCE_PREFLIGHT)` — перевод автопилота в безопасный режим подготовки. Проверяются датчики, параметры и готовность; коптер не движется. Этот шаг запускают первым, чтобы корректно инициировать взлёт.
- Взлёт: `ap.push(Ev.MCE_TAKEOFF)` — команда на подъём до штатной высоты `Flight_com_takeoffAlt`. По достижению высоты платформа генерирует событие `Ev.TAKEOFF_COMPLETE`.
- Посадка: `ap.push(Ev.MCE_LANDING)` — команда на снижение и касание. По завершении посадки генерируется `Ev.COPTER_LANDED`.
- Выключение двигателей: `ap.push(Ev.ENGINES_DISARM)` — выключение моторов (обычно после `COPTER_LANDED` или при аварии).
- Полёт к точке: `ap.goToLocalPoint(x, y, z)` — установка целевой точки в локальной системе координат (метры). По достижению цели генерируется `Ev.POINT_REACHED`.
- Обновление курса: `ap.updateYaw(rad)` — установка курса в радианах; для градусов применяют перевод $\text{rad} = \text{deg} * \text{math.pi} / 180$.
- Считывание RC: `Sensors.rc()` — возвращает 8 отдельных значений каналов радиоуправления (а не таблицу), поэтому для индексации их сначала оборачивают: `local rc_chans = table.pack(Sensors.rc())`. Для тумблера SwA обычно анализируют 8-й канал (`rc_chans[8]`).
- Обработчик событий: `function callback(event)` — функция, вызываемая платформой при наступлении `Ev.*` (`TAKEOFF_COMPLETE`,

POINT_REACHED и др.). Внутри callback строятся переходы автомата миссии.

- Таймеры: `Timer.callLater(delay, fn)` — безопасная «задержка» без блокировки потока; `Timer.new(period, fn)` — периодическое действие через фиксированный интервал (анимации, траектории).
- Принципы неблокирующей логики: избегать `sleep()`; строить сценарий через события и таймеры, чтобы обработчик callback оставался кратким.

6.4. Светодиоды для отладки миссии

Светодиодная индикация — простой способ визуально подтверждать прохождение этапов миссии: изменение цвета служит отметкой «состояние автомата изменилось». Это помогает отладке без логов: по текущему цвету можно понять, выполнены ли *предстарт*, *взлёт*, *полет*, *посадка* или *аварийная реакция*. В учебных примерах далее сосредотачиваемся только на полёте; использование линейки оставляем как вспомогательный инструмент для реальных испытаний.

6.5. Справочник функций полёта

Для управления полётом используется модуль `ap` (Autopilot). Ниже приведены основные функции, необходимые для выполнения миссий.

Команды управления

- `ap.push(Ev.MCE_PREFLIGHT)` — **Обязательная команда перед стартом.** Вводит дрон в режим предстартовой подготовки: проверяет датчики, калибрует гироскопы. Двигатели не запускаются.
- `ap.push(Ev.MCE_TAKEOFF)` — Команда на взлёт. Дрон поднимается на высоту, заданную параметром `Flight_com_takeoffAlt` (обычно около 1 метра).
- `ap.push(Ev.MCE_LANDING)` — Команда на посадку. Дрон плавно снижается, выключает двигатели после касания земли.
- `ap.goToLocalPoint(x, y, z)` — Полёт в точку в локальной системе координат.
 - `x, y, z` — координаты в метрах.

- Ось X направлена вперёд, Y — влево, Z — вверх.
- Начало координат (0, 0, 0) — фиксированная точка сцены (там, где дрон изначально размещён), а не место последнего взлёта: повторный вызов `ap.goToLocalPoint(0,0,z)` всегда возвращает дрон в одну и ту же точку, даже после нескольких перелётов.
- `ap.updateYaw(rad)` — Поворот дрона вокруг своей оси (рысканье). Угол задаётся в радианах.

⚠ Аварийное выключение

`ap.push(Ev.ENGINES_DISARM)` — мгновенное выключение двигателей. Дрон упадёт камнем вниз. **Использовать только в экстренных случаях** (например, угроза столкновения с человеком) или после гарантированной посадки.

6.5.1. Таймеры и события

- `Timer.callLater(delay, fn)` — «Сделать один раз через delay секунд».
- `Timer.new(period, fn)` — «Делать каждые period секунд».
- `function callback(event)` — Функция, которую система вызывает сама, когда что-то происходит (взлёт завершён, точка достигнута, села батарейка).

6.6. Учебные ступени полёта

Предисловие. Итак, мы готовы приступить к программированию квадрокоптера. В этом разделе последовательно пройдем ключевые этапы: от безопасной предстартовой подготовки до взлёта, полёта по заданным точкам и посадки.

Любая автономная миссия дрона строится по стандартному жизненному циклу:

i Жизненный цикл миссии

1. **Инициализация:** Настройка таймеров, светодиодов, переменных.
2. **Предстарт (Preflight):** Проверка систем. Дрон стоит на земле.
3. **Взлёт (Takeoff):** Набор безопасной высоты.

4. **Маршрут (Waypoints):** Основная часть программы. Полёт по точкам.
5. **Возврат (RTL):** Возвращение в точку старта (опционально).
6. **Посадка (Landing):** Снижение и выключение двигателей.

Отдельной команды для «возврата» в API нет: этот этап — просто ещё один вызов `ap.goToLocalPoint(0, 0, z)`, возвращающий дрон в исходную точку сцены (см. §6) перед посадкой.

Каждый пример далее предваряется кратким пояснением, а строки кода снабжены комментариями, чтобы было понятно, что делает каждая команда.

Только запуск двигателей

```
lua/examples/06_missions/missions_engines_arm.lua

ap.push(Ev.MCE_PREFLIGHT)           -- безопасная предстартовая
  ↳ подготовка
Timer.callLater(1, function()       -- отложить действие на 1
  ↳ секунду
  ap.push(Ev.ENGINES_ARM)           -- команда запуска двигателей
  ↳ (без взлёта)
end)

function callback(event)
end
```

Пояснение. На практике запуск двигателей выполняется в составе взлёта (`Ev.MCE_TAKEOFF`). Явная команда `Ev.ENGINES_ARM` полезна для проверки работы моторов без взлёта.

Запуск, взлёт, посадка

```
lua/examples/06_missions/mission_takeoff_landing.lua

ap.push(Ev.MCE_PREFLIGHT)           -- предстарт
Timer.callLater(1, function()       -- через 1 секунду
  ap.push(Ev.MCE_TAKEOFF)           -- инициировать взлёт
end)

function callback(event)             -- обработчик системных
  ↳ событий
  if event == Ev.TAKEOFF_COMPLETE then -- взлёт завершён
    ap.push(Ev.MCE_LANDING)         -- выполнить посадку
  end
  if event == Ev.COPTER_LANDED then  -- посадка завершена
    ap.push(Ev.ENGINES_DISARM)      -- выключить двигатели
  end
end
```

```

end
end

```

Пояснение. В обработчике callback реагируем на завершение взлёта и инициируем посадку. После касания — выключаем двигатели.

Запуск, взлёт, полёт по прямой, посадка

```

lua/examples/06_missions/missions_straight_flight.lua

ap.push(Ev.MCE_PREFLIGHT)           -- предстарт
Timer.callLater(1, function()       -- через 1 секунду
    ap.push(Ev.MCE_TAKEOFF)          -- взлёт
end)

local unpack = table.unpack         -- сокращение доступа к
    ↳ распаковке
local target = {1.0, 0.0, 0.8}      -- целевая точка (x,y,z)

function callback(event)            -- обработчик событий
    if event == Ev.TAKEOFF_COMPLETE then -- после взлёта
        ap.goToLocalPoint(unpack(target)) -- команда полёта к точке
    end
    if event == Ev.POINT_REACHED then -- точка достигнута
        ap.push(Ev.MCE_LANDING)         -- выполнить посадку
    end
end
end

```

Пояснение. После взлёта выполняем полёт к точке (x=1.0, y=0.0, z=0.8). По событию POINT_REACHED — посадка.

Полет по фигуре (набор точек)

```

lua/examples/06_missions/missions_points_flight.lua

local unpack = table.unpack         -- сокращение для распаковки
local points = {                   -- набор точек (маршрут)
    {0.0, 0.5, 0.8}, {0.5, 0.5, 0.8}, {0.5, 0.0, 0.8}, {0.0, 0.0, 0.8}
}
local curr = 1                     -- индекс текущей точки

local function nextPoint()          -- функция перехода к
    ↳ следующей точке
    if curr <= #points then         -- если есть непройденные
        ↳ точки
        ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
        curr = curr + 1             -- увеличить индекс
    else
        ap.push(Ev.MCE_LANDING)     -- иначе посадка
    end
end

```

```

end

ap.push(Ev.MCE_PREFLIGHT)           -- предстарт
Timer.callLater(1, function()       -- через 1 секунду
    ap.push(Ev.MCE_TAKEOFF)         -- взлёт
end)

function callback(event)             -- обработчик событий
    if event == Ev.TAKEOFF_COMPLETE then nextPoint() end -- старт маршрута
    if event == Ev.POINT_REACHED then nextPoint() end   -- шаг по
    ↪ достижению точки
end

```

Пояснение. Идиоматичный «пошаговый» обход точек: каждое POINT_REACHED инициирует переход к следующей цели.

Полет по треугольнику

```

lua/examples/06_missions/missions_triangle_flight.lua

local unpack = table.unpack           -- распаковка таблицы
local r = 0.6                         -- радиус описанной окружности
local z = 0.9                         -- высота полёта
local angles = {0, 120, 240}          -- углы вершин (градусы)
local points = {}                     -- массив координат вершин
for i=1, #angles do                   -- вычисление координат
    local a = angles[i] * math.pi / 180 -- радианы
    points[i] = {r*math.cos(a), r*math.sin(a), z} -- (x,y,z) вершины
end
local curr = 1                        -- текущая вершина
local function nextPoint()            -- переход к следующей
    if curr <= #points then           -- если есть вершины
        ap.goToLocalPoint(unpack(points[curr])) -- команда полёта
        curr = curr + 1               -- увеличить индекс
    else
        ap.push(Ev.MCE_LANDING)       -- завершение ☐ посадка
    end
end
ap.push(Ev.MCE_PREFLIGHT)             -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)               -- обработчик событий
    if event == Ev.TAKEOFF_COMPLETE then nextPoint() end -- первый шаг
    if event == Ev.POINT_REACHED then nextPoint() end   -- шаг по
    ↪ событию
end

```

Пояснение. Вершины равностороннего треугольника расположены на окружности: углы 0° , 120° , 240° . Переводим градусы в радианы и вычисляем координаты по формулам $x = r \cos(a)$, $y = r \sin(a)$.

Полет по квадрату

```
lua/examples/06_missions/missions_square_flight.lua

local unpack = table.unpack          -- распаковка таблиц
local a = 1.0                        -- длина стороны квадрата
local z = 0.9                        -- высота полёта
local h = a/2                        -- половина стороны
    ⇨ (координаты ±h)
local points = {                    -- вершины квадрата по часовой
    ⇨ стрелке
    {-h, -h, z}, {h, -h, z}, {h, h, z}, {-h, h, z}
}
local curr = 1                      -- индекс текущей вершины
local function nextPoint()          -- переход к следующей вершине
    if curr <= #points then         -- если есть вершины
        ap.goToLocalPoint(unpack(points[curr])) -- полёт к вершине
        curr = curr + 1             -- увеличить индекс
    else
        ap.push(Ev.MCE_LANDING)     -- завершение ☐ посадка
    end
end
ap.push(Ev.MCE_PREFLIGHT)           -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
function callback(event)             -- обработчик событий
    if event == Ev.TAKEOFF_COMPLETE then nextPoint() end -- первый шаг
    if event == Ev.POINT_REACHED then nextPoint() end   -- шаг по
    ⇨ достижению
end
```

Пояснение. Квадрат центрирован в начале координат. Его вершины расположены в точках

$$\left(-\frac{a}{2}, -\frac{a}{2}, z\right), \left(\frac{a}{2}, -\frac{a}{2}, z\right), \left(\frac{a}{2}, \frac{a}{2}, z\right), \left(-\frac{a}{2}, \frac{a}{2}, z\right),$$

что соответствует стороне длины a и фиксированной высоте z . Центр квадрата — в начале координат $(0, 0, z)$.

Полёт по окружности с поворотом квадрокоптера

```
lua/examples/06_missions/missions_circle_yaw_timer.lua

local value = 3                    -- шаг приращения угла
    ⇨ (градусы)
local i = 0                        -- текущий угол для курса
angleT = Timer.new(0.1, function() -- таймер обновления курса
    ap.updateYaw(-i/180*math.pi)    -- курс в радианах (из
    ⇨ градусов i)
    i = i + value                  -- приращение угла курса
end)
```

```

local r = 0.3 -- радиус окружности
local angle = 0 -- текущий угол точки
  → (градусы)
local height = 0.7 -- высота полёта
pointT = Timer.new(0.1, function() -- таймер обновления точки
  angle = (angle + value) % 360 -- приращение угла точки
  local x = r*math.cos(angle * math.pi / 180) -- X по окружности
  local y = r*math.sin(angle * math.pi / 180) -- Y по окружности
  ap.goToLocalPoint(x, y, height) -- команда на точку
end)

ap.push(Ev.MCE_PREFLIGHT) -- предстарт
Timer.callLater(2, function () -- через 2 секунды
  ap.push(Ev.MCE_TAKEOFF) -- взлёт
end)

function callback(event) -- обработчик событий
  if event == Ev.TAKEOFF_COMPLETE then -- после взлёта
    angleT:start() -- запуск таймера курса
    pointT:start() -- запуск таймера точки
    Timer.callLater(6, function () -- завершить через 6 секунд
      pointT:stop() -- остановить таймер точки
      angleT:stop() -- остановить таймер курса
      ap.goToLocalPoint(0, 0, height) -- вернуться в центр
      ap.push(Ev.MCE_LANDING) -- посадка
    end)
  end
end
end

```

Пояснение. Два таймера управляют курсом и точкой окружности. Только неблокирующие обновления; завершение — по отложенному действию.

6.7. Геометрия траекторий: окружность

Для полёта по сложным траекториям часто используется параметрическое задание координат. Самый простой пример — окружность.

Окружность радиуса r в плоскости $X-Y$ задаётся формулами:

$$x(\theta) = r \cdot \cos(\theta), \quad y(\theta) = r \cdot \sin(\theta), \quad z = \text{const.}$$

Здесь θ (тета) — текущий угол в радианах.

Градусы и Радианы

В математике и программировании тригонометрические функции ($\sin$, $\cos$) обычно принимают аргумент в **радианах**. Если вы привыкли мыслить градусами ($0 \dots 360$), используйте формулу перевода:

$$\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \frac{\pi}{180}$$

Или встроенную функцию Lua: `math.rad(degrees)`.

Реализация на Lua

 lua/examples/06_missions/missions_geometry_circle_calc.lua

```
local angle = 0          -- угол в градусах
local r = 1.0            -- радиус 1 метр

-- Внутри таймера:
angle = angle + 5        -- шаг 5 градусов
local rad = math.rad(angle)
local x = r * math.cos(rad)
local y = r * math.sin(rad)
ap.goToLocalPoint(x, y, 1.0)
```

6.8. Архитектурные подходы

При создании сложных автономных миссий важно соблюдать правильную архитектуру программы. Это обеспечивает надёжность и предсказуемость поведения дрона.

Ключевые принципы

- 1. Событийно-ориентированное программирование:** Логика переходов строится внутри функции `callback(event)`. Дрон сам сообщает нам, когда он завершил предыдущее действие (например, `Ev.POINT_REACHED`). Не пытайтесь угадать время полёта с помощью таймеров — используйте события.
- 2. Асинхронность:** Никогда не используйте `sleep()` или циклы `while` для ожидания. Это заблокирует автопилот. Используйте `Timer.callLater` и `Timer.new`.
- 3. Безопасность (Failsafe):** Всегда обрабатывайте аварийные события:

- Ev.SHOCK (Удар) — немедленно выключайте двигатели (DISARM).
- Ev.LOW_VOLTAGE2 (Критический разряд) — выполняйте посадку.

6.8.1. Пример структуры программы

Типовая программа состоит из:

1. Объявления переменных и настроек.
2. Функций-обработчиков для каждого этапа полёта (взлёт, полёт по точкам, посадка).
3. Таблицы переходов (State Machine).
4. Функции callback, которая переключает состояния.

6.9. Разбор примеров миссий

Урок 12: окружность по точкам

```
lua/examples/06_missions/missions_lesson12_circle_points.lua

local ledNumber = 29 -- количество видимых диодов
    ↳ на плате
local leds = Ledbar.new(ledNumber) -- объект управления
    ↳ светодиодами
local unpack = table.unpack -- сокращение для распаковки
    ↳ таблиц
local curr_state = "PREPARE_FLIGHT" -- начальное состояние
    ↳ автомата

local function changeColor(color) -- функция смены цвета всех
    ↳ диодов
    for i=0, ledNumber - 1 do -- проход по индексам
        ↳ 0..(ledNumber-1)
        leds:set(i, unpack(color)) -- установка RGB для индекса
        ↳ i
    end
end

local colors = { -- палитра используемых
    ↳ цветов
    {1,0,0}, {1,1,1}, {0,1,0}, {1,1,0},
    {1,0,1}, {0,0,1}, {0,0,0}
}

local r = 0.5 -- радиус окружности
local angle = 30 -- угловой шаг (градусы)
local points = {} -- массив точек окружности
```

```

for i=1, 12 do                                     -- генерация 12 равномерных
    ↪ точек
    local a = i * angle
    local xCoord = r * math.cos(a * math.pi / 180) -- X координата
    local yCoord = r * math.sin(a * math.pi / 180) -- Y координата
    local yaw = (a + 90) * math.pi / 180           -- курс
    ↪ (тангенциальный)
    points[i] = {xCoord, yCoord, 0.8, yaw}          -- точка с высотой и
    ↪ курсом
end

local j = 1                                         -- индекс текущей точки
action = {                                          -- таблица функций по
    ↪ состояниям
    ["PREPARE_FLIGHT"] = function()                -- подготовка полёта
        changeColor(colors[2])                     -- белый ◻ подготовка
        Timer.callLater(2, function () ap.push(Ev.MCE_PREFLIGHT) end) --
        ↪ предстарт
        Timer.callLater(4, function () changeColor(colors[3]) end) --
        ↪ зелёный ◻ готовность
        Timer.callLater(6, function ()              -- через 6 секунд
            ap.push(Ev.MCE_TAKEOFF)                  -- взлёт
            curr_state = "FLIGHT"                    -- переход к полёту
        end)
    end,
    ["FLIGHT"] = function ()                        -- состояние полёта
        if j <= #points then                         -- если есть следующая точка
            local x, y, z, yaw = table.unpack(points[j])
            ap.updateYaw(yaw)                         -- сначала поворачиваемся
            ap.goToLocalPoint(x, y, z)                 -- затем летим
            j = j + 1                                 -- увеличить индекс
            curr_state = "FLIGHT"                     -- остаться в состоянии для
            ↪ следующего POINT_REACHED
        else
            curr_state = "PIONEER_LANDING"            -- переход к посадке
            action[curr_state]()                      -- запуск логики посадки
        end
    end,
    ["PIONEER_LANDING"] = function ()               -- состояние посадки
        changeColor(colors[6])                       -- синий ◻ посадка
        Timer.callLater(2, function () ap.push(Ev.MCE_LANDING) end) --
        ↪ команда посадки
    end
}

function callback(event)                           -- обработчик системных
    ↪ событий
    if (event == Ev.TAKEOFF_COMPLETE) then action[curr_state]() end --
    ↪ старт логики
    if (event == Ev.SHOCK) then                      -- авария (столкновение)
        changeColor(colors[1])                      -- красный цвет
    end
end

```

```

    ap.push(Ev.ENGINES_DISARM)                -- выключение двигателей
    → (требуется ENGINES_DISARM)
end
if (event == Ev.POINT_REACHED) then action[curr_state]() end --
→ переход по достижению точки
if (event == Ev.COPTER_LANDED) then changeColor(colors[7]) end --
→ выключить индикацию
end

changeColor(colors[1])                        -- начальная индикация
→ (красный)
Timer.callLater(2, function () action[curr_state]() end) -- запуск
→ автомата через 2с

```

Комментарий. Генерация 12 точек окружности, переходы по событиям. Рекомендуется заменить while/break на явный шаг через таймер или по событию POINT_REACHED.

Урок 14: непрерывная окружность и курс

 lua/examples/06_missions/missions_lesson14_circle.lua

```

local ledNumber = 29                        -- количество диодов
local leds = Ledbar.new(ledNumber)          -- объект управления линейкой
local unpack = table.unpack                -- распаковка таблиц
local curr_state = "PREPARE_FLIGHT"         -- начальное состояние

local function changeColor(color)           -- смена цвета всех диодов
    for i=0, ledNumber - 1 do              -- индексы 0..(ledNumber-1)
        leds:set(i, unpack(color))         -- установить RGB
    end
end

local colors = {                           -- палитра
    {1,0,0}, {1,1,1}, {0,1,0}, {1,1,0},
    {1,0,1}, {0,0,1}, {0,0,0}
}

local value = 3                             -- шаг увеличения угла
→ (градусы)
local angle = 0                             -- текущий угол точки
→ (градусы)
local r = 0.5                               -- радиус окружности (метры)
local height = 0.8                         -- высота полёта (метры)

pointT = Timer.new(0.1, function()         -- таймер движения по
→ траектории
    angle = angle + value                  -- приращение угла
    if angle > 360 then angle = angle - 360 end -- нормализация

```

```

local xCord = r * math.cos(angle * math.pi / 180) -- расчёт X
local yCord = r * math.sin(angle * math.pi / 180) -- расчёт Y

-- Для полёта "головой вперед" курс должен быть касательным к
--   ↪ окружности.
-- Касательный угол  $\psi$  = angle + 90 градусов.
local yaw = (angle + 90) * math.pi / 180

ap.updateYaw(yaw) -- установка курса
ap.goToLocalPoint(xCord, yCord, height) -- команда полёта в точку
end)

action = { -- таблица состояний
  ["PREPARE_FLIGHT"] = function(x) -- подготовка полёта
    changeColor(colors[2]) -- белый
    Timer.callLater(2, function () ap.push(Ev.MCE_PREFLIGHT) end) --
    ↪ предстарт
    Timer.callLater(4, function () changeColor(colors[3]) end) --
    ↪ зелёный
    Timer.callLater(6, function () -- через 6 секунд
      ap.push(Ev.MCE_TAKEOFF) -- взлёт
      ap.goToLocalPoint(0,0,height) -- выход в центр
      curr_state = "FLIGHT_TO_CIRCLE" -- переход к полёту
    end)
  end,
  ["FLIGHT_TO_CIRCLE"] = function (x) -- выход на начальную точку
    ↪ круга
    changeColor(colors[4]) -- жёлтый
    local startX = r * math.cos(0)
    local startY = r * math.sin(0)
    ap.goToLocalPoint(startX, startY, height) -- летим в (r, 0)
    curr_state = "CIRCLE_FLIGHT" -- ждем POINT_REACHED
  end,
  ["CIRCLE_FLIGHT"] = function (x) -- полёт по кругу
    changeColor(colors[5]) -- фиолетовый
    curr_state = "FLYING_CIRCLE" -- меняем состояние, чтобы не
    ↪ реагировать на POINT_REACHED
    pointT:start() -- запуск таймера траектории
    Timer.callLater(12, function () -- летим один полный круг
      ↪ (360/3 * 0.1 = 12с)
      curr_state = "PIONEER_LANDING" -- переход к посадке
      action[curr_state]() -- запуск логики посадки
    end)
  end,
  ["FLYING_CIRCLE"] = function (x) -- пустое состояние, чтобы
    ↪ игнорировать POINT_REACHED
  end,
  ["PIONEER_LANDING"] = function (x) -- состояние посадки
    changeColor(colors[2]) -- белый
    pointT:stop() -- остановить таймер точки
    ap.goToLocalPoint(0, 0, height) -- возврат в центр
  end
}

```

```

    Timer.callLater(2, function ()
        ap.push(Ev.MCE_LANDING)
    end)
end
}

function callback(event)
    -- обработчик системных
    -- событий
    if (event == Ev.TAKEOFF_COMPLETE) then action[curr_state]() end --
    -- запуск логики
    if (event == Ev.SHOCK) then
        changeColor(colors[1])
        pointT:stop()
    end
    -- авария
    -- красный
    -- останов таймера траектории
    if (event == Ev.POINT_REACHED) then action[curr_state]() end -- шаг
    -- логики
    if (event == Ev.COPTER_LANDED) then changeColor(colors[7]) end --
    -- выключение индикации
end

changeColor(colors[1])
Timer.callLater(2, function () action[curr_state]() end) --
-- начальный красный цвет
-- старт
-- автомата через 2с

```

Комментарий. Реализована непрерывная подача команды на полёт по окружности и обновление курса.

RC-инициация миссии

```

lua/examples/06_missions/missions_rc_initiation.lua

local unpack = table.unpack
local rc = Sensors.rc
local points = {{0,2,1},{0,1,1},{0,2,1},{0,1,1}}
local curr_point = 1

local function nextPoint()
    if(curr_point <= #points) then
        Timer.callLater(1, function()
            ap.goToLocalPoint(unpack(points[curr_point]))
            curr_point = curr_point + 1
        end)
    else
        Timer.callLater(1, function() ap.push(Ev.MCE_LANDING) end)
    end
end

function callback(event)

```

```

    if(event == Ev.TAKEOFF_COMPLETE) then Timer.callLater(5, nextPoint)
    ↪ end -- старт
    if(event == Ev.POINT_REACHED) then Timer.callLater(5, nextPoint) end
    ↪ -- шаг
end

startTimer = Timer.new(1, function()
    rc_chans = table.pack(rc())
    if rc_chans[8] > 0 then
        curr_point = 1
        ap.push(Ev.MCE_PREFLIGHT)
        Timer.callLater(6, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт
    end
end)
startTimer:start()

```

-- опрос RC каждую секунду
-- считанные каналы
-- тумблер включён
-- сброс маршрута
-- предстарт
-- взлёт
-- запуск опроса RC

Комментарий. Запуск миссии по положению тумблера, отложенные переходы по событиям. Рекомендуется добавлять аварийные реакции на Ev.SHOCK и Ev.LOW_VOLTAGE2.

Локальные точки и индикация

 lua/examples/06_missions/missions_local_points_leds.lua

```

local boardNumber = boardNumber
↪ используется
local unpack = table.unpack
local points = {
    {-0.6, 0.3, 0.2}, {0.6, 0.3, 0.2}, {0, 0, 0.5}, {0.6, -0.3, 0.2}
}
local curr_point = 1

local function nextPoint()
    if(#points >= curr_point) then
        ap.goToLocalPoint(unpack(points[curr_point])) -- полёт к точке
        curr_point = curr_point + 1
    else
        ap.push(Ev.MCE_LANDING)
    end
end

function callback(event)
    if(event == Ev.TAKEOFF_COMPLETE) then nextPoint() end
    ↪ маршрута
    if(event == Ev.POINT_REACHED) then nextPoint() end
    ↪ событию
end

local leds = Ledbar.new(29)
local blink = 0

```

-- номер борта (если используется)
-- распаковка таблиц
-- заранее заданные точки
-- текущая точка
-- переход к следующей точке
-- есть непройденные точки
-- полёт к точке
-- увеличить индекс
-- завершение  посадка
-- обработчик событий
-- старт
-- шаг по
-- объект линейки (буфер 29)
-- состояние мигания

```

for i=0,3 do leds:set(i,1,1,1) end          -- начальный белый цвет
timerBlink = Timer.new(1, function ()      -- таймер мигания, период 1с
    blink = (blink == 1) and 0 or 1        -- переключение состояния
    for i=0,3 do leds:set(i, blink, 0, 0) end -- мигание красным
end)
timerBlink:start()                         -- запуск мигания
ap.push(Ev.MCE_PREFLIGHT)                  -- предстарт
Timer.callLater(1, function() ap.push(Ev.MCE_TAKEOFF) end) -- взлёт

```

Комментарий. Для корректной индикации используйте буфер размером 29, полный проход по индексам 0..28.

6.10. Рекомендации по разработке миссий

- Делайте таймеры и состояния глобальными; избегайте локальных объектов в короткоживущих функциях.
- Исключайте блокировки (`sleep()`); опирайтесь на `Timer` и событийную модель.
- Чётко определяйте состояния автомата; держите обработчики событий короткими.
- Реализуйте аварийные реакции и визуальную индикацию этапов.
- Работайте с курсом φ в радианах; переводите градусы по формуле $\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \pi / 180$.

6.11. Проверка знаний

1. Как инициировать предстартовую подготовку? `ap.push(____)`

- (a) `Ev.MCE_PREFLIGHT`
- (b) `Ev.PREFLIGHT`
- (c) `Ev.START`
- (d) `Ev.PRE`

Ответ: (a). Пояснение: используется пространство имён `Ev.MCE_*`.

2. Чем реагировать на завершение взлёта?

- (a) `callback(Ev.TAKEOFF_COMPLETE)`
- (b) `function callback(event) if event==Ev.TAKEOFF_COMPLETE then ... end end`
- (c) `ap.onTakeoff()`

(d) `Timer.new(5, ...)`

Ответ: (b). Пояснение: события обрабатываются внутри `callback`.

3. Как задать полёт к точке?

(a) `ap.goToLocalPoint(x,y,z)`

(b) `ap.fly(x,y,z)`

(c) `ap.moveTo(x,y,z)`

(d) `ap.goto(x,y,z)`

Ответ: (a). Пояснение: корректный API платформы.

4. Как обновить курс по градусам `deg`?

(a) `ap.updateYaw(deg)`

(b) `ap.updateYaw(deg*math.pi/180)`

(c) `ap.updateYaw(deg/180)`

(d) `ap.updateYaw(math.pi*deg)`

Ответ: (b). Пояснение: `updateYaw` принимает радианы.

5. Как безопасно выполнить действие через 2 секунды?

(a) `sleep(2); action()`

(b) `Timer.callLater(2, action)`

(c) `Timer.new(2, action)`

(d) `ap.delay(2, action)`

Ответ: (b). Пояснение: отложенный неблокирующий вызов.

6. Как задать таблицу точек окружности?

(a) `cos(i), sin(i)`

(b) `r*math.cos(...), r*math.sin(...)`

(c) `r*cos(...), r*sin(...)`

(d) `x=i, y=i`

Ответ: (b). Пояснение: вычисление по параметрическим формулам.

7. Как корректно выключить двигатели?

(a) `ap.push(ENGINES_DISARM)`

(b) `ap.push(Ev.ENGINES_DISARM)`

(c) `ap.disarm()`

(d) `ap.push(Ev.MCE_STOP)`

Ответ: (b). Пояснение: требуется пространство имён `Ev`.

8. Как реагировать на удар/столкновение?

(a) `event==Ev.SHOCK`

- (b) `event==Ev.COLLISION`
- (c) `event==Ev.CRASH`
- (d) `event==Ev.HIT`

Ответ: (a). Пояснение: используйте `Ev.SHOCK`.

9. Как организовать непрерывное обновление точки окружности?

- (a) `while ... do ap.goToLocalPoint(...) end`
- (b) `Timer.new(period, function() ap.goToLocalPoint(...) end)`
- (c) `repeat ap.goToLocalPoint(...) until false`
- (d) `sleep(0.1); ap.goToLocalPoint(...)`

Ответ: (b). Пояснение: периодический таймер без блокировок.

10. Как прочитать RC-каналы?

- (a) `Sensors.rc()`
- (b) `ap.rc()`
- (c) `rc.read()`
- (d) `Sensors.readRC()`

Ответ: (a). Пояснение: API сенсоров.

11. Как проверить положение тумблера SwA в `rc_chans`?

- (a) `rc_chans[1]`
- (b) `rc_chans[8]`
- (c) `rc_chans["SwA"]`
- (d) `rc_chans[0]`

Ответ: (b). Пояснение: восьмой канал.

12. Как построить переходы состояний?

- (a) `if ... then ...`
- (b) таблица функций `action[state]()`
- (c) `switch(state)`
- (d) `goto state`

Ответ: (b). Пояснение: идиоматичный подход через таблицу функций.

13. Как задать высоту полёта по окружности?

- (a) `ap.setHeight(z)`
- (b) `ap.goToLocalPoint(x,y,z)`
- (c) `ap.updateYaw(z)`
- (d) `Sensors.altimeter(z)`

Ответ: (b). Пояснение: высота `z` задаётся в целевой точке.

14. Как обеспечить аварийную индикацию при низком напряжении?

- (a) в callback по `Ev.LOW_VOLTAGE2` остановить таймеры и включить красный
- (b) задержка через `sleep`
- (c) увеличить `height`
- (d) проигнорировать событие

Ответ: (a). Пояснение: минимизация нагрузки и чёткая реакция.

6.12. Задания для лабораторной работы**6.12.1. Базовые миссии****Задание 18: Полёт по окружности радиуса $r=0.5$ на высоте $z=1.0$**

Требуется реализовать алгоритм, обеспечивающий устойчивый полёт по окружности:

- выполнить `ap.push(Ev.MCE_PREFLIGHT)` и `ap.push(Ev.MCE_TAKEOFF)`;
- запускать обновление точки через `Timer.new(0.1, fn)` после `Ev.TAKEOFF_COMPLETE`;
- вычислять $x = r \cos(\theta)$, $y = r \sin(\theta)$ с переводом градусов в радианы;
- при необходимости сопровождать курс вызовом `ap.updateYaw(angle * math.pi / 180)`;
- использовать индикацию `Ledbar.new(29)` для этапов;
- по `Ev.SHOCK` — останов таймеров и `ap.push(Ev.ENGINES_DISARM)`, по `Ev.LOW_VOLTAGE2` — посадку.

Ожидаемый результат: квадрокоптер следует окружности радиуса 0.5 на высоте 1.0 , выполняет посадку, индикация отражает этапы.

Задание 19: Полёт по многоугольнику из N точек окружности

Требуется реализовать генерацию точек по равномерным углам $a = 360 \cdot i / N$:

- переводить углы в радианы при вычислении координат;
- стартовать маршрут по `Ev.TAKEOFF_COMPLETE`;
- переходить к следующей точке по `Ev.POINT_REACHED`;
- обеспечить безопасные реакции и индикацию как в окружности.

Ожидаемый результат: последовательный обход всех N точек и посадка после завершения маршрута.

Задание 20: Траектория «восьмёрка»

Требуется реализовать два состояния (FIRST, SECOND) для обхода двух окружностей с противоположным направлением:

- сместить центры по оси X ;
- выбрать шаг угла разного знака для фаз;
- завершить миссию возвратом в $(0, 0, z)$ и посадкой.

Ожидаемый результат: полёт по «восьмёрке» без рывков, завершение в центре и посадка.

Задание 21: Спираль подъёма

Требуется реализовать параметрическую спираль с приращениями $r = r + dr$ и $z = z + dz$ на каждом тике; по достижении z_max остановить таймер, перейти в безопасную точку базы и выполнить посадку.

Ожидаемый результат: плавный подъём до z_max , возврат к базе и посадка.

Задание 22: Сопровождение курса yaw

Требуется синхронизировать `ar.updateYaw` с параметром окружности, использовать радианы; ограничить шаг изменения угла разумным значением, чтобы исключить резкие скачки.

Ожидаемый результат: курс следует траектории без резких изменений, ориентация корректна.

Задание 23: Запуск и останов миссии RC-тумблером

Требуется считывать `Sensors.rc()`, анализировать `rc_chans[8]`:

- при включении тумблера выполнять предстарт и взлёт;
- при выключении — останавливаться, переходить в $(0, 0, z)$ и выполнять посадку;
- опрашивать RC через периодический `Timer.new(0.5, fn)`.

Ожидаемый результат: надёжный старт/останов миссии по положению тумблера с безопасным завершением.

Задание 24: Аварийная реакция на Ev.SHOCK

Требуется немедленно остановить все таймеры миссии, включить красный цвет на всех 0..28 индексах линейки и выполнить `ар.push(Ev.ENGINES_DISARM)`.

Ожидаемый результат: мгновенное выключение двигателей и красная индикация аварии.

Задание 25: Посадка при низком напряжении Ev.LOW_VOLTAGE2

Требуется остановить миссию, перейти в безопасную точку и выполнить `ар.push(Ev.MCE_LANDING)`, включить красную индикацию низкого напряжения.

Ожидаемый результат: безопасная посадка при низком напряжении, отчёт индикацией.

Задание 26: Обход квадрата по точкам, два цикла

Требуется задать вершины $(-a/2, -a/2, z)$, $(a/2, -a/2, z)$, $(a/2, a/2, z)$, $(-a/2, a/2, z)$, выполнять переходы по `Ev.POINT_REACHED` и совершить два полных обхода перед посадкой.

Ожидаемый результат: два полных обхода квадрата и посадка в конце маршрута.

Задание 27: «Лоитер» по малой окружности $r=0.2$

Требуется периодически обновлять точку окружности таймером, обеспечить безопасную остановку по событию или RC, сопровождать миссию световой индикацией этапов.

Ожидаемый результат: стабильное удержание по малой окружности и корректная остановка по команде.

6.12.2. Продвинутое задание

Задание 28: Плавная смена цвета этапов миссии

Требуется сопоставить цвета состояниям миссии (подготовка / полёт / посадка / ожидание):

- обновлять все индексы $0 \dots 28$ без блокировок `sleep()`;
- реализовать смену через функции стадий и отложенные вызовы.

Ожидаемый результат: плавная индикация переходов между этапами без пропусков и блокировок.

Задание 29: Динамический радиус окружности по RC7

Требуется считывать `Sensors.rc()` и управлять радиусом:

- нормировать значение канала к $[0, 1]$;
- вычислять радиус $r = 0.2 + 0.8 \cdot \text{norm}$;
- периодически обновлять точку окружности.

Ожидаемый результат: радиус плавно меняется от 0.2 до 1.0 по RC7, траектория стабильна.

Задание 30: Ограничение скорости изменения угла `angle`

Требуется реализовать `advanceAngle(step)`:

- с ограничением `step` диапазоном $[-\text{maxStep}, \text{maxStep}]$;
- применять его при каждом обновлении параметра окружности.

Ожидаемый результат: угол изменяется с ограниченной скоростью, резкие скачки отсутствуют.

Задание 31: Перезапуск миссии после `Ev.COPTER_LANDED`

Требуется по событию выполнить `Timer.callLater(3, fn)` и инициировать новый взлёт, исключая блокирующие задержки.

Ожидаемый результат: автоматический повтор миссии после посадки с паузой, без блокировок.

Задание 32: «Морзянка» этапов на линейке

Требуется реализовать мигание заданное число раз для каждого этапа, используя короткие функции и отложенные вызовы.

Ожидаемый результат: корректный световой отчет по морзянке для каждого этапа.

Задание 33: Траектория «лепесток»

Требуется реализовать формулу $r = r_0 \cdot (1 + 0.2 \sin(k \cdot \theta))$:

- при $\theta = \text{angle} \cdot \pi / 180$;
- выбрать $k \geq 3$ и обновлять точку параметрически.

Ожидаемый результат: визуально различимый «лепесток» по параметрической траектории.

Задание 34: Проверка точности навигации

Требуется обход 8 точек окружности, измерение интервалов dt между `Ev.POINT_REACHED` с использованием `os.clock()`.

Ожидаемый результат: получены интервалы времени, оценено постоянство периода между достижениями точек.

Задание 35: Миссия «периметр» вокруг базы

Требуется определить набор точек по квадрату вокруг $(0, 0)$, включить возврат в исходную точку и выполнить посадку.

Ожидаемый результат: полный обход периметра и посадка в исходной точке.

Задание 36: «Окно безопасности» для yaw

Требуется ограничить изменение yaw на так не более $\text{maxYawRate} \cdot dt$ и обновлять точку только при соблюдении ограничения.

Ожидаемый результат: курс изменяется не быстрее установленной скорости, миссия сохраняет устойчивость.

Задание 37: Демонстрационный сценарий (RC старт, ≈ 2 минуты, посадка)

Требуется запуск по RC, полёт по окружности около двух минут, посадка, световой отчёт этапов, отсутствие блокировок и проверка безопасного завершения.

Ожидаемый результат: демонстрационный полёт по окружности, корректная посадка и полная индикация этапов.

Итоги раздела

Вы научились использовать события платформы, интерфейсы ар.*, таймеры и индикаторы для построения автономных миссий: от окружности и многоугольника до RC-управления и аварийных реакций. Практические задания позволяют закрепить навыки и подготовить собственные сценарии полётов.

Часть II.

Программирование на Python

7. Программирование на Python

Цель главы

В этой главе мы познакомимся с языком программирования Python, научимся писать простые программы, работать с данными и управлять логикой выполнения. Эти знания станут фундаментом для программирования автономных миссий квадрокоптера.

7.1. Что такое Python и зачем он нужен?

Python — это высокоуровневый язык программирования, который был создан Гвидо ван Россумом и впервые выпущен в 1991 году. Назван он не в честь змеи, а в честь популярного британского комедийного шоу «Летающий цирк Монти Пайтона».

Почему Python?

- **Простота:** Синтаксис Python похож на обычный английский язык.
- **Универсальность:** Используется в веб-разработке, науке, AI и робототехнике.
- **Мощь:** Огромное количество библиотек (OpenCV, NumPy, TensorFlow) для решения сложных задач.

Для программирования квадрокоптеров Pioneer мы выбрали Python, так как он позволяет легко взаимодействовать с оборудованием и обрабатывать видеопоток с камеры.

Сравнение Python и Lua

Мы уже знакомы с языком Lua, который используется для написания скриптов автопилота. Давайте посмотрим, чем Python отличается от Lua:

Характеристика	Lua	Python
Назначение	Встраиваемый скрип- товый язык (для игр, контроллеров)	Универсальный язык об- щего назначения
Синтаксис	Использует <code>end</code> для блоков кода	Использует отступы для блоков кода
Индексация	С 1 (первый элемент — 1)	С 0 (первый элемент — 0)
Библиотеки	Ограниченный набор	Огромная экосистема (PyPI)
Скорость	Очень быстрый	Медленнее, но мощнее

Таблица 1: Сравнение Lua и Python

i Важное отличие

Главное, что нужно запомнить: в Python блоки кода (циклы, функции, условия) выделяются **отступами** (обычно 4 пробела), а не ключевыми словами `do ... end`.

Первая программа

Традиционно изучение любого языка начинается с программы, которая выводит приветствие на экран.

 `python/examples/01_python_basics/01_hello_world.py`

```
# Первая программа на Python
print("Hello, World!")
```

Здесь функция `print()` выводит текст, заключенный в кавычки, в консоль.

Задание 38: Привет, мир!

Напишите программу, которая выводит ваше имя и фамилию на экран.

7.2. Переменные и типы данных

Переменная — это именованная область памяти, в которой хранится значение. В Python не нужно заранее объявлять тип переменной, он определяется автоматически при присваивании.

Основные типы данных

- `int` (integer) — целые числа (например, 10, -5).
- `float` — числа с плавающей точкой (например, 3.14, 1.5).
- `str` (string) — строки текста (например, "Pioneer").
- `bool` (boolean) — логические значения (True, False).

 `python/examples/01_python_basics/02_variables.py`

```
# Переменные и типы данных
name = "Pioneer" # Строка (str)
altitude = 1.5    # Число с плавающей точкой (float)
battery = 100     # Целое число (int)
is_flying = False # Логический тип (bool)

print("Дрон:", name)
print("Высота:", altitude)
print("Тип переменной altitude:", type(altitude))
```

Функция `type()` позволяет узнать тип переменной.

Задание 39: Личная карточка

Создайте переменные для хранения вашего имени, возраста и любимого предмета. Выведите их на экран одной строкой.

Задачи для самостоятельного решения

1. Создайте переменную `speed` и присвойте ей значение 5. Выведите её на экран.
2. Создайте переменную `height` и присвойте ей значение 1.5. Выведите её.
3. Создайте переменную `name` со значением "Pioneer". Выведите её.
4. Создайте переменную `score = 10`. Затем измените её значение на 20. Выведите `score`.
5. Создайте переменную `x = 100`. Затем присвойте ей строку "Сто". Выведите тип переменной до и после изменения.

6. Даны $a = 5$ и $b = 7$. Создайте переменную c , равную их сумме. Выведите c .
7. Даны $part1 = "Geo"$ и $part2 = "skan"$. Создайте $full$, сложив их.
8. Создайте строку $s = "Fly"$. Выведите её 3 раза подряд (используйте умножение).
9. Какого типа переменная $pi = 3.1415$? Выведите результат функции $type(pi)$.
10. Присвойте переменным x, y, z значения 1, 2, 3 в одной строке.
11. Даны $a = 10, b = 20$. Поменяйте их значения местами без использования третьей переменной.
12. Создайте переменную is_flying со значением $True$. Выведите её.
13. Дано целое число $n = 5$. Преобразуйте его в дробное и сохраните в f .
14. Дана строка $s_num = "123"$. Преобразуйте её в число num и прибавьте 1.
15. Даны $mass = 10$ и $accel = 9.8$. Вычислите $force = mass * accel$.

7.3. Арифметические операторы

Python поддерживает все стандартные математические операции.

- + Сложение
- - Вычитание
- * Умножение
- / Деление (всегда возвращает float)
- // Целочисленное деление (отбрасывает дробную часть)
- % Остаток от деления
- ** Возведение в степень

 python/examples/01_python_basics/03_arithmetic.py

```
# Арифметические операции
x = 10
y = 3

print("Сложение:", x + y)
print("Деление:", x / y)
print("Целочисленное деление:", x // y) # 3
print("Остаток от деления:", x % y)    # 1
print("Возведение в степень:", x ** y) # 1000
```

Обратите внимание на приоритет операций: умножение и деление вы-

полняются раньше сложения и вычитания, как в математике. Скобки () используются для изменения порядка действий.

Задание 40: Калькулятор

Напишите программу, которая считает площадь прямоугольника со сторонами 5 и 8 метров.

Задачи для самостоятельного решения

1. Сложите числа 15 и 38. Выведите результат.
2. Вычтите из 100 число 45. Выведите результат.
3. Умножьте 7 на 8.
4. Разделите 50 на 4. Какой тип данных получился?
5. Сколько раз число 4 помещается в 50 целиком? Используйте //.
6. Чему равен остаток от деления 20 на 3? Используйте %.
7. Возведите 2 в степень 10 (2 в 10-й степени).
8. Вычислите $2 + 2 * 2$. Затем $(2 + 2) * 2$. Сравните результаты.
9. Сторона квадрата $a = 5$. Найдите периметр $P = 4 * a$.
10. Даны стороны $a = 10$, $b = 20$. Найдите площадь $S = a * b$.
11. Сколько секунд в одном часе? ($60 \text{ секунд} * 60 \text{ минут}$).
12. Даны числа 10, 20, 30. Найдите их среднее арифметическое.
13. Дано число 12345. Найдите его последнюю цифру, используя оператор % 10.
14. Дано четное число $n = 10$. Получите следующее четное число (+2).
15. Умножьте -5 на -5. Проверьте знак результата.

7.4. Логические операции

Логические операторы позволяют комбинировать условия. Результатом всегда является True (Истина) или False (Ложь).

Операторы сравнения

- == Равно
- != Не равно
- > Больше

- < Меньше
- >= Больше или равно
- <= Меньше или равно

Логические связки

- and (И) — истина, только если оба условия истинны.
- or (ИЛИ) — истина, если хотя бы одно условие истинно.
- not (НЕ) — инвертирует значение (делает истину ложью и наоборот).

```
python/examples/01_python_basics/04_logic.py

# Логические операции
voltage = 3.8
is_connected = True

# Проверка условий
is_low_battery = voltage < 3.5
ready_to_fly = is_connected and not is_low_battery

print("Низкий заряд?", is_low_battery)
print("Готов к полету?", ready_to_fly)
```

Задание 41: Проверка безопасности

Создайте переменную `wind_speed = 7`. Напишите логическое выражение, которое возвращает `True`, если скорость ветра меньше 5 м/с.

Задачи для самостоятельного решения

1. Проверьте, больше ли 10 числа 5.
2. Проверьте, меньше или равно 5 числу 5.
3. Равны ли числа 100 и 100?
4. Правда ли, что 10 не равно 20?
5. Проверьте условие: $(5 > 0)$ и $(5 < 10)$.
6. Проверьте условие: $(5 > 0)$ или $(5 > 10)$.
7. Инвертируйте значение `True` с помощью `not`.
8. Дано $x = 15$. Проверьте, лежит ли x между 10 и 20 (строго).
9. Дано $n = 4$. Проверьте, делится ли оно на 2 без остатка ($n \% 2 == 0$).
10. Равны ли строки "test" и "Test"?

11. Дано `battery = 4.0` и `connected = True`. Разрешен ли полет? (нужно и то, и другое).
12. Дано `obstacle = False`. Есть ли опасность? (Если препятствие есть ИЛИ высота < 0.5).
13. Проверьте сложное условие: $(2 + 2 == 4)$ и (не $(3 > 5)$).
14. Равны ли число 5 и строка "5"?
15. Чему равно `bool(1)`? А `bool(0)`?

7.5. Ветвление (Условные операторы)

Условные операторы `if`, `elif` и `else` позволяют программе принимать решения в зависимости от условий.

 `python/examples/01_python_basics/05_branching.py`

```
# Ветвление: Робот на перекрестке
direction = "forward"

if direction == "left":
    print("Поворачиваю налево")
elif direction == "right":
    print("Поворачиваю направо")
elif direction == "forward":
    print("Еду прямо")
else:
    print("Останавливаюсь, путь неясен")
```

Обратите внимание на отступы (4 пробела) — они обязательны в Python и обозначают блоки кода.

Задание 42: Светофор

Напишите программу, которая принимает цвет светофора ("red", "yellow", "green") и выводит действие ("Стоп", "Приготовиться", "Ехать").

Задачи для самостоятельного решения

1. Дано число `x = 10`. Если оно больше 0, выведите "Положительное".
2. Дано `n = 7`. Если делится на 2 без остатка — "Четное", иначе — "Нечетное".
3. Дано `age = 15`. Если `age >= 18`, выведите "Взрослый", иначе "Ребенок".

4. Дано `password = "1234"`. Если пароль совпадает с `"secret"`, пишем `"Доступ разрешен"`, иначе `"Отказ"`.
5. Даны `a = 10`, `b = 20`. Выведите большее из них.
6. Дано `x = 0`. Используя `elif`, выведите `"Плюс"`, `"Минус"` или `"Ноль"`.
7. Дано `color = "red"`. Если `"green"` — `"Ехать"`, `"yellow"` — `"Ждать"`, `"red"` — `"Стоять"`.
8. `voltage = 3.6`. Если `< 3.5` — `"Посадка!"`, иначе `"Полет нормальный"`.
9. Делится ли число 15 на 3 и на 5 одновременно?
10. Дано `name = "Alex"`. Если длина имени `> 5` букв, напишите `"Длинное"`, иначе `"Короткое"`.
11. Дана строка `s = ""`. Если она пустая (длина 0), напишите `"Пусто"`.
12. `weight = 80`. `< 50` — `"Легкий"`, `50-100` — `"Средний"`, `> 100` — `"Тяжелый"`.
13. `armed = True`, `ready = False`. Если оба `True` — `"Взлет"`, иначе `"Ждать"`.
14. `login = "admin"`. Если не равен `"admin"`, вывести `"Ошибка"`.
15. `a=5`, `b=2`, `c=8`. Найдите самое большое число из трех через `if`.

7.6. Циклы

Циклы позволяют выполнять один и тот же блок кода многократно.

Цикл for

Используется, когда известно количество повторений. Функция `range(N)` создает последовательность от 0 до `N-1`.

Цикл while

Выполняется, пока условие истинно. Будьте осторожны: если условие никогда не станет ложным, получится «бесконечный цикл».

```
python/examples/01_python_basics/06_loops.py

# Циклы
import time

# Цикл for: мигаем светодиодом 5 раз (имитация)
print("Начинаю мигать...")
for i in range(5):
    print(f"Мигание №{i+1}: ВКЛ")
```

```
time.sleep(0.5)
print(f"Мигание №{i+1}: Выкл")
time.sleep(0.5)

# Цикл while: летим пока есть заряд
battery = 10
while battery > 0:
    print(f"Летим... Заряд: {battery}%")
    battery -= 2
print("Посадка!")
```

- `break` — немедленно прерывает цикл.
- `continue` — переходит к следующей итерации.

Задание 43: Обратный отсчет

Напишите программу, которая считает от 10 до 1, а затем пишет «Поехали!». Используйте цикл `for` или `while`.

Задачи для самостоятельного решения

1. Выведите числа от 0 до 4 используя `range(5)`.
2. Выведите слово "Привет" 3 раза.
3. Выведите числа от 1 до 5 (используйте `range(1, 6)`).
4. Выведите четные числа от 0 до 8 (`range(0, 9, 2)`).
5. Найдите сумму чисел от 1 до 100.
6. Выведите числа от 5 до 1 (`range(5, 0, -1)`).
7. Выведите таблицу умножения на 5 (от $1 \cdot 5$ до $10 \cdot 5$).
8. Дана строка "Дрон". Выведите каждую букву с новой строки.
9. Используя `while`, выводите `x`, пока `x < 3`. Начальное `x = 0`.
10. Ищите число 7 в диапазоне от 0 до 10. Если нашли — используйте `break`.
11. Выведите числа от 0 до 4, пропустив 2 (используйте `continue`).
12. Выведите квадраты чисел от 1 до 5.
13. Посчитайте длину строки "Hello" с помощью цикла (без `len()`).
14. Выведите 5 строк, в каждой по одной звездочке `*`.
15. Выведите лесенку из звезд (1, 2, 3...).

7.7. Функции

Функция — это именованный блок кода, который можно вызывать многократно. Функции делают код чище и понятнее.

Ключевое слово `def` объявляет функцию. `return` возвращает результат.

```
python/examples/01_python_basics/07_functions.py

# Функции
def calculate_area(length):
    """Вычисляет площадь квадрата."""
    return length * length

side = 5
area = calculate_area(side)
print(f"Площадь квадрата со стороной {side} равна {area}")

def check_battery(voltage):
    if voltage < 3.5:
        return "Critical"
    elif voltage < 3.8:
        return "Low"
    else:
        return "Normal"

print("Статус батареи 3.6В:", check_battery(3.6))
```

Область видимости

Переменные, созданные внутри функции, не видны снаружи. Это называется локальной областью видимости.

Задание 44: Конвертер

Напишите функцию `celsius_to_fahrenheit(c)`, которая переводит градусы Цельсия в Фаренгейты по формуле $F = C \times 1.8 + 32$.

Задачи для самостоятельного решения

1. Напишите функцию `say_hello()`, которая печатает "Привет!". Вызовите её.
2. Напишите функцию `greet(name)`, которая печатает "Привет, <name>!".
3. Напишите функцию `add(a, b)`, которая возвращает сумму двух чисел.
4. Напишите функцию `square(x)`, которая возвращает x в квадрате.

5. Напишите функцию `my_max(a, b)`, которая возвращает большее из двух чисел.
6. Напишите функцию `is_even(n)`, которая возвращает `True`, если число четное.
7. Напишите функцию `perimeter(a, b)`, возвращающую периметр прямоугольника.
8. Напишите функцию `m_to_cm(meters)`, которая переводит метры в сантиметры.
9. Напишите функцию `check_battery(voltage)`. Если < 3.5 -> "Low", иначе "OK".
10. Напишите функцию `my_abs(x)`. Если $x < 0$, возвращает $-x$, иначе x .
11. Напишите функцию `avg(a, b)`, возвращающую среднее арифметическое.
12. Напишите функцию `log_error(msg)`, которая печатает "ERROR: <msg>".
13. Напишите функцию `repeat(s, n)`, возвращающую строку s , повторенную n раз.
14. Напишите функцию `flight_time(cap, cons)`, считающую время полета (емкость / расход).
15. Напишите функцию `full_name(first, last)`, возвращающую "Фамилия Имя".

7.8. Модуль «Квадрокоптер»: Pioneer SDK

Для управления квадрокоптером Pioneer используется специальная библиотека `pioneer_sdk`. Она позволяет отправлять команды (взлет, посадка, полет в точку) и получать данные (заряд батареи, видеопоток).

Установка библиотеки

Для начала работы необходимо установить библиотеку через терминал:

Терминал

```
pip install pioneer_sdk
```

Подключение к дрону

Дрон создает Wi-Fi сеть (обычно `Pioneer_...`). Подключитесь к ней с вашего компьютера.

```
python/examples/02_pioneer_sdk/08_drone_connect.py

from pioneer_sdk import Pioneer
import time

# Подключение к дрону
drone = Pioneer()
print("Дрон подключен!")

try:
    # Запрашиваем состояние батареи несколько раз
    for _ in range(3):
        voltage = drone.get_battery_status()
        print(f"Напряжение: {voltage} В")
        time.sleep(1)
finally:
    # Всегда закрываем соединение
    drone.close_connection()
    print("Соединение закрыто")
```

- `Pioneer()` — создает объект для управления дроном.
- `drone.get_battery_status()` — возвращает напряжение батареи в Вольтах.
- `drone.close_connection()` — разрывает связь, освобождая порт для других программ.

Задание 45: Диагностика

Напишите скрипт, который подключается к дрону, проверяет заряд батареи и, если он ниже 3.6 В, выводит предупреждение «Low Battery!».

7.9. Светодиоды и диагностическая индикация

После подключения к дрону полезно получить первый видимый результат без взлета. Для этого удобно использовать светодиодную индикацию: она безопасна, мгновенно показывает состояние программы и помогает отлаживать реакции на данные датчиков.

Зачем начинать со светодиодов?

- Светодиоды дают быстрый и наглядный результат без полета.
- Через цвет удобно кодировать состояния: норма, предупреждение, ошибка.
- Эта тема естественно связывает базовый Python с реальным API квадрокоптера.

Базовая команда

Для управления индикацией используется метод `led_control`. Чаще всего в учебных задачах применяется идентификатор 255, который означает управление всей подсветкой сразу.

```
python/examples/03_flight_control/LED_hight_change.py

from pioneer_sdk import Pioneer
import time

# Определяем временной интервал обновления данных
delta_time = 0.1

# Инициализируем переменные для цвета светодиодов (RGB)
r = 0
g = 0
b = 0

# Определяем пороговые значения высоты для смены цвета
height_1 = 0.25
height_2 = 0.5
height_3 = 0.75

# <----- height_1 ----- height_2 ----- height_3 ----->
```

```

#    красный              зеленый              синий              белый

if __name__ == "__main__":                                # Проверяем, что скрипт
    ↪ запущен как основной файл
    pioneer_mini = Pioneer(logger=False)                   # Создаем объект
    ↪ управления дроном, отключая логирование
    curr_time = time.time()                                # Фиксируем текущее время

    try:
        while True:                                        #
            ↪ Запускаем бесконечный цикл
            if time.time() - curr_time > delta_time:        #
                ↪ Проверяем, прошло ли заданное время (delta_time)
                tof_data = pioneer_mini.get_dist_sensor_data() #
                ↪ Получаем данные с датчика расстояния
                print(tof_data)                              #
                ↪ Выводим данные в консоль для мониторинга

                if tof_data is not None:                      #
                    ↪ Проверяем, что данные успешно получены с датчика

                    # Определяем цвет светодиодов в зависимости от
                    ↪ высоты
                    if tof_data <= height_1:
                        r = 0.1 # Красный
                        g = 0
                        b = 0
                    elif height_1 < tof_data <= height_2:
                        r = 0
                        g = 0.1 # Зеленый
                        b = 0
                    elif height_2 < tof_data <= height_3:
                        r = 0
                        g = 0
                        b = 0.1 # Синий
                    elif tof_data >= height_3:
                        r = 0.1
                        g = 0.1
                        b = 0.1 # Белый

                    # Управляем светодиодами дрона
                    pioneer_mini.led_control(led_id=255, r=r, g=g, b=b)

                    # Обновляем время последнего обновления
                    curr_time = time.time()

    except KeyboardInterrupt:
        print("Остановка программы. Выключаем светодиоды...")
        pioneer_mini.led_control(led_id=255, r=0, g=0, b=0) # Выключаем
        ↪ светодиоды

```

```
finally:
    pioneer_mini.close_connection() # Закрываем
    ↪ соединение с дроном
```

i Как читать вызов

`drone.led_control(led_id=255, r, g, b)` устанавливает цвет подсветки. Компоненты `r`, `g`, `b` задают интенсивность красного, зеленого и синего каналов.

Минимальная палитра

- (1, 0, 0) — красный: ошибка, авария, низкий заряд.
- (0, 1, 0) — зеленый: норма, готовность.
- (0, 0, 1) — синий: ожидание, служебный режим.
- (1, 1, 0) — желтый: предупреждение.
- (0, 0, 0) — выключено.

⚠ Безопасный режим

На этом этапе не нужно взлетать. Все упражнения со светодиодами можно выполнять на столе, проверяя только подключение, чтение телеметрии и корректную реакцию программы.

Светодиоды как первый дебаг-интерфейс

Даже если консоль недоступна или оператор смотрит не в терминал, цветовая индикация позволяет сразу понять состояние программы:

- зеленый — соединение установлено;
- желтый — состояние требует внимания;
- красный — нужно остановить сценарий и проверить проблему.

Задание 46: Первая индикация

Напишите скрипт, который:

1. подключается к дрону;
2. включает зеленую подсветку на 2 секунды;
3. затем выключает ее;
4. корректно закрывает соединение.

Задание 47: Светофор батареи

Используя `get_battery_status()`, реализуйте простую логику:

- выше 4.0 В — зеленый;
- от 3.7 до 4.0 В — желтый;
- ниже 3.7 В — красный.

Задание 48: Индикация по высоте

Измените пример `LED_high_change.py`, чтобы цвет зависел от высоты по вашим порогам. Например: низко — синий, рабочая высота — зеленый, слишком высоко — красный.

Что это готовит дальше

После этого раздела можно переходить к полету значительно увереннее: ученик уже умеет подключаться к дрону, вызывать методы API и связывать данные датчиков с понятной реакцией устройства.

7.10. Первый полет: Взлет и посадка

Управление дроном состоит из нескольких простых этапов. Рассмотрим их по порядку.

1. Запуск моторов (Arming)

Первым делом нужно «завести» дрон. Команда `arm()` запускает моторы на холостых оборотах.

```
python/examples/02_pioneer_sdk/09_01_arm_disarm.py

from pioneer_sdk import Pioneer
import time

drone = Pioneer()
try:
    print("Запуск моторов...")
    drone.arm()
    time.sleep(3) # Моторы вращаются 3 секунды
finally:
    print("Остановка моторов")
    drone.disarm()
```

⚠️ Безопасность

Всегда используйте блок `try...finally`, чтобы гарантировать остановку моторов (`disarm`) даже в случае ошибки в программе или нажатия `Ctrl+C`.

2. Взлет и посадка

Теперь, когда моторы работают, можно взлететь. Команда `takeoff()` поднимает дрон на высоту около 1 метра.

 `python/examples/02_pioneer_sdk/09_02_takeoff_land.py`

```
from pioneer_sdk import Pioneer
import time

drone = Pioneer()
try:
    drone.arm()
    print("Взлет!")
    drone.takeoff()
    time.sleep(5) # Висение в воздухе
    print("Посадка...")
    drone.land()
finally:
    drone.disarm() # Гарантированная остановка
```

Обратите внимание:

1. `takeoff()` — автоматический взлет.
2. `time.sleep(5)` — ожидание (висение) в течение 5 секунд.
3. `land()` — автоматическая посадка.

7.11. Маневры и автоматический полет

Мы уже умеем взлетать и садиться. Теперь научимся управлять движением дрона в пространстве.

1. Система координат

Pioneer использует локальную систему координат. Представьте, что дрон находится в центре комнаты:

- **X (Вперед/Назад):** Положительные значения — вперед, отрицательные — назад.

- **Y (Влево/Вправо):** Положительные значения — влево, отрицательные — вправо.
- **Z (Вверх/Вниз):** Положительные значения — вверх, отрицательные — вниз.
- **Yaw (Рыскание):** Поворот носа дрона (в градусах или радианах).

Команда `go_to_local_point(x, y, z, yaw)` отправляет дрон в точку с координатами (x, y, z) относительно места включения (старта).

2. Простой маневр: Вперед и назад

Попробуем пролететь 1 метр вперед, подождать и вернуться обратно.

```
python/examples/03_flight_control/10_01_forward_back.py

from pioneer_sdk import Pioneer
import time

drone = Pioneer()

try:
    print("Арминг и взлет...")
    drone.arm()
    drone.takeoff()

    # Высота полета
    h = 1.0

    # 1. Летим вперед на 1 метр
    print("Вперед!")
    drone.go_to_local_point(x=1.0, y=0.0, z=h, yaw=0)
    time.sleep(4) # Ждем, пока долетит

    # 2. Возвращаемся в точку старта (0, 0)
    print("Домой!")
    drone.go_to_local_point(x=0.0, y=0.0, z=h, yaw=0)
    time.sleep(4)

    print("Посадка...")
    drone.land()

finally:
    drone.disarm()
```

3. Полет по квадрату

Теперь усложним задачу. Пусть дрон облетит квадрат со стороной 1 метр. Координаты углов:

1. $(1, 0)$ — Вперед на 1 метр.
2. $(1, 1)$ — Влево на 1 метр (теперь мы в точке $x = 1, y = 1$).
3. $(0, 1)$ — Назад на 1 метр (но остаемся смещенными влево).
4. $(0, 0)$ — Возврат в точку старта.

 python/examples/03_flight_control/10_02_square.py

```
from pioneer_sdk import Pioneer
import time

drone = Pioneer()

try:
    print("Взлет...")
    drone.arm()
    drone.takeoff()
    h = 1.0 # Высота

    # Квадрат 1x1 метр
    # Точка 1: (1, 0)
    drone.go_to_local_point(1.0, 0.0, h, 0)
    time.sleep(4)

    # Точка 2: (1, 1)
    drone.go_to_local_point(1.0, 1.0, h, 0)
    time.sleep(4)

    # Точка 3: (0, 1)
    drone.go_to_local_point(0.0, 1.0, h, 0)
    time.sleep(4)

    # Точка 4: (0, 0) - Домой
    drone.go_to_local_point(0.0, 0.0, h, 0)
    time.sleep(4)

    drone.land()
finally:
    drone.disarm()
```

4. Полет по кругу (Математика)

Как заставить дрон лететь по кругу? Можно задать много точек вручную, но это долго. Лучше использовать математику! Координаты точки на

окружности вычисляются по формулам:

$$x = R \cdot \cos(\alpha)$$

$$y = R \cdot \sin(\alpha)$$

где R — радиус, а α — угол.

В Python для этого используется модуль `math`.

 `python/examples/03_flight_control/10_03_circle_math.py`

```
from pioneer_sdk import Pioneer
import time
import math

drone = Pioneer()

try:
    print("Взлет...")
    drone.arm()
    drone.takeoff()

    h = 1.0          # Высота
    radius = 0.5      # Радиус круга (0.5 метра)
    points_count = 12 # Количество точек (чем больше, тем плавнее круг)

    # Цикл по точкам
    for i in range(points_count + 1):
        # Вычисляем угол в градусах (от 0 до 360)
        angle_deg = (360 / points_count) * i

        # Переводим в радианы (для sin/cos)
        angle_rad = math.radians(angle_deg)

        # Считаем координаты X и Y
        # x = R * cos(a)
        # y = R * sin(a)
        x = radius * math.cos(angle_rad)
        y = radius * math.sin(angle_rad)

        # В начале координат (0,0) дрон стоит в центре круга?
        # Нет, если мы хотим летать ВОКРУГ точки старта, то координаты
        # → правильные.
        # Если хотим летать так, чтобы старт был НА окружности, нужно
        # → смещение.
        # Для простоты летим вокруг точки старта (радиус 0.5м от
        # → старта).

    print(f"Точка {i}: x={x:.2f}, y={y:.2f}")
```

```
        drone.go_to_local_point(x, y, h, 0)
        time.sleep(2) # Пауза между точками

    drone.land()
finally:
    drone.disarm()
```

Задание 49: Восьмерка

Используя формулы для лемнискаты Бернулли или просто соединив два круга, заставьте дрон пролететь по траектории «восьмерки».

7.12. Система контрольных упражнений

Для проверки навыков пилотирования разработана система последовательных упражнений с автоматической проверкой точности (Grader).

Упражнение 1: Полёт вперёд

Задача: Прямолинейный полет на заданную дистанцию (1 м) без возврата.

 python/examples/exercises/ex1_forward.py

```
from pioneer_sdk import Pioneer
from grader import FlightGrader
import time

"""
Упражнение 1 – «Полёт вперёд»: модуль должен обеспечить ровный
→ прямолинейный проход заданной дистанции без возврата.
"""

def exercise_01():
    drone = Pioneer()
    grader = FlightGrader(drone)

    try:
        print("[System] Упражнение 1: Полет вперед")
        print("Ожидание LPS...")
        # Проверка готовности LPS (упрощенно)

        grader.start_mission()

        # 1. Взлет
        print("Взлет...")
        drone.arm()
        drone.takeoff()
        time.sleep(5)

        # 2. Установка цели (x=1.0, y=0, z=1.0)
        target_x = 1.0
        grader.set_target(target_x, 0.0, 1.0, 0.0)

        # 3. Полет вперед
        print(f"Полет к точке x={target_x}...")
        drone.go_to_local_point(x=target_x, y=0.0, z=1.0, yaw=0.0)

        # Ожидание прибытия (около 5 с)
        time.sleep(5)
```

```

# 4. Проверка позиции
grader.check_checkpoint("Точка Вперед")

# 5. Посадка (Конец упражнения, без возврата)
print("Посадка...")
drone.land()

finally:
    grader.finish_mission()
    drone.disarm()
    drone.close_connection()

if __name__ == '__main__':
    exercise_01()

```

Упражнение 2: Вперед-разворот-назад

Задача: Полет вперед, немедленный разворот на 180 градусов и возврат в точку старта (летим «носом» к дому).

 python/examples/exercises/ex2_forward_turn_return.py

```

from pioneer_sdk import Pioneer
from grader import FlightGrader
import time
import math

"""
Упражнение 2 – «Полёт вперёд и назад»: модуль должен выполнить точный
→ проход вперёд на заданное расстояние, немедленный разворот на 180° и
→ возвращение на исходную точку без смещения по бокам.
"""

def exercise_02():
    drone = Pioneer()
    grader = FlightGrader(drone)

    try:
        print("[System] Упражнение 2: Вперед и Назад (с разворотом)")
        grader.start_mission()

        # 1. Взлет
        drone.arm()
        drone.takeoff()
        time.sleep(5)

        # 2. Полет вперед (x=1.0)
        dist = 1.0
        print(f"Этап 1: Вперед к x={dist}...")

```

```

grader.set_target(dist, 0.0, 1.0, 0.0)
drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=0.0)
time.sleep(5)
grader.check_checkpoint("Точка Вперед")

# 3. Разворот на 180 (Немедленный)
print("Этап 2: Немедленный разворот на 180...")
# Примечание: go_to_local_point с новым yaw. Позиция остается
→ прежней.
# Используем радианы для yaw, так как это стандарт.
yaw_180 = math.radians(180)

grader.set_target(dist, 0.0, 1.0, 180.0) # Ожидаем yaw=180 в
→ точке x=1
drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=yaw_180)
time.sleep(3)
grader.check_checkpoint("Разворот 180")

# 4. Возврат на старт (x=0)
# Мы сохраняем yaw=180, то есть летим "носом" к точке старта.
print("Этап 3: Возврат на старт...")
grader.set_target(0.0, 0.0, 1.0, 180.0)
drone.go_to_local_point(x=0.0, y=0.0, z=1.0, yaw=yaw_180)
time.sleep(5)
grader.check_checkpoint("Точка Дома (Задом)")

drone.land()

finally:
    grader.finish_mission()
    drone.disarm()
    drone.close_connection()

if __name__ == '__main__':
    exercise_02()

```

Упражнение 3: Строгий разворот

Задача: Аналогично упр. 2, но с жестким контролем угла рыскания (Yaw) на всех этапах.

 python/examples/exercises/ex3_forward_turn_return.py

```

from pioneer_sdk import Pioneer
from grader import FlightGrader
import time
import math

```

```

"""

```

Упражнение 3 – «Полёт вперёд, разворот 180°, возврат»: модуль должен
 ↳ совершить проход вперёд, строгий разворот на 180° вокруг
 ↳ вертикальной оси и точный возврат к стартовой координате с тем же
 ↳ курсом.
 """

```
def exercise_03():
    drone = Pioneer()
    grader = FlightGrader(drone)

    try:
        print("[System] Упражнение 3: Вперед, Разворот 180, Возврат
        ↳ (Строгий Yaw)")
        grader.start_mission()

        # 1. Взлет
        drone.arm()
        drone.takeoff()
        time.sleep(5)

        # 2. Полет вперед (x=1.0)
        dist = 1.0
        print(f"Этап 1: Вперед к x={dist}...")
        grader.set_target(dist, 0.0, 1.0, 0.0)
        drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=0.0)
        time.sleep(5)
        grader.check_checkpoint("Точка Вперед")

        # 3. Строгий разворот на 180
        print("Этап 2: Строгий разворот на 180...")
        yaw_180 = math.radians(180)
        grader.set_target(dist, 0.0, 1.0, 180.0) # Ожидаем 180 градусов
        drone.go_to_local_point(x=dist, y=0.0, z=1.0, yaw=yaw_180)
        time.sleep(3)
        grader.check_checkpoint("Разворот Завершен")

        # 4. Возврат на старт (x=0) с ТЕМ ЖЕ КУРСОМ (Heading)
        # "Возврат к стартовой координате с тем же курсом".
        # Это означает сохранение ориентации yaw=180.
        print("Этап 3: Возврат на старт (Удержание курса)...")
        grader.set_target(0.0, 0.0, 1.0, 180.0)
        drone.go_to_local_point(x=0.0, y=0.0, z=1.0, yaw=yaw_180)
        time.sleep(5)
        grader.check_checkpoint("Точка Дома")

        drone.land()

    finally:
        grader.finish_mission()
        drone.disarm()
        drone.close_connection()
```

```
if __name__ == '__main__':  
    exercise_03()
```

8. Задания к главе

Уровень 1: Обязательный

1. **Калькулятор полета.** Напишите программу, которая запрашивает у пользователя время полета (в минутах) и среднюю скорость (в м/с), а затем выводит пройденное расстояние в метрах и километрах.
2. **Чек-лист.** Создайте список (`list`) из 5 пунктов проверки перед вылетом (например, «Батарея заряжена», «Винты целы»). Используя цикл `for`, выведите каждый пункт с номером.

Уровень 2: Усложненный

1. **Умная батарея.** Напишите функцию `check_battery(voltage)`, которая принимает напряжение и возвращает статус:
 - > 4.0 В: «Отличный заряд»
 - $3.7 - 4.0$ В: «Средний заряд»
 - < 3.7 В: «Посадка немедленно!»Интегрируйте эту функцию в скрипт подключения к дрону.

Уровень 3: Творческий (Мини-проект)

Миссия «Квадрат». Используя команды `drone.go_to_local_point(x, y, z, yaw)`, напишите программу, которая заставляет дрон пролететь по траектории квадрата со стороной 1 метр на высоте 1.5 метра. *Подсказка:* Вам понадобятся координаты точек: $(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 0)$. Не забывайте про `time.sleep()` между командами!

8.1. Компьютерное зрение: ArUco и AprilTag

Для роботов существуют специальные визуальные метки (маркеры), которые легко распознать на изображении. Самые популярные — **ArUco** и **AprilTag**.

Что такое ArUco-маркер?

Это черно-белый квадрат с уникальным узором внутри. У каждого маркера есть свой **ID** (номер).

- **Словарь (Dictionary):** Набор допустимых маркеров. Pioneer обычно использует словарь `DICT_6X6_50` (размер сетки 6x6 бит, всего 50 уникальных маркеров).
- **AprilTag:** Другой популярный формат, поддерживаемый той же библиотекой (например, `DICT_APRILTAG_36h11`).

Детекция маркеров

Для работы с камерой и маркерами мы используем библиотеку **OpenCV** (cv2).

```
python/examples/aruco_examples/11_aruco_basics_example1_detect.py

import cv2
import cv2.aruco as aruco
import numpy as np
from pioneer_sdk import Camera
import threading
import time

class ArucoDetectorThread:
    def __init__(self):
        self.camera = Camera()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        # 1. Настройка словаря ArUco (6x6, 50 маркеров - стандарт для
        #    Pioneer)
        self.aruco_dict =
            aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        # Обеспечение совместимости разных версий OpenCV
```

```

    if hasattr(aruco, 'DetectorParameters_create'):
        self.parameters = aruco.DetectorParameters_create()
    else:
        self.parameters = aruco.DetectorParameters()

def detect_markers_compat(self, image):
    if hasattr(aruco, 'ArucoDetector'):
        # Новый API (OpenCV 4.7+)
        detector = aruco.ArucoDetector(self.aruco_dict,
            ↪ self.parameters)
        return detector.detectMarkers(image)
    else:
        # Старый API
        return aruco.detectMarkers(image, self.aruco_dict,
            ↪ parameters=self.parameters)

def capture_thread(self):
    while self.running:
        frame = self.camera.get_cv_frame()
        if frame is not None:
            with self.frame_lock:
                self.latest_frame = frame.copy()
        time.sleep(0.01)

def processing_thread(self):
    while self.running:
        frame_to_process = None
        with self.frame_lock:
            if self.latest_frame is not None:
                frame_to_process = self.latest_frame.copy()

        if frame_to_process is None:
            time.sleep(0.01)
            continue

        # Переводим в оттенки серого (для ускорения)
        gray = cv2.cvtColor(frame_to_process, cv2.COLOR_BGR2GRAY)

        # 3. Детекция маркеров
        corners, ids, rejected = self.detect_markers_compat(gray)

        # Если маркеры найдены
        if ids is not None:
            # Рисуем рамки вокруг маркеров
            aruco.drawDetectedMarkers(frame_to_process, corners,
                ↪ ids)
            # Выводим ID найденных маркеров в консоль (можно
            ↪ закомментировать, если спамит)
            # print(f"Виджу маркеры: {ids.flatten()}")

        with self.frame_lock:

```

```
        self.processed_frame = frame_to_process

def run(self):
    print("Запуск детекции маркеров... Нажмите ESC для выхода.")

    t_capture = threading.Thread(target=self.capture_thread)
    t_process = threading.Thread(target=self.processing_thread)

    t_capture.daemon = True
    t_process.daemon = True

    t_capture.start()
    t_process.start()

    try:
        while True:
            display_frame = None
            with self.frame_lock:
                if self.processed_frame is not None:
                    display_frame = self.processed_frame.copy()
                elif self.latest_frame is not None:
                    display_frame = self.latest_frame.copy()

            if display_frame is not None:
                cv2.imshow('Pioneer Camera', display_frame)

            if cv2.waitKey(1) == 27:
                self.running = False
                break

            time.sleep(0.03)

        finally:
            self.running = False
            t_capture.join(timeout=1.0)
            t_process.join(timeout=1.0)
            cv2.destroyAllWindows()

if __name__ == "__main__":
    detector = ArucoDetectorThread()
    detector.run()
```

Разбор кода

1. `camera.get_cv_frame()` получает изображение с камеры дрона. 2. `cv2.cvtColor` переводит картинку в ч/б формат (так алгоритм работает быстрее и надежнее). 3. `detectMarkers` ищет квадраты и сверяет их узор со словарем. 4. Возвращает:

- `corners`: Координаты углов найденных маркеров.
- `ids`: Список номеров (ID) найденных маркеров.

Задание 50: Охота на маркеры

Распечатайте несколько маркеров ArUco (или откройте их на экране телефона). Напишите программу, которая пишет в консоль «Вижу маркер X!», где X — номер маркера.

Определение координат маркера

Зная размер маркера и параметры камеры (полученные при калибровке), можно вычислить расстояние и ориентацию относительно маркера.

Определение координат

```
import cv2
import numpy as np
from pioneer_sdk import Camera
import cv2.aruco as aruco
import threading
import time

def load_coefficients(path):
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()
    cv_file.release()
    return camera_matrix, dist_coeffs

class PoseEstimatorThread:
    def __init__(self, camera_matrix, dist_coeffs):
        self.camera = Camera()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        self.camera_matrix = camera_matrix
        self.dist_coeffs = dist_coeffs

        # Dictionary of aruco-markers
        self.aruco_dict =
        ↪ aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        # Parameters for marker detection
        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
```

```

    else:
        self.parameters = aruco.DetectorParameters()

    self.size_of_marker = 0.05 # side length in meters

    # Coordinates of marker corners
    self.points_of_marker = np.array([
        (self.size_of_marker / 2, -self.size_of_marker / 2, 0),
        (-self.size_of_marker / 2, -self.size_of_marker / 2, 0),
        (-self.size_of_marker / 2, self.size_of_marker / 2, 0),
        (self.size_of_marker / 2, self.size_of_marker / 2, 0),
    ])

def detect_markers_compat(self, image):
    if hasattr(aruco, 'ArucoDetector'):
        # Новый API (OpenCV 4.7+)
        detector = aruco.ArucoDetector(self.aruco_dict,
            ↪ self.parameters)
        return detector.detectMarkers(image)
    else:
        # Старый API
        return aruco.detectMarkers(image, self.aruco_dict,
            ↪ parameters=self.parameters)

def capture_thread(self):
    while self.running:
        frame = self.camera.get_cv_frame()
        if frame is not None:
            with self.frame_lock:
                self.latest_frame = frame.copy()
        time.sleep(0.01)

def processing_thread(self):
    while self.running:
        frame_to_process = None
        with self.frame_lock:
            if self.latest_frame is not None:
                frame_to_process = self.latest_frame.copy()

        if frame_to_process is None:
            time.sleep(0.01)
            continue

        # Detect markers
        corners, ids, rejected =
            ↪ self.detect_markers_compat(frame_to_process)

        if ids is not None and len(ids) > 0:
            # Calculate pose for first detected marker
            success, rvecs, tvecs = cv2.solvePnP(

```

```

        self.points_of_marker, corners[0],
        ↪ self.camera_matrix, self.dist_coeffs
    )

    if success:
        # Draw axes
        cv2.drawFrameAxes(frame_to_process,
            ↪ self.camera_matrix, self.dist_coeffs, rvecs,
            ↪ tvecs, 0.1)
        # print(f"Marker Pose: {tvecs.T}") # Optional
        ↪ Logging

    with self.frame_lock:
        self.processed_frame = frame_to_process

def run(self):
    print("Запуск оценки позы... Нажмите ESC для выхода.")

    t_capture = threading.Thread(target=self.capture_thread)
    t_process = threading.Thread(target=self.processing_thread)

    t_capture.daemon = True
    t_process.daemon = True

    t_capture.start()
    t_process.start()

    try:
        while True:
            display_frame = None
            with self.frame_lock:
                if self.processed_frame is not None:
                    display_frame = self.processed_frame.copy()
                elif self.latest_frame is not None:
                    display_frame = self.latest_frame.copy()

            if display_frame is not None:
                cv2.imshow("Pose Estimation", display_frame)

            if cv2.waitKey(1) == 27:
                self.running = False
                break

            time.sleep(0.03)

    finally:
        self.running = False
        t_capture.join(timeout=1.0)
        t_process.join(timeout=1.0)
        cv2.destroyAllWindows()

```

```

if __name__ == "__main__":
    # Ensure data.yml exists or provide correct path
    try:
        camera_matrix, dist_coeffs = load_coefficients("data.yml")
        estimator = PoseEstimatorThread(camera_matrix, dist_coeffs)
        estimator.run()
    except Exception as e:
        print(f"Ошибка загрузки калибровки или запуска: {e}")
        print("Убедитесь, что файл 'data.yml' с параметрами калибровки
        ↳ существует.")

```

Удержание позиции по маркеру

В этом примере дрон использует координаты маркера для удержания позиции над ним.

Полет по маркеру

```

import cv2
import cv2.aruco as aruco
from pioneer_sdk import Pioneer, Camera
import threading
import time

class DroneFlightThread:
    def __init__(self):
        self.camera = Camera()
        self.drone = Pioneer()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()
        self.detected_ids = []

        # ArUco setup
        self.aruco_dict =
            ↳ aruco.getPredefinedDictionary(aruco.DICT_6X6_50)

        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):
            detector = aruco.ArucoDetector(self.aruco_dict,
            ↳ self.parameters)
            return detector.detectMarkers(image)
        else:

```

```

        return aruco.detectMarkers(image, self.aruco_dict,
                                   ↪ parameters=self.parameters)

def capture_thread(self):
    while self.running:
        frame = self.camera.get_cv_frame()
        if frame is not None:
            with self.frame_lock:
                self.latest_frame = frame.copy()
            time.sleep(0.01)

def processing_thread(self):
    while self.running:
        frame_to_process = None
        with self.frame_lock:
            if self.latest_frame is not None:
                frame_to_process = self.latest_frame.copy()

        if frame_to_process is None:
            time.sleep(0.01)
            continue

        gray = cv2.cvtColor(frame_to_process, cv2.COLOR_BGR2GRAY)
        corners, ids, rejected = self.detect_markers_compat(gray)

        current_ids = []
        if ids is not None:
            aruco.drawDetectedMarkers(frame_to_process, corners,
                                       ↪ ids)
            current_ids = ids.flatten().tolist()

        # Update shared state
        self.detected_ids = current_ids

        with self.frame_lock:
            self.processed_frame = frame_to_process

def flight_logic(self):
    print("Арминг и взлет...")
    self.drone.arm()
    self.drone.takeoff()
    time.sleep(2)

    print("Летим в точку наблюдения...")
    self.drone.go_to_local_point(x=0, y=0, z=1, yaw=0)

    landing_marker_id = 42

    while self.running:
        if landing_marker_id in self.detected_ids:

```

```

        print(f"Посадочный маркер ({landing_marker_id}) найден!
        ↪ Сажусь...")
        self.drone.land()
        self.running = False # Stop other threads
        break

    time.sleep(0.1)

def run(self):
    print("Запуск полетной миссии... Нажмите ESC для выхода.")

    t_capture = threading.Thread(target=self.capture_thread)
    t_process = threading.Thread(target=self.processing_thread)
    t_flight = threading.Thread(target=self.flight_logic)

    t_capture.daemon = True
    t_process.daemon = True
    t_flight.daemon = True # Flight Logic runs in background

    t_capture.start()
    t_process.start()
    t_flight.start()

    try:
        while self.running:
            display_frame = None
            with self.frame_lock:
                if self.processed_frame is not None:
                    display_frame = self.processed_frame.copy()
                elif self.latest_frame is not None:
                    display_frame = self.latest_frame.copy()

            if display_frame is not None:
                cv2.imshow('Pioneer Camera', display_frame)

            if cv2.waitKey(1) == 27:
                print("Принудительная посадка...")
                self.drone.land()
                self.running = False
                break

            time.sleep(0.03)

    finally:
        self.running = False
        # Wait a bit for threads to notice
        time.sleep(0.5)
        cv2.destroyAllWindows()
        self.drone.close_connection()

if __name__ == "__main__":

```

```
mission = DroneFlightThread()
mission.run()
```

8.2. Визуальная навигация

Зная координаты углов маркера на изображении, мы можем вычислить положение дрона относительно маркера. Это называется **Pose Estimation** (Оценка позы).

Определение положения (Pose Estimation)

Для точного определения расстояния и угла наклона используется функция `cv2.aruco.estimatePoseSingleMarkers`. Она возвращает два вектора:

- **tvec (Translation Vector):** Вектор смещения (x, y, z) — где маркер относительно камеры.
- **rvec (Rotation Vector):** Вектор вращения — как маркер наклонен.

Для работы этой функции необходимо знать параметры камеры (матрицу калибровки), которые можно получить с помощью скрипта калибровки.

Простая логика следования

Даже без сложной математики можно реализовать следование за маркером:

1. Найти центр маркера на изображении (среднее арифметическое углов).
2. Найти центр кадра (обычно 320x240 для разрешения 640x480).
3. Если $X_{marker} > X_{center}$, значит маркер справа → поворачиваем вправо.
4. Если $X_{marker} < X_{center}$, значит маркер слева → поворачиваем влево.

 `python/examples/aruco_examples/12_follow_aruco.py`

```
import cv2
import cv2.aruco as aruco
import numpy as np
from pioneer_sdk import Pioneer, Camera
import threading
import time
from collections import deque
```

```

def load_coefficients(path):
    try:
        cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)
        camera_matrix = cv_file.getNode("mtx").mat()
        dist_coeffs = cv_file.getNode("dist").mat()
        cv_file.release()
        return camera_matrix, dist_coeffs
    except Exception:
        print("Warning: Calibration file not found, using dummy
        ↪ values.")
        return np.eye(3), np.zeros((5, 1))

class FollowArucoThread:
    def __init__(self, camera_matrix, dist_coeffs):
        self.camera = Camera()
        self.drone = Pioneer()
        self.running = True
        self.latest_frame = None
        self.processed_frame = None
        self.frame_lock = threading.Lock()

        self.camera_matrix = camera_matrix
        self.dist_coeffs = dist_coeffs

        # Flight control variables
        self.latest_coordinates = None # [x, y, z] relative to camera
        self.x_center = None

        # ArUco setup
        self.aruco_dict =
        ↪ aruco.getPredefinedDictionary(aruco.DICT_4X4_50)

        if hasattr(aruco, 'DetectorParameters_create'):
            self.parameters = aruco.DetectorParameters_create()
        else:
            self.parameters = aruco.DetectorParameters()

        self.size_of_marker = 0.1 # m
        self.points_of_marker = np.array([
            (self.size_of_marker / 2, -self.size_of_marker / 2, 0),
            (-self.size_of_marker / 2, -self.size_of_marker / 2, 0),
            (-self.size_of_marker / 2, self.size_of_marker / 2, 0),
            (self.size_of_marker / 2, self.size_of_marker / 2, 0),
        ])

        # FPS calculation
        self.frame_times = deque(maxlen=30)
        self.last_fps_log_time = time.time()

    def detect_markers_compat(self, image):
        if hasattr(aruco, 'ArucoDetector'):

```

```

        detector = aruco.ArucoDetector(self.aruco_dict,
        ↪ self.parameters)
        return detector.detectMarkers(image)
    else:
        return aruco.detectMarkers(image, self.aruco_dict,
        ↪ parameters=self.parameters)

def capture_thread(self):
    while self.running:
        frame = self.camera.get_cv_frame()
        if frame is not None:
            with self.frame_lock:
                self.latest_frame = frame.copy()
            time.sleep(0.01)

def processing_thread(self):
    while self.running:
        frame_to_process = None
        with self.frame_lock:
            if self.latest_frame is not None:
                frame_to_process = self.latest_frame.copy()

        if frame_to_process is None:
            time.sleep(0.01)
            continue

        try:
            corners, ids, _ =
            ↪ self.detect_markers_compat(frame_to_process)

            if ids is not None and len(corners) > 0:
                # Calculate center for visualization
                x_c = int(sum([p[0] for p in corners[0][0]]) / 4)
                y_c = int(sum([p[1] for p in corners[0][0]]) / 4)

                # Draw visual feedback
                dot_size = 5
                frame_to_process[y_c - dot_size:y_c + dot_size, x_c
                ↪ - dot_size:x_c + dot_size] = [0, 0, 255]
                aruco.drawDetectedMarkers(frame_to_process, corners)

                # Solve PnP
                success, rvecs, tvecs = cv2.solvePnP(
                    self.points_of_marker, corners[0],
                    ↪ self.camera_matrix, self.dist_coeffs
                )

                if success:
                    self.latest_coordinates = [tvecs.item(0),
                    ↪ tvecs.item(1), tvecs.item(2)]
                    self.x_center = x_c

```

```

        else:
            self.latest_coordinates = None
            self.x_center = None
    else:
        self.latest_coordinates = None
        self.x_center = None

    # FPS Logging
    now = time.time()
    self.frame_times.append(now)
    if now - self.last_fps_log_time > 2.0 and
    ↪ len(self.frame_times) > 1:
        fps = len(self.frame_times) / (self.frame_times[-1]
        ↪ - self.frame_times[0])
        # print(f"[Video Thread] FPS: {fps:.2f}")
        self.last_fps_log_time = now

    with self.frame_lock:
        self.processed_frame = frame_to_process

except cv2.error as e:
    print(f"OpenCV error: {e}")

def control_loop(self):
    """Main control loop for following the marker"""
    # Note: This logic assumes we want to control the drone based on
    ↪ vision.
    # Implementation depends on specific requirements (e.g., keep
    ↪ marker in center).

    # Simple P-controller constants
    k_p_yaw = 0.1
    k_p_fwd = 0.2
    target_dist = 1.0

    while self.running:
        if self.latest_coordinates is not None:
            x, y, z = self.latest_coordinates
            dist = z # Z in camera frame is distance forward

            # Simple Logic:
            # 1. Rotate to keep x near 0 (center of image
            ↪ horizontally)
            # 2. Move forward/back to keep z near target_dist

            yaw_cmd = 0 # Calculate based on x
            vx_cmd = 0 # Calculate based on z

            # Placeholder for actual control logic
            # self.drone.set_manual_speed(vx=vx_cmd, vy=0, vz=0,
            ↪ yaw_rate=yaw_cmd)

```

```

        pass

        time.sleep(0.05)

    def run(self):
        print("Запуск следования за маркером... Нажмите ESC для
        ↪ выхода.")

        t_capture = threading.Thread(target=self.capture_thread)
        t_process = threading.Thread(target=self.processing_thread)
        t_control = threading.Thread(target=self.control_loop)

        t_capture.daemon = True
        t_process.daemon = True
        t_control.daemon = True

        t_capture.start()
        t_process.start()
        t_control.start()

    try:
        while self.running:
            display_frame = None
            with self.frame_lock:
                if self.processed_frame is not None:
                    display_frame = self.processed_frame.copy()
                elif self.latest_frame is not None:
                    display_frame = self.latest_frame.copy()

            if display_frame is not None:
                cv2.imshow('Follow ArUco', display_frame)

            if cv2.waitKey(1) == 27:
                self.running = False
                break

            time.sleep(0.03)

    finally:
        self.running = False
        time.sleep(0.5)
        cv2.destroyAllWindows()
        self.drone.close_connection()

if __name__ == "__main__":
    camera_matrix, dist_coeffs = load_coefficients("data.yml")
    follower = FollowArucoThread(camera_matrix, dist_coeffs)
    follower.run()

```

Этот пример показывает базовую логику: дрон взлетает, висит в воздухе и ищет маркеры. Если он видит маркер с ID 42 (ответ на главный вопрос

жизни, вселенной и всего такого), он совершает автоматическую посадку.

Задание 51: Посадка на базу

Реализуйте алгоритм: дрон висит в воздухе. Если он видит маркер ID 1, он летит вперед. Если видит маркер ID 2, летит назад. Если видит маркер ID 0 — садится.

8.3. Продвинутая работа с камерой

Мы уже использовали камеру для поиска маркеров. Теперь разберемся, как работать с видеопотоком более гибко.

Класс VideoStream

Для получения «сырого» видеопотока (без автоматического декодирования в OpenCV формат, что иногда нужно для оптимизации) используется класс VideoStream.

```
python/examples/camera_examples/camera_stream.py
from pioneer_sdk import VideoStream

if __name__ == "__main__":
    stream = VideoStream()           # Создаем объект потока видео

    while True:                     # Бесконечный цикл, ожидающий команд
        cmd = input()               # Ожидаем ввода команды от пользователя
        ↵ в консоль

        if cmd == "start":          # Если введена команда 'start'
            stream.start()          # Запускаем видеопоток

        elif cmd == "stop":         # Если введена команда 'stop'
            stream.stop()           # Останавливаем видеопоток
```

Этот код запускает передачу видео по Wi-Fi. Вы можете открыть этот поток в плеере VLC (адрес обычно `rtsp://192.168.4.1:8554/front`).

Захват кадров и фотографирование

Чтобы сделать фото, нужно получить текущий кадр и сохранить его в файл с помощью `cv2.imwrite`.

Задание 52: Фотограф

Напишите программу, которая показывает видео с дрона на экране. При нажатии клавиши S (Save) текущий кадр должен сохраняться в файл `photo_N.jpg`, где N — порядковый номер фото.

Калибровка камеры

Для точных измерений (например, расстояния до маркера) нужно знать оптические параметры камеры: матрицу камеры и коэффициенты дисторсии.

Процесс калибровки: 1. Распечатать шахматную доску (Chessboard). 2. Сделать серию снимков доски под разными углами. 3. Запустить скрипт калибровки (`camera_calibration.py`), который вычислит параметры и сохранит их в файл (обычно `.yaml` или `.xml`).

Зачем это нужно?

Без калибровки прямые линии на изображении могут казаться изогнутыми (эффект «рыбьего глаза»), что приведет к ошибкам в навигации.

8.4. Телеметрия и работа с датчиками

У Pioneer есть множество датчиков. Мы научимся получать от них данные.

Основные датчики

- **Барометр:** Измеряет высоту.
- **IMU (Гироскоп + Акселерометр):** Измеряет углы наклона и ускорения.
- **Дальномер (ToF):** Лазерный датчик расстояния (смотрит вниз), измеряет дистанцию до пола.
- **Система оптического потока (OptiFlow):** Камера, смотрящая вниз, для удержания позиции.

Чтение данных дальномера

Дальномер полезен для точного удержания высоты при посадке или полете над препятствиями.

```
python/examples/get_telemetry/GET_dist_sensor_data.py

from pioneer_sdk import Pioneer
import time

# Создаем объект mini класса Pioneer
mini = Pioneer()

# Бесконечный цикл для постоянного опроса датчика
while True:
    # Получаем данные с датчика (в метрах)
    distance = mini.get_dist_sensor_data()

    # Проверяем, получены ли данные (не None)
    if distance is not None:
        # Выводим расстояние в консоль
        print(f'Расстояние: {distance:.2f} м')

    # Задержка вывода сообщений в консоль
    time.sleep(2)
```

Локальная позиция

Дрон постоянно вычисляет свои координаты (x, y, z) относительно точки старта. Эти данные можно получить через `get_local_position_lps()`.

Задание 53: Черный ящик

Напишите программу, которая во время полета каждые 0.5 секунды записывает в текстовый файл `flight_log.txt` текущее время, высоту и заряд батареи.

Задание 54: Мягкая посадка

Используя датчик, реализуйте алгоритм мягкой посадки: дрон снижается со скоростью 0.5 м/с. Когда высота становится меньше 0.3 метра, скорость снижения уменьшается до 0.1 м/с.

8.5. Трекинг человека и жестов

Используя библиотеку **MediaPipe** от Google, мы можем распознавать позу человека (скелет) в реальном времени даже на обычном процессоре.

Как это работает?

1. Получаем кадр с камеры. 2. MediaPipe находит ключевые точки (суставы): плечи, локти, запястья, нос и т.д. 3. Анализируя взаимное расположение точек, мы можем определить жест.

Пример: Управление жестами

В примере `human-tracking.py` реализована сложная логика управления. Давайте разберем основные идеи.

Определение жеста «Руки вверх»: Если запястья (y_{wrist}) выше носа (y_{nose}), значит человек поднял руки.

Логика жеста

```
if (left_wrist.y < nose.y) and (right_wrist.y < nose.y):  
    print("Жест: Взлет!")  
    drone.takeoff()
```

Пропорциональное управление (P-регулятор)

Чтобы дрон поворачивался за человеком, мы вычисляем ошибку: насколько центр человека смещен от центра кадра.

$$Error = X_{person} - X_{center}$$

Управляющее воздействие (скорость поворота) пропорционально ошибке:

$$YawRate = K_p \times Error$$

Где K_p — коэффициент чувствительности.

Задание 55: Джедай

Напишите упрощенную программу:

1. Дрон стоит на земле (не взлетает).
2. Камера включена.
3. Если вы поднимаете левую руку — дрон зажигает левый светодиод красным.
4. Если правую — правый зеленым.

5. Если скрещиваете руки (расстояние между запястьями мало) — мигает всеми цветами.

8.6. Групповой полет (Рой дронов)

Управление роем (Swarm) — сложная, но захватывающая задача. Главное здесь — синхронизация и единая система координат.

Архитектура роя

Мы будем использовать централизованную архитектуру:

- **Один компьютер (Наземная станция)** управляет всеми дронами.
- Каждый дрон имеет свой уникальный IP-адрес.
- Программа создает отдельный поток (Thread) для каждого дрона.

Пример: Синхронный взлет

В файле `group_flight_examples/swarm_OPT.py` показан пример, где несколько дронов выполняют задачи параллельно.

Ключевые моменты: 1. Создаем список IP-адресов. 2. Для каждого IP создаем объект `Pioneer`. 3. Используем `threading.Thread` для запуска функций управления одновременно.

Псевдокод роя

```
drones = [Pioneer(ip1), Pioneer(ip2)]

def mission(drone):
    drone.arm()
    drone.takeoff()
    drone.land()

# Запуск потоков
threads = []
for d in drones:
    t = threading.Thread(target=mission, args=(d,))
    t.start()
```

```
threads.append(t)

# Ожидание завершения
for t in threads:
    t.join()
```

⚠️ **Безопасность роя**

При групповых полетах риск столкновения возрастает многократно! 1. Всегда проверяйте координаты старта. 2. Расстояние между дронами должно быть не менее 1.5 метров. 3. Держите палец на кнопке аварийного отключения (или имейте общий скрипт `disarm_all`).

Задание 56: Танец

Напишите программу для двух дронов. Дрон А взлетает. Дрон Б ждет 5 секунд, затем взлетает. Дрон А садится. Дрон Б садится. Используйте `time.sleep` и потоки.

8.7. Итоговые проекты курса

Поздравляем! Вы освоили управление дроном Pioneer на языке Python. Теперь пришло время применить знания в комплексных проектах.

Проект 1: Автономный инспектор (Склад)

Задача: Дрон должен облететь 4 стеллажа (маркеры ID 0, 1, 2, 3), расположенных по углам комнаты. **Алгоритм:** 1. Взлет. 2. Поиск маркера 0. Подлет к нему на 1 метр. 3. Поворот на 90 градусов, поиск маркера 1. 4. ... 5. Возврат на точку старта и посадка.

Проект 2: «Следуй за мной» (Follow Me)

Задача: Дрон должен удерживать человека в центре кадра на безопасном расстоянии. **Требования:** 1. Использовать MediaPipe для детекции человека. 2. Если человек подходит близко (плечи занимают весь кадр) — дрон

отлетает назад. 3. Если человек уходит — дрон летит вперед. 4. Поворот за смещением человека влево/вправо.

Проект 3: Гонка по точкам

Задача: Пролететь через полосу препятствий (ворота), отмеченные ArUco-маркерами, как можно быстрее. **Усложнение:** Маркеры могут быть расположены на разной высоте. Дрон должен пролетать сквозь ворота (или над ними).

Критерии оценки проектов

1. **Стабильность:** Программа не падает с ошибками.
2. **Безопасность:** Дрон не совершает резких опасных маневров.
3. **Читаемость кода:** Код разбит на функции, есть комментарии.
4. **Функциональность:** Задача выполнена полностью.

Часть III.

Профессиональные навигационные системы

9. Теоретические основы визуальной навигации

9.1. Введение в ArUco и AprilTag

Визуальная навигация — это способ определения положения робота в пространстве с помощью анализа изображений. В отличие от GPS, который работает только на улице и имеет погрешность в несколько метров, визуальные маркеры позволяют дрону позиционироваться с точностью до сантиметра даже в закрытых помещениях.

ArUco (Augmented Reality University of Cordoba) и **AprilTag** — это самые популярные библиотеки фидуциальных (опорных) маркеров. Они представляют собой черно-белые квадраты, которые содержат закодированную информацию (ID маркера) и позволяют алгоритмам компьютерного зрения однозначно определить ориентацию маркера в пространстве.

9.2. Принцип работы алгоритма обнаружения

Процесс обнаружения маркера на изображении состоит из нескольких этапов. Понимание этого конвейера (pipeline) важно для диагностики проблем, когда дрон «не видит» маркеры.

9.2.1. 1. Предварительная обработка (Preprocessing)

Камера передает цветное изображение (RGB). Алгоритм сначала переводит его в оттенки серого (Grayscale), так как цвет не несет информации для черно-белых маркеров.

Затем применяется **адаптивная бинаризация** (Adaptive Thresholding). В отличие от простого порога (где все пиксели ярче 127 становятся белы-

ми, а темнее — черными), адаптивный метод вычисляет порог для каждой небольшой области изображения. Это позволяет находить маркеры даже при неравномерном освещении (например, когда одна часть маркера в тени).

Пример бинаризации (OpenCV)

```
import cv2

# ... (получение кадра frame) ...
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

# Адаптивный порог: (src, maxValue, adaptiveMethod, thresholdType,
#   ↪ blockSize, C)
binary = cv2.adaptiveThreshold(gray, 255,
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY_INV, 7, 2)
```

9.2.2. 2. Поиск контуров (Contour Extraction)

Алгоритм ищет границы связных областей белых пикселей на черном фоне. Из всех найденных контуров отбираются только те, которые:

- Являются замкнутыми.
- Аппроксимируются четырехугольником (имеют 4 вершины).
- Являются выпуклыми (Convex).
- Имеют достаточный размер (площадь).

9.2.3. 3. Исправление перспективы (Perspective Warping)

Найденный четырехугольник может быть искажен перспективой (выглядеть как трапеция или ромб), если камера смотрит на него под углом. Алгоритм вычисляет матрицу гомографии, чтобы «выпрямить» изображение и получить идеальный квадрат (Canonical Marker Image).

9.2.4. 4. Декодирование (Decoding)

Выпрямленный квадрат разбивается на сетку (например, 6×6 клеток). Каждая клетка анализируется: черная — 0, белая — 1. Полученная матрица битов сверяется со словарем (Dictionary).

9.3. Коррекция ошибок и Расстояние Хэмминга

Маркеры ArUco спроектированы так, чтобы быть устойчивыми к ошибкам считывания. Даже если несколько пикселей (битов) будут распознаны неверно (из-за блика или грязи), алгоритм все равно сможет узнать маркер.

Это достигается за счет избыточности информации. Каждый маркер имеет параметр **Hamming Distance** (Расстояние Хэмминга) — минимальное количество битов, на которое отличаются любые два маркера в словаре.

- Если словарь имеет Hamming Distance = 4, система может обнаружить до 3 ошибок и исправить до 1 ошибки.
- Чем больше размер словаря (например, 6×6 против 4×4), тем больше информации можно закодировать и тем выше устойчивость к ложным срабатываниям.

9.4. Сравнение типов словарей (Dictionaries)

При выборе словаря нужно искать баланс между количеством доступных ID и размером маркера (читаемостью).

Словарь	Размер сетки	Кол-во ID	Применение
DICT_4X4_50	4×4 бита	50	Дальние дистанции, низкое разрешение. Меньше деталей — легче распознать.
DICT_5X5_1000	5×5 бит	1000	Стандарт для большинства задач.
DICT_6X6_250	6×6 бит	250	Высокая надежность, используется в GeoScan Pioneer.
DICT_APRILTAG_36h11	6×6 бит	587	Промышленный стандарт, очень высокая устойчивость к ложным срабатываниям.

Таблица 2: Характеристики словарей маркеров

9.5. Математика позиционирования (PnP Problem)

После того как маркер найден и идентифицирован, нужно определить положение камеры относительно него. Эта задача называется **Perspective-n-Point (PnP)**.

Мы знаем:

1. Координаты 4 углов маркера в его собственной системе координат (3D object points): $(0, 0, 0)$, $(S, 0, 0)$, $(S, S, 0)$, $(0, S, 0)$, где S — размер маркера.
2. Координаты этих же 4 углов на изображении камеры (2D image points) в пикселях.

3. Внутренние параметры камеры (Camera Matrix и Distortion Coefficients), полученные при калибровке.

Функция `cv2.solvePnP` решает систему уравнений и возвращает два вектора:

- `rvec` (Rotation Vector) — вектор вращения (в формате Родригеса), показывающий наклон камеры.
- `tvec` (Translation Vector) — вектор смещения (x, y, z) , показывающий позицию маркера относительно центра камеры.

! Важно

Важно! Вектор `tvec` показывает координаты *маркера в системе координат камеры*. Чтобы получить координаты *дрона в системе координат маркера* (для навигации), нужно инвертировать эту трансформацию:

$$R = Rodrigues(rvec)$$

$$Position_{drone} = -R^T \cdot tvec$$

9.6. Практические задания

Задание 57: Задание 1: Анализ словарей

Сгенерируйте маркер ID=1 из словарей 4x4, 5x5 и 6x6 одного физического размера (например, 200 пикселей). 1. Распечатайте их. 2. Отойдите с камерой на максимальное расстояние. 3. Какой маркер перестанет распознаваться первым? Почему?

Задание 58: Задание 2: Влияние освещения

Запустите скрипт обнаружения маркера. 1. Постепенно затемняйте комнату. На каком уровне освещенности маркер пропадет? 2. Посветите фонариком прямо на маркер (создайте блик). Как это влияет на бинаризацию (посмотрите окно с ч/б изображением)?

? Контрольные вопросы

1. Почему для навигации не используются QR-коды? (Подсказка: сравните сложность детектирования угла наклона).
2. Что такое «адаптивная бинаризация» и зачем она нужна?
3. Если дрон видит маркер под углом 45 градусов, как алгоритм понимает, что это квадрат, а не ромб?
4. В чем физический смысл вектора $tvec$?

10. Создание навигационных карт

10.1. Калибровка камеры

Перед тем как создавать карту, необходимо откалибровать камеру. Камера дрона — это не идеальный оптический прибор. Линзы (особенно широкоугольные «рыбий глаз») вносят искажения: прямые линии на краях кадра могут выглядеть изогнутыми. Это называется **дисторсией** (Distortion).

Если не учесть эти искажения, координаты маркеров будут вычислены с ошибкой в десятки сантиметров.

10.1.1. Процесс калибровки

Для калибровки используется специальный шаблон — шахматная доска (Chessboard).

1. Распечатайте шаблон (например, 9×6 клеток) на жестком листе.
2. Запустите скрипт калибровки (обычно доступен в примерах OpenCV).
3. Покажите доску камере под разными углами (20-30 кадров).
4. Скрипт вычислит:
 - **Camera Matrix:** Фокусное расстояние (f_x, f_y) и оптический центр (c_x, c_y) .
 - **Distortion Coefficients:** Коэффициенты радиальной (k_1, k_2, k_3) и тангенциальной (p_1, p_2) дисторсии.

Полученные параметры нужно сохранить в файл (например, `camera_params.yaml`) и загружать при запуске навигации.

Скрипт калибровки камеры

```
#!/usr/bin/python3

"""
Performs camera calibration using checkerboard pattern. Saves
→ calibration
values into a file.
More: https://docs.opencv.org/4.x/dc/dbb/tutorial\_py\_calibration.html
"""

import cv2
import numpy as np
import glob
import time
```

```

from pioneer_sdk import Camera

def get_images_from_folder(folder_name, file_type="*.jpg"):
    """
    Retrieve images from a directory using a provided path
    ↪ (`folder_name`)
    """
    images_list = glob.glob(folder_name + "/" + file_type)
    images = []
    for fname in images_list:
        img = cv2.imread(fname)
        images.append(img)
    return images

def get_images_from_drone_camera(camera, save=False):
    """
    Retrieve images from a drone's camera
    """
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        frame = camera.get_cv_frame()
        if frame is not None:
            cv2.imshow("frames", frame)

            key = cv2.waitKey(1)
            if key == 27 or key == ord("q"):
                cv2.destroyAllWindows()
                return images
            if key == ord("1"):
                images.append(frame)
                print("Frame #" + str(len(images)))
                if save:
                    cv2.imwrite("frame_" + str(len(images)) + ".jpg",
                                ↪ frame)
                time.sleep(1)

def get_images_from_computer_camera(capture, save=False):
    """
    Retrieve images using the PC's camera device
    """
    images = []
    print("Press '1' to take frame\nPress ESC or 'q' to finish")
    while True:
        ret, frame = capture.read()
        cv2.imshow("frames", frame)

        key = cv2.waitKey(1)

```

```

    if key == 27 or key == ord("q"):
        cv2.destroyAllWindows()
        return images
    if key == ord("1"):
        images.append(frame)
        print("Frame #" + str(len(images)))
        if save:
            cv2.imwrite("frame_" + str(len(images)) + ".jpg", frame)
            time.sleep(1)

def calibrate(images):
    # Defining the dimensions of checkerboard
    CHECKERBOARD = (6, 9)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30,
        ↪ 0.001)

    # Creating vector to store vectors of 3D points for each
    ↪ checkerboard image
    objpoints = []
    # Creating vector to store vectors of 2D points for each
    ↪ checkerboard image
    imgpoints = []

    # Defining the world coordinates for 3D points
    objp = np.zeros((1, CHECKERBOARD[0] * CHECKERBOARD[1], 3),
        ↪ np.float32)
    objp[0, :, :2] = np.mgrid[0 : CHECKERBOARD[0], 0 :
        ↪ CHECKERBOARD[1]].T.reshape(-1, 2)

    # Extracting path of individual image stored in a given directory
    for img in images:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # Find the chess board corners
        # If desired number of corners are found in the image then ret =
        ↪ true
        ret, corners = cv2.findChessboardCorners(
            gray,
            CHECKERBOARD,
            cv2.CALIB_CB_ADAPTIVE_THRESH
            + cv2.CALIB_CB_FAST_CHECK
            + cv2.CALIB_CB_NORMALIZE_IMAGE,
        )

        if ret == True:
            objpoints.append(objp)
            # refining pixel coordinates for given 2d points.
            corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1,
                ↪ -1), criteria)

            imgpoints.append(corners2)

```

```

        # Draw and display the corners
        img = cv2.drawChessboardCorners(img, CHECKERBOARD, corners2,
        ↪ ret)

        cv2.imshow("img", img)
        cv2.waitKey(0)

    cv2.destroyAllWindows()

    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
        objpoints, imgpoints, gray.shape[:::-1], None, None
    )

    return mtx, dist

def save_coefficients(mtx, dist, path):
    """Save the camera matrix and the distortion coefficients to given
    ↪ path/file."""
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_WRITE)
    cv_file.write("mtx", mtx)
    cv_file.write("dist", dist)
    # note you *release* you don't close() a FileStorage object
    cv_file.release()

def load_coefficients(path):
    """Loads camera matrix and distortion coefficients."""
    # FILE_STORAGE_READ
    cv_file = cv2.FileStorage(path, cv2.FILE_STORAGE_READ)

    # note we also have to specify the type to retrieve other wise we
    ↪ only get a
    # FileNode object back instead of a matrix
    camera_matrix = cv_file.getNode("mtx").mat()
    dist_coeffs = cv_file.getNode("dist").mat()

    cv_file.release()
    return camera_matrix, dist_coeffs

def main():
    """
    В этом примере изображения берутся с камеры дрона.
    Однако вы можете изменить источник изображений, заменив
    функцию `get_images_from_drone_camera` на другую, например:
    ...

    # Загрузить изображения из папки
    images = get_images_from_folder('images')

```

```

# ...
'''

'''
# Получить изображения с веб-камеры компьютера
capture = cv2.VideoCapture(0)
images = get_images_from_computer_camera(capture)
capture.release()
# ...
'''

'''
# Acquire images from the drone's camera
camera = Camera()
images = get_images_from_drone_camera(camera)

mtx, dist = calibrate(images)

save_coefficients(mtx, dist, "data.yml")

if __name__ == "__main__":
    main()

```

10.2. Генерация и печать маркеров

Используйте скрипт `19_map_creation_example2_generation.py` для создания изображений.

Генерация маркеров ArUco/AprilTag

```

import cv2
import cv2.aruco as aruco
import numpy as np
import os

# Create directory for output if it doesn't exist
output_dir = "../../images/markers"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# Define dictionaries to generate
dictionaries = {
    "DICT_4X4_50": aruco.DICT_4X4_50,
    "DICT_5X5_100": aruco.DICT_5X5_100,
    "DICT_6X6_250": aruco.DICT_6X6_250,
    "DICT_7X7_1000": aruco.DICT_7X7_1000,
    "DICT_APRILTAG_36h11": aruco.DICT_APRILTAG_36h11
}

```

```

}

def add_visual_aids(image, size_pixels):
    """
    Adds coordinate axes and grid to the marker image for educational
    ↪ purposes.
    """
    # Convert to color to add colored axes
    color_img = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)

    # Draw border
    border_size = int(size_pixels * 0.1)
    color_img = cv2.copyMakeBorder(color_img, border_size, border_size,
    ↪ border_size, border_size,
                                cv2.BORDER_CONSTANT, value=[255, 255,
    ↪ 255])

    new_size = color_img.shape[0]
    center = new_size // 2

    # Draw Axes (X - Red, Y - Green)
    # Origin is top-left of the marker content (inside border)
    origin_x = border_size
    origin_y = border_size

    # X Axis
    cv2.arrowedLine(color_img, (origin_x, origin_y), (origin_x + 100,
    ↪ origin_y), (0, 0, 255), 2)
    cv2.putText(color_img, "X", (origin_x + 110, origin_y + 10),
    ↪ cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)

    # Y Axis
    cv2.arrowedLine(color_img, (origin_x, origin_y), (origin_x, origin_y
    ↪ + 100), (0, 255, 0), 2)
    cv2.putText(color_img, "Y", (origin_x + 10, origin_y + 110),
    ↪ cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    return color_img

def generate_marker(dict_name, marker_id, size_pixels=200,
    ↪ visual_aids=False):
    """
    Generates a single ArUco marker image.
    """
    try:
        aruco_dict =
        ↪ aruco.getPredefinedDictionary(dictionaries[dict_name])

        # Draw the marker
        marker_image = np.zeros((size_pixels, size_pixels),
        ↪ dtype=np.uint8)

```

```

marker_image = aruco.generateImageMarker(aruco_dict, marker_id,
    ↪ size_pixels)

if visual_aids:
    final_image = add_visual_aids(marker_image, size_pixels)
    suffix = "_viz"
else:
    final_image = marker_image
    suffix = ""

# Save the marker
filename = f"{output_dir}/{dict_name}_id{marker_id}{suffix}.png"
cv2.imwrite(filename, final_image)
print(f"Generated: {filename}")
except Exception as e:
    print(f"Error generating marker {dict_name} ID {marker_id}:
    ↪ {e}")

def generate_json_metadata(dict_name, count, size_pixels):
    import json

    # Infer bits from name (e.g. DICT_4X4_50 -> 4)
    bits = 0
    if "4X4" in dict_name: bits = 4
    elif "5X5" in dict_name: bits = 5
    elif "6X6" in dict_name: bits = 6
    elif "7X7" in dict_name: bits = 7
    elif "APRILTAG" in dict_name: bits = "variable"

    data = {
        "dictionary": dict_name,
        "bits": bits,
        "count": count,
        "size_pixels": size_pixels,
        "description": "Marker set for Geoskan Pioneer"
    }
    filename = f"{output_dir}/{dict_name}_metadata.json"
    with open(filename, 'w') as f:
        json.dump(data, f, indent=4)
    print(f"Metadata saved: {filename}")

def generate_batch(dict_name, start_id, count, size_pixels=200,
    ↪ visual_aids=False):
    """
    Generates a batch of markers.
    """
    print(f"--- Generating {count} markers for {dict_name} ---")
    for i in range(start_id, start_id + count):
        generate_marker(dict_name, i, size_pixels, visual_aids)

    generate_json_metadata(dict_name, count, size_pixels)

```

```

if __name__ == "__main__":
    # 1. Standard sets
    generate_batch("DICT_4X4_50", 0, 5)
    generate_batch("DICT_5X5_100", 0, 2)
    generate_batch("DICT_6X6_250", 0, 5) # Standard for Pioneer
    generate_batch("DICT_7X7_1000", 0, 2)
    generate_batch("DICT_APRILTAG_36h11", 0, 5)

    # 2. Visualization examples (with axes)
    generate_marker("DICT_6X6_250", 0, 300, visual_aids=True)
    generate_marker("DICT_APRILTAG_36h11", 0, 300, visual_aids=True)

    # 3. Full range examples (uncomment to generate large sets)
    # generate_batch("DICT_6X6_250", 0, 50)
    # generate_batch("DICT_APRILTAG_36h11", 0, 50)

    print(f"Done! Markers saved in '{output_dir}' folder.")

```

10.2.1. Векторные маркеры

Для печати в типографии или создания масштабируемых карт рекомендуется использовать векторные форматы (PDF/SVG). Следующий скрипт генерирует маркеры с использованием векторной графики.

Генерация векторных маркеров

```

import cv2
import cv2.aruco as aruco
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
import os

# Define output directory
output_dir = "../..../images/markers/vector"
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# Helper function to generate raw bits of a marker
def get_marker_bits(dictionary, marker_id, side_pixels):
    """
    Generates a marker image with the exact pixel size matching its bit
    ↪ grid.
    For ArUco 4x4, the grid is 4x4 + 1 bit border = 6x6.
    For ArUco 6x6, the grid is 6x6 + 1 bit border = 8x8.
    For AprilTag 36h11, the grid is 8x8 + 1 bit border = 10x10.
    """

```

```

# Create a small image (e.g. 10x10) to get raw bits
# generateImageMarker scales up, but if we request small size it
  ↳ should work
# OpenCV's generateImageMarker takes sidePixels as argument.
# If sidePixels is small, it returns the raw bits.
img = aruco.generateImageMarker(dictionary, marker_id, side_pixels)
return img

def draw_vector_marker(ax, img, x_offset=0, y_offset=0, size=1.0):
    """
    Draws a marker on a matplotlib axis using Rectangle patches (Vector
    ↳ format).
    """
    rows, cols = img.shape
    pixel_size = size / rows

    # Draw white background
    rect = patches.Rectangle((x_offset, y_offset), size, size,
        ↳ linewidth=0, facecolor='white')
    ax.add_patch(rect)

    # Draw black pixels
    for r in range(rows):
        for c in range(cols):
            if img[r, c] == 0:
                # In matplotlib, origin is bottom-left usually, but we
                ↳ can invert axis
                # Let's assume standard image coordinates (0,0 is
                ↳ top-left) and invert Y later or now
                # Drawing from top-left (y = size - ...)
                rect_x = x_offset + c * pixel_size
                rect_y = y_offset + (rows - 1 - r) * pixel_size

                rect = patches.Rectangle((rect_x, rect_y), pixel_size,
                    ↳ pixel_size, linewidth=0, facecolor='black')
                ax.add_patch(rect)

def save_single_marker(dict_name, marker_id, bit_size, filename):
    dictionary = aruco.getPredefinedDictionary(getattr(aruco,
        ↳ dict_name))
    img = get_marker_bits(dictionary, marker_id, bit_size)

    fig, ax = plt.subplots(figsize=(4, 4))
    draw_vector_marker(ax, img, size=10)

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)
    ax.axis('off')
    ax.set_aspect('equal')

```

```

plt.savefig(filename, format='pdf', bbox_inches='tight',
    ↪ pad_inches=0)
plt.close()
print(f"Saved vector marker: {filename}")

def save_grid_map(dict_name, rows, cols, bit_size, filename):
    dictionary = aruco.getPredefinedDictionary(getattr(aruco,
    ↪ dict_name))

    # A4 Page size approx (8.27 x 11.69 inches)
    fig, ax = plt.subplots(figsize=(8.27, 11.69))

    margin_x = 1
    margin_y = 1
    cell_width = (8.27 - 2 * margin_x) / cols
    cell_height = (11.69 - 2 * margin_y) / rows
    marker_size = min(cell_width, cell_height) * 0.8

    # Start drawing from top-left
    start_x = margin_x
    start_y = 11.69 - margin_y - cell_height

    for i in range(rows):
        for j in range(cols):
            marker_id = i * cols + j
            img = get_marker_bits(dictionary, marker_id, bit_size)

            x = start_x + j * cell_width + (cell_width - marker_size) /
            ↪ 2
            y = start_y - i * cell_height + (cell_height - marker_size)
            ↪ / 2

            draw_vector_marker(ax, img, x_offset=x, y_offset=y,
            ↪ size=marker_size)

            # Add Label
            ax.text(x + marker_size/2, y - 0.2, f"ID: {marker_id}",
            ↪ ha='center', va='top', fontsize=8)

    ax.set_xlim(0, 8.27)
    ax.set_ylim(0, 11.69)
    ax.axis('off')
    ax.set_aspect('equal')

    plt.savefig(filename, format='pdf', bbox_inches='tight')
    plt.close()
    print(f"Saved vector grid: {filename}")

if __name__ == "__main__":
    # 1. Single Markers
    # ArUco 4x4 (Size 6x6 pixels)

```

```
save_single_marker("DICT_4X4_50", 0, 6,  
    ↪ f"{output_dir}/aruco_4x4_id0.pdf")  
# AprilTag 36h11 (Size 10x10 pixels usually? Let's verify)  
# AprilTag 36h11 is 8x8 data + 1 bit border = 10x10 total  
save_single_marker("DICT_APRILTAG_36h11", 0, 10,  
    ↪ f"{output_dir}/apriltag_id0.pdf")  
  
# 2. Grid Maps (4x4 = 16 markers)  
save_grid_map("DICT_4X4_50", 4, 4, 6,  
    ↪ f"{output_dir}/aruco_grid_4x4.pdf")  
save_grid_map("DICT_APRILTAG_36h11", 4, 4, 10,  
    ↪ f"{output_dir}/apriltag_grid_4x4.pdf")
```

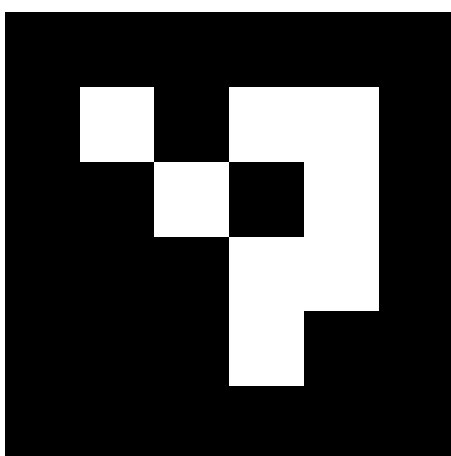


Рис. 10: ArUco 4x4 (Вектор)

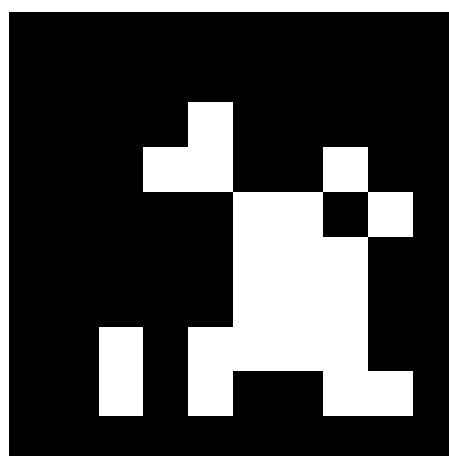


Рис. 11: AprilTag 36h11 (Вектор)

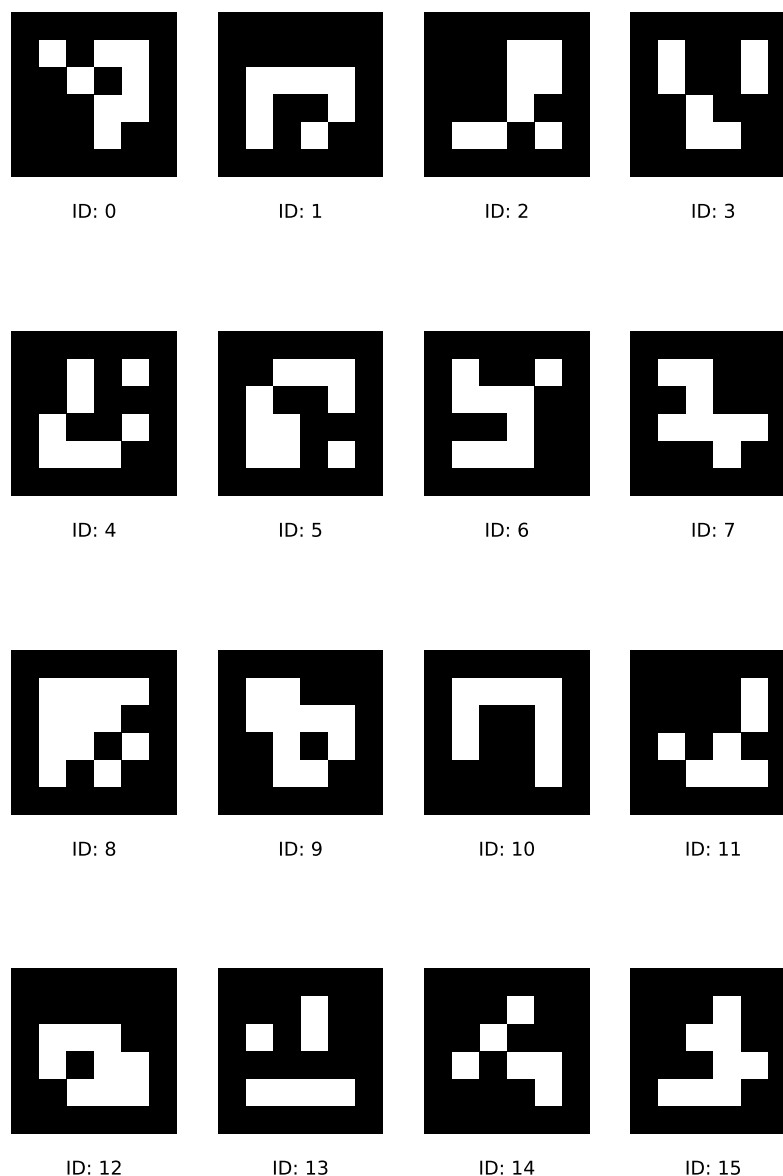


Рис. 12: Карта маркеров 4x4 (ArUco, вектор)

⚠ Внимание

Важно! При печати в настройках принтера выберите «Реальный размер» (Actual Size) или «Масштаб 100%». Иначе размер маркера изменится, и навигация будет врать.

10.2.2. Требования к монтажу

1. **Жесткость:** Маркер должен быть идеально плоским. Изгиб бумаги даже на 5 мм приведет к ошибке определения угла (Yaw/Pitch/Roll) на 10-15 градусов. Наклейте маркер на картон, пластик или фанеру.
2. **Quiet Zone:** Вокруг черной рамки маркера обязательно должно быть белое поле шириной не менее одного бита. Если наклеить черный маркер на черный пол, он не распознается.
3. **Матовое покрытие:** Глянцевая бумага или скотч создают блики от ламп. В месте блика камера видит белое пятно вместо черного пикселя. Используйте матовую ламинацию или матовый скотч.
4. **Освещение:** Избегайте размещения маркеров в глубокой тени или напротив ярких окон (контражур).

10.3. Форматы карт (JSON, XML, KML)

Карта маркеров — это файл, описывающий положение каждого маркера. Наш инструмент `19_map_creation_example3_builder.py` поддерживает несколько форматов.

Инструмент создания карт

```
import json
import math
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.path import Path
import numpy as np

class Marker:
    def __init__(self, marker_id, size, x, y, z, yaw=0,
        ↪ dictionary="DICT_6X6_250"):
        self.id = marker_id
        self.size = size # in meters
        self.x = x # local x (meters)
        self.y = y # local y (meters)
        self.z = z # local z (meters) - height
        self.yaw = yaw # orientation (degrees)
        self.dictionary = dictionary

    def to_dict(self):
        return {
            "id": self.id,
```

```

        "size": self.size,
        "position": {"x": self.x, "y": self.y, "z": self.z},
        "orientation": {"yaw": self.yaw},
        "dictionary": self.dictionary
    }

class MarkerMap:
    def __init__(self, name="MyMap", origin_gps=(0, 0)):
        self.name = name
        self.origin_lat = origin_gps[0]
        self.origin_lon = origin_gps[1]
        self.markers = []

    def add_marker(self, marker):
        self.markers.append(marker)

    def generate_grid(self, rows, cols, step_x, step_y, start_id=0,
        ↪ marker_size=0.2, dictionary="DICT_4X4_50"):
        """
        Generates a grid of markers.
        """
        for i in range(rows):
            for j in range(cols):
                # Calculate position
                # Assuming grid starts at (0,0) and extends in +X and
                ↪ +Y
                x = i * step_x
                y = j * step_y
                z = 0 # On the floor

                marker_id = start_id + i * cols + j

                self.add_marker(Marker(
                    marker_id=marker_id,
                    size=marker_size,
                    x=x,
                    y=y,
                    z=z,
                    yaw=0,
                    dictionary=dictionary
                ))
            print(f"Generated grid: {rows}x{cols}, {len(self.markers)}
            ↪ markers.")

    def save_yaml(self, filename):
        """
        Exports map to YAML format (Clever 4 style).
        """
        import yaml

        # Prepare data structure for ROS/Clever

```

```

# Usually it's a list of markers with id, size, frame_id, pose
markers_data = []
for m in self.markers:
    markers_data.append({
        "id": int(m.id),
        "size": float(m.size),
        "frame_id": "map", # Global frame
        "pose": {
            "position": {
                "x": float(m.x),
                "y": float(m.y),
                "z": float(m.z)
            },
            "orientation": {
                # Simplified quaternion for yaw rotation around
                ↪ Z
                "x": 0.0,
                "y": 0.0,
                "z": math.sin(math.radians(m.yaw)/2),
                "w": math.cos(math.radians(m.yaw)/2)
            }
        }
    })

with open(filename, 'w') as f:
    yaml.dump({"markers": markers_data}, f,
        ↪ default_flow_style=False)
    print(f"Map YAML saved to {filename}")

def save_png_grid(self, filename, resolution_px=4096):
    """
    Exports the map as a high-resolution PNG grid (Clever 4 style).
    """
    import cv2
    import cv2.aruco as aruco

    # Determine map bounds
    xs = [m.x for m in self.markers]
    ys = [m.y for m in self.markers]
    min_x, max_x = min(xs), max(xs)
    min_y, max_y = min(ys), max(ys)

    # Add padding (e.g., 1 meter)
    padding_m = 1.0
    width_m = (max_x - min_x) + 2 * padding_m
    height_m = (max_y - min_y) + 2 * padding_m

    # Calculate scale (pixels per meter)
    # We want the largest dimension to fit in resolution_px
    scale = resolution_px / max(width_m, height_m)

```

```

# Create white image
img_h = int(height_m * scale)
img_w = int(width_m * scale)
image = np.ones((img_h, img_w), dtype=np.uint8) * 255

print(f"Generating PNG map: {img_w}x{img_h} pixels (scale:
↳ {scale:.1f} px/m)")

# Draw markers
for m in self.markers:
    # Get marker dictionary
    try:
        # Map string dictionary name to cv2.aruco constant
        if hasattr(aruco, m.dictionary):
            adict = aruco.getPredefinedDictionary(getattr(aruco,
↳ m.dictionary))
        else:
            # Fallback to default
            adict =
↳ aruco.getPredefinedDictionary(aruco.DICT_6X6_250)

    # Generate marker image
    marker_size_px = int(m.size * scale)
    marker_img = aruco.generateImageMarker(adict, m.id,
↳ marker_size_px)

    # Calculate position in image
    # Transform (x,y) from center-based to top-left image
    ↳ coordinates
    # Image origin is top-left. Map origin (0,0) is
    ↳ somewhere in the middle.
    # x_img = (m.x - min_x + padding) * scale
    # y_img = (max_y - m.y + padding) * scale (invert Y for
    ↳ image coords if needed, but typically maps align
    ↳ X-right Y-up vs image X-right Y-down)

    # Let's assume standard map: X right, Y up. Image: X
    ↳ right, Y down.
    # So we flip Y.

    center_x_px = int((m.x - min_x + padding_m) * scale)
    center_y_px = int((max_y - m.y + padding_m) * scale) #
    ↳ Invert Y

    top_left_x = center_x_px - marker_size_px // 2
    top_left_y = center_y_px - marker_size_px // 2

    # Place marker
    if 0 <= top_left_y < img_h - marker_size_px and 0 <=
    ↳ top_left_x < img_w - marker_size_px:

```

```

        image[top_left_y:top_left_y+marker_size_px,
        ↪ top_left_x:top_left_x+marker_size_px] =
        ↪ marker_img

        # Add simple text ID for verification (optional,
        ↪ might clutter)
        # cv2.putText(image, str(m.id), (top_left_x,
        ↪ top_left_y - 10), cv2.FONT_HERSHEY_SIMPLEX, 1,
        ↪ 0, 2)

    except Exception as e:
        print(f"Error drawing marker {m.id}: {e}")

    cv2.imwrite(filename, image)
    print(f"Map PNG saved to {filename}")

def save_json(self, filename):
    data = {
        "map_name": self.name,
        "origin": {"lat": self.origin_lat, "lon": self.origin_lon},
        "markers": [m.to_dict() for m in self.markers]
    }
    with open(filename, 'w') as f:
        json.dump(data, f, indent=4)
    print(f"Map saved to {filename}")

def save_pdf(self, filename, trajectory=None):
    """
    Exports the map to a PDF file for printing/visualization.
    Includes marker positions, IDs, and optional trajectory.
    """
    fig, ax = plt.subplots(figsize=(11.69, 8.27)) # A4 Landscape
    ↪ (inches)

    # Plot markers
    x_coords = []
    y_coords = []

    for m in self.markers:
        x_coords.append(m.x)
        y_coords.append(m.y)

        # Draw marker as a square
        # m.x, m.y is the center of the marker
        half_size = m.size / 2
        rect = patches.Rectangle(
            (m.x - half_size, m.y - half_size),
            m.size, m.size,
            linewidth=1, edgecolor='black', facecolor='none'
        )
        ax.add_patch(rect)

```

```

# Add ID text
ax.text(m.x, m.y, str(m.id), ha='center', va='center',
        ↪ fontsize=8, fontweight='bold')

# Add orientation arrow (small)
# Arrow pointing in yaw direction
arrow_len = m.size * 0.4
dx = arrow_len * math.cos(math.radians(m.yaw))
dy = arrow_len * math.sin(math.radians(m.yaw))
ax.arrow(m.x, m.y, dx, dy, head_width=m.size*0.1,
        ↪ head_length=m.size*0.1, fc='blue', ec='blue', alpha=0.5)

# Plot trajectory if provided
if trajectory:
    traj_x = [p[0] for p in trajectory]
    traj_y = [p[1] for p in trajectory]
    ax.plot(traj_x, traj_y, 'r--', label='Drone Trajectory',
            ↪ linewidth=2, alpha=0.7)

# Mark start and end
ax.scatter(traj_x[0], traj_y[0], c='green', marker='o',
        ↪ label='Start')
ax.scatter(traj_x[-1], traj_y[-1], c='red', marker='x',
        ↪ label='End')
ax.legend()

# Set plot Limits with some padding
if x_coords and y_coords:
    ax.set_xlim(min(x_coords) - 1, max(x_coords) + 1)
    ax.set_ylim(min(y_coords) - 1, max(y_coords) + 1)

ax.set_aspect('equal')
ax.set_xlabel('X (meters)')
ax.set_ylabel('Y (meters)')
ax.set_title(f'Marker Map: {self.name}')
ax.grid(True, linestyle=':', alpha=0.6)

# Add metadata text
info_text = f"Total Markers: {len(self.markers)}\nOrigin:
        ↪ {self.origin_lat}, {self.origin_lon}"
plt.figtext(0.02, 0.02, info_text, fontsize=10)

plt.savefig(filename, format='pdf')
plt.close()
print(f"Map visualization saved to {filename}")

def save_kml(self, filename):
    """
    Simple KML export for Google Earth.

```

```

    Note: This uses a rough approximation for meters to degrees (1
    ↪ deg lat ~ 111km).
    For production, use a proper geodetic library like 'pyproj'.
    """

    kml_header = """<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <name>{name}</name>
    """

    kml_footer = """  </Document>
</kml>"""

    kml_body = ""

    # Approximate conversion (very rough, valid only for small
    ↪ areas)
    lat_deg_per_meter = 1 / 111132.928
    lon_deg_per_meter = 1 / (111132.928 *
    ↪ math.cos(math.radians(self.origin_lat)))

    for m in self.markers:
        # Calculate GPS position from local offset
        m_lat = self.origin_lat + (m.x * lat_deg_per_meter)
        m_lon = self.origin_lon + (m.y * lon_deg_per_meter)

        kml_body += f"""
<Placemark>
  <name>Marker {m.id}</name>
  <description>Size: {m.size}m, Yaw: {m.yaw}</description>
  <Point>
    <coordinates>{m_lon},{m_lat},{m.z}</coordinates>
  </Point>
</Placemark>"""

    with open(filename, 'w') as f:
        f.write(kml_header.format(name=self.name) + kml_body +
        ↪ kml_footer)
    print(f"KML saved to {filename}")

    def save_xml(self, filename):
        """
        Generic XML export.
        """
        xml_content = f"""<?xml version="1.0" encoding="UTF-8"?>
<map>
  <name>{self.name}</name>
  <origin>
    <lat>{self.origin_lat}</lat>
    <lon>{self.origin_lon}</lon>
  </origin>
  <markers>

```

```

"""
    for m in self.markers:
        xml_content += f"""          <marker id="{m.id}">
            <size>{m.size}</size>
            <position>
                <x>{m.x}</x>
                <y>{m.y}</y>
                <z>{m.z}</z>
            </position>
            <orientation>
                <yaw>{m.yaw}</yaw>
            </orientation>
            <dictionary>{m.dictionary}</dictionary>
        </marker>
    """

    xml_content += """    </markers>
</map>"""

    with open(filename, 'w') as f:
        f.write(xml_content)
    print(f"XML saved to {filename}")

if __name__ == "__main__":
    # Example: Create a grid map for a flight zone (Clever 4 Style)
    # Origin: Let's assume a field coordinate
    my_map = MarkerMap("Clever_Map_16x16", origin_gps=(59.9343,
        ↪ 30.3351)) # St. Petersburg center approx

    # Generate a 16x16 grid with 0.5m spacing, marker size 0.2m
    # Dictionary for Clever 4 is usually 4x4_50 or 6x6_250 depending on
    ↪ setup. Let's stick to 4x4_50 as per common educational kits or
    ↪ 6x6_250 for Pioneer.
    # User request mentions "0-999 for ArUco", which implies larger
    ↪ dictionary. But usually grid maps use a subset.
    # Let's use DICT_4X4_50 as it is common for small grids, or extend
    ↪ if needed. 16x16 = 256 markers. DICT_4X4_50 has only 50 markers!
    # We need a larger dictionary. DICT_6X6_250 has 250 markers (almost
    ↪ enough, need 256).
    # DICT_5X5_1000 or DICT_6X6_1000 is safer. Pioneer standard is
    ↪ DICT_6X6_50 (only 50!).
    # If we need 16x16=256 unique markers, we MUST use a larger
    ↪ dictionary.
    # Let's use DICT_6X6_1000 to be safe and cover 0-255 IDs.

    my_map.generate_grid(rows=16, cols=16, step_x=0.5, step_y=0.5,
        ↪ start_id=0, marker_size=0.2, dictionary="DICT_6X6_1000")

    # Define a dummy trajectory (square path)
    trajectory = [
        (0.5, 0.5, 1.0),
        (3.5, 0.5, 1.0),

```

```

        (3.5, 3.5, 1.0),
        (0.5, 3.5, 1.0),
        (0.5, 0.5, 1.0)
    ]

    # Export standard formats
    my_map.save_json("../data/maps/field_map.json")
    my_map.save_pdf("../data/maps/field_map.pdf",
        ↪ trajectory=trajectory)

    # Export Clever 4 style artifacts
    my_map.save_yaml("../data/maps/clever_map.yaml")
    my_map.save_png_grid("../data/maps/clever_map_4096.png",
        ↪ resolution_px=4096)

    # Save config
    config = {
        "grid": {
            "rows": 16,
            "cols": 16,
            "step_x": 0.5,
            "step_y": 0.5
        },
        "marker": {
            "size": 0.2,
            "dictionary": "DICT_6X6_1000"
        }
    }
    with open("../data/maps/map_config.json", 'w') as f:
        json.dump(config, f, indent=4)
        print("Config saved to ../data/maps/map_config.json")

    # my_map.save_kml("field_map.kml") # Optional
    # my_map.save_xml("field_map.xml") # Optional

```

10.3.1. JSON (JavaScript Object Notation)

Основной формат для роботов. Легко читается программами на Python/C++.

Пример карты JSON

```

{
    "map_name": "Warehouse_Zone_A",
    "origin": {"lat": 59.9, "lon": 30.3},
    "markers": [
        {
            "id": 10,
            "size": 0.2,

```

```

    "position": {"x": 5.0, "y": 0.0, "z": 1.5},
    "orientation": {"yaw": 90.0}
  }
]
}

```

10.3.2. XML (eXtensible Markup Language)

Более строгий формат, часто используется в промышленных системах.

Пример карты XML

```

<map>
  <name>Warehouse_Zone_A</name>
  <marker id="10">
    <size>0.2</size>
    <position>
      <x>5.0</x> <y>0.0</y> <z>1.5</z>
    </position>
  </marker>
</map>

```

10.3.3. KML (Keyhole Markup Language)

Формат для отображения на географических картах (Google Earth, QGIS). Позволяет визуализировать схему расположения маркеров поверх спутниковых снимков. Это полезно для планирования полетов на улице.

10.4. Визуализация и планирование

Для больших полигонов рекомендуется сначала создать план в CAD-системе или нарисовать схему на миллиметровке, определить координаты установки маркеров, и только потом заносить их в файл карты.

Задание 59: Задание 3: Калибровка

Проведите процедуру калибровки камеры Pioneer. 1. Сделайте 20 снимков шахматной доски. 2. Запустите скрипт калибровки. 3. Сравните изображение до и после устранения дисторсии (функция `cv2.undistort`). Прямые ли линии косяков дверей на краях кадра?

Задание 60: Задание 4: Создание полигона

Создайте карту для полета «восьмеркой». 1. Нарисуйте траекторию на полу мелом. 2. Разместите 2 маркера в центрах петель восьмерки. 3. Измерьте их координаты рулеткой относительно точки старта. 4. Создайте JSON файл карты.

❓ Контрольные вопросы

1. Что такое дисторсия и почему она мешает навигации?
2. Почему нельзя ламинировать маркеры глянцевой пленкой?
3. Зачем нужно белое поле вокруг маркера?
4. В чем преимущество формата KML перед JSON?

11. Алгоритмы комплексной навигации

11.1. Проблема шума и фильтрация

Данные, получаемые от системы компьютерного зрения, всегда содержат шум. Координаты дрона «дрожат» даже если он висит неподвижно. Это происходит из-за дискретности матрицы камеры, изменений освещения и вибраций корпуса.

Если передавать эти «сырые» координаты напрямую в полетный контроллер, дрон будет дергаться, пытаясь отработать каждое ложное смещение.

11.1.1. Фильтр Калмана (Kalman Filter)

Это стандарт де-факто в робототехнике. Фильтр Калмана не просто усредняет значения, он предсказывает, где дрон *должен* оказаться в следующий момент времени, и корректирует это предсказание на основе новых измерений.

Алгоритм работает циклически в два этапа:

1. **Prediction (Предсказание):** На основе физической модели (скорость, ускорение) мы оцениваем новую позицию. При этом растет неопределенность (мы не уверены в прогнозе).
2. **Update (Коррекция):** Мы получаем измерение от камеры. Сравниваем его с прогнозом. Чем точнее датчик (меньше шум измерения R), тем больше мы верим ему. Чем точнее наша модель (меньше шум процесса Q), тем больше верим прогнозу.

Пример 1D фильтра Калмана

```
# См. полный код в examples/advanced_navigation/kalman_filter.py
def update(self, measurement):
    # 1. Prediction
    x_pred = self.x
    P_pred = self.P + self.Q

    # 2. Update
    K = P_pred / (P_pred + self.R) # Kalman Gain
    self.x = x_pred + K * (measurement - x_pred)
    self.P = (1 - K) * P_pred
    return self.x
```

11.2. Системы координат

Одна из самых частых ошибок новичков — путаница в системах координат.

- **Body Frame (Связанная система):** Ось X всегда смотрит вперед (нос дрона), Y — влево/вправо, Z — вверх. Если дрон поворачивается, оси поворачиваются вместе с ним.
- **World Frame / NED (North-East-Down):** Ось X смотрит на Север, Y — на Восток, Z — Вниз (к земле). Эта система неподвижна.
- **Marker Frame:** Ось Z обычно смотрит перпендикулярно плоскости маркера (вверх от него).

Для навигации нам нужно переводить координаты маркера (полученные камерой в Body Frame) в глобальные координаты дрона (World Frame).

$$Pos_{world} = Pos_{drone} + R_{drone} \cdot Pos_{marker_relative}$$

11.3. Точная посадка с PID-регулятором

Для автоматической посадки на зарядную станцию используется PID-регулятор.

- **P (Proportional):** Чем дальше мы от центра, тем сильнее наклоняемся к нему.
- **I (Integral):** Если нас сносит ветром, мы накапливаем ошибку и наклоняемся сильнее, чтобы компенсировать снос.
- **D (Derivative):** Если мы летим к центру слишком быстро, мы «тормозим» заранее, чтобы не пролететь мимо.

Задание 61: Задание 5: Сравнение фильтров

Запустите скрипт `kalman_filter.py`. 1. Измените `measurement_noise` (шум измерения) с 0.1 до 5.0. Как ведет себя фильтр? 2. Сравните работу фильтра Калмана и простого скользящего среднего (Moving Average) при резком скачке координат (например, дрон толкнули). Кто быстрее реагирует?

Задание 62: Задание 6: Посадка в круг

Напишите скрипт, который: 1. Ищет маркер с высоты 1.5 метра. 2. Снижается до 0.5 метра, удерживая маркер в центре кадра (используя P-регулятор

по скоростям V_x, V_y). 3. Если ошибка позиционирования < 5 см в течение 2 секунд — совершает посадку.

❓ Контрольные вопросы

1. Зачем нужен фильтр Калмана, если можно просто взять среднее арифметическое за 10 кадров?
2. Что произойдет с PID-регулятором, если перепутать знак у коэффициента P?
3. В какой системе координат (Body или World) камера выдает вектор $tvec$?

12. Техническая реализация программных модулей

В этой главе мы разберем исходный код инструментов, которые используются для создания карт и навигации.

12.1. Генератор карт (Marker Map Builder)

Скрипт `marker_map_builder.py` позволяет создавать цифровые карты полигонов и экспортировать их в форматы JSON, XML и KML.

12.1.1. Структура класса `Marker`

Каждый маркер описывается классом `Marker`, который хранит:

- `id`: Уникальный номер маркера.
- `size`: Физический размер стороны в метрах.
- `x, y, z`: Локальные координаты относительно начала координат карты.
- `yaw`: Угол поворота (ориентация).
- `dictionary`: Тип словаря (например, `DICT_6X6_250`).

Класс `Marker`

```
class Marker:
    def __init__(self, marker_id, size, x, y, z, yaw=0,
        ↪ dictionary="DICT_6X6_250"):
        self.id = marker_id
        self.size = size # in meters
        self.x = x # local x (meters)
        self.y = y # local y (meters)
        self.z = z # local z (meters) - height
        self.yaw = yaw # orientation (degrees)
        self.dictionary = dictionary
```

12.1.2. Экспорт в XML

Метод `save_xml` генерирует структуру, понятную большинству промышленных систем. Мы используем f-строки для формирования XML-тегов.

Экспорт в XML

```
def save_xml(self, filename):
    xml_content = f"""<?xml version="1.0" encoding="UTF-8"?>
    <map>
```

```

    <name>{self.name}</name>
    <origin>
        <lat>{self.origin_lat}</lat>
        <lon>{self.origin_lon}</lon>
    </origin>
    <markers>
    """
    for m in self.markers:
        xml_content += f"""          <marker id="{m.id}">
            <size>{m.size}</size>
            <position>
                <x>{m.x}</x>
                <y>{m.y}</y>
                <z>{m.z}</z>
            </position>
            <orientation>
                <yaw>{m.yaw}</yaw>
            </orientation>
          </marker>
        """
    xml_content += "    </markers>
</map>"""
    with open(filename, 'w') as f:
        f.write(xml_content)

```

12.1.3. Экспорт в KML для Google Earth

Для отображения маркеров на глобусе Google Earth используется формат KML. Главная сложность здесь — перевод локальных метров (x, y) в глобальные градусы широты и долготы.

Мы используем приближенную формулу (для малых расстояний):

- 1 градус широты ≈ 111132 метра.
- 1 градус долготы $\approx 111132 \cdot \cos(\text{широта})$ метров.

⚠ Внимание

Точность GPS Данная формула имеет погрешность. Для профессиональной геодезии следует использовать библиотеку `pyproj` и проекцию UTM.

12.2. Модуль визуализации

Для проверки правильности карты полезно отрисовать её на 2D-плоскости перед полетом. Библиотека `matplotlib` позволяет построить схе-

му расположения маркеров.

Визуализация карты

```
import matplotlib.pyplot as plt

def plot_map(self):
    x_coords = [m.x for m in self.markers]
    y_coords = [m.y for m in self.markers]
    ids = [m.id for m in self.markers]

    plt.figure(figsize=(10, 10))
    plt.scatter(x_coords, y_coords, c='red', marker='s', s=100)

    for i, txt in enumerate(ids):
        plt.annotate(f"ID {txt}", (x_coords[i], y_coords[i]), xytext=(5,
            ↪ 5), textcoords='offset points')

    plt.grid(True)
    plt.axis('equal')
    plt.title(f"Map: {self.name}")
    plt.xlabel("X (meters)")
    plt.ylabel("Y (meters)")
    plt.show()
```

Задание 63: Задание 7: Свой парсер

Напишите функцию `load_json(filename)`, которая считывает файл карты и создает объекты класса `Marker`. Проверьте, что карта корректно загружается после сохранения.

13. Банк практических заданий

В этом разделе собраны задачи для закрепления материала. Задания разделены по темам и уровням сложности:

- **Базовый (Б):** Проверка понимания основных команд.
- **Продвинутый (П):** Требуется написания алгоритмов.
- **Комплексный (К):** Мини-проект, объединяющий несколько тем.

13.1. Блок 1: Работа с камерой и ArUco (aruco_examples, camera_examples)

1. **(Б) «Фотограф»:** Напишите скрипт, который делает фото каждые 5 секунд и сохраняет их в папку с текущей датой и временем в названии.
2. **(Б) «Детектор движения»:** Сравнивайте два последовательных кадра. Если количество изменившихся пикселей превышает порог — выводите сообщение «Движение обнаружено!».
3. **(П) «Умный светофор»:** Распечатайте красный и зеленый круги. Если дрон видит зеленый цвет — он взлетает на 1 метр. Если красный — садится.
4. **(П) «Поиск маркера»:** Дрон взлетает и вращается вокруг своей оси (yaw). Как только он видит любой ArUco маркер — останавливается и «смотрит» на него 3 секунды.
5. **(П) «Дистанция»:** Дрон висит напротив маркера. Если маркер приближается (человек подходит с маркером в руках) — дрон отлетает назад, сохраняя дистанцию 1 метр.
6. **(К) «Инвентаризация»:** В комнате разложены 5 маркеров с ID от 0 до 4. Дрон должен облететь комнату, найти все маркеры и записать в файл: «ID 3 найден в 12:00», «ID 1 найден в 12:05».
7. **(К) «Посадка на платформу»:** Дрон должен найти маркер на полу, снизиться, зависнуть точно над центром и совершить посадку.

13.2. Блок 2: Телеметрия (get_telemetry)

1. **(Б) «Черный ящик»:** Записывайте высоту, заряд батареи и углы наклона (roll, pitch) в CSV файл 10 раз в секунду во время полета.

2. (Б) «Сигнализация»: Если заряд аккумулятора падает ниже 10.5 Вольт, дрон должен включить красную подсветку и начать посадку.
3. (П) «Альтиметр»: Взлетите и висите. Выводите разницу между высотой по барометру и высотой по дальномеру (ToF). В каких ситуациях они отличаются?
4. (П) «Компас»: Напишите программу, которая разворачивает дрон носом строго на Север (0 градусов), затем на Восток (90), Юг (180) и Запад (270).
5. (К) «Виртуальная стена»: Задайте границы зоны полета (например, X от -2 до 2 метров). Если телеметрия показывает выход за границы — дрон должен остановиться и вернуться в центр.

13.3. Блок 3: Миссии и полет по точкам (mission_examples)

1. (Б) «Квадрат»: Пролетите по точкам $(0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 0)$ на высоте 1 метр.
2. (Б) «Лестница»: Пролетите вперед на 1 метр, набрав 0.5 метра высоты. Повторите 3 раза (как по ступенькам).
3. (П) «Спираль»: Напишите генератор точек для полета по спирали (расширяющийся круг с набором высоты).
4. (П) «Патруль»: Дрон летает между двумя точками бесконечно, пока не увидит препятствие (по датчику расстояния или камере).
5. (К) «Сканирование поля»: Реализуйте полет «змейкой» над зоной 3×3 метра для создания фотоплана местности.

13.4. Блок 4: Групповой полет (group_flight_examples)

1. (Б) «Синхронный старт»: Два дрона взлетают одновременно по команде с ПК, висят 5 секунд и садятся.
2. (П) «Карусель»: Два дрона висят напротив друг друга и меняются местами, пролетая по дуге (чтобы не столкнуться).
3. (П) «Ведущий-ведомый»: Первый дрон летит по программе (квадрат). Второй дрон получает телеметрию первого и повторяет его движения со смещением (на 1 метр сзади).
4. (К) «Шоу дронов»: Составьте хореографию для 4 дронов под музыку (используйте мигание светодиодами в такт).

13.5. Блок 5: Трекинг объектов (PioneerHumanTracking)

1. **(Б) «Приветствие»:** Если дрон видит лицо человека — он мигает зеленым и делает «кивок» (pitch вперед-назад).
2. **(П) «Следи за цветом»:** Возьмите яркий оранжевый мяч. Дрон должен поворачиваться (yaw) так, чтобы мяч всегда был в центре кадра.
3. **(П) «Держи дистанцию»:** Дрон видит лицо. Если лицо занимает слишком много места в кадре (человек близко) — отлетаем. Слишком мало (человек далеко) — подлетаем.
4. **(К) «Оператор»:** Дрон летит боком параллельно идущему человеку, снимая его профиль (требуется сложной координации движений).

13.6. Критерии оценки выполнения

Задание считается выполненным, если:

1. Код запускается без синтаксических ошибок.
2. Дрон выполняет поставленную задачу в реальных условиях не менее 3 раз подряд.
3. Код содержит комментарии, объясняющие логику работы.
4. Соблюдены меры безопасности (дрон не вылетает за пределы зоны, экстренная посадка работает).

14. Типичные ошибки и их устранение

В процессе работы с дроном часто возникают ситуации, когда «код правильный, но ничего не работает». В этой главе собраны самые частые проблемы.

14.1. Проблемы с камерой и маркерами

14.1.1. Дрон не видит маркер

- **Причина 1: Плохое освещение.** Камере темно, выдержка увеличивается, появляется «смаз» (Motion Blur) при движении.
- **Решение:** Включите дополнительный свет. Маркер должен быть освещен равномерно.
- **Причина 2: Блики.** Маркер заламинирован глянцевой пленкой или лежит под лампой.
- **Решение:** Переместите маркер или используйте матовый скотч.
- **Причина 3: Неверный размер.** В коде указан размер 0.2 метра, а напечатан маркер 18 см.
- **Решение:** Измерьте черную рамку линейкой и исправьте константу в коде.
- **Причина 4: Quiet Zone.** Маркер вырезан по черному контуру без белого поля.
- **Решение:** Наклейте его на белый лист бумаги.

14.1.2. Дрон летит не туда

- **Причина 1: Перепутаны оси.** Вы думаете, что X — это вправо, а это вперед.
- **Решение:** Проверьте ориентацию (Body Frame). Ось X всегда смотрит «из камеры» (вперед).
- **Причина 2: Ошибка калибровки.** Камера «рыбий глаз» сильно искажает картинку по краям.
- **Решение:** Проведите калибровку камеры заново.
- **Причина 3: Yaw (Рысканье).** Дрон повернулся на 90 градусов, но код продолжает отправлять команды в старой системе координат.
- **Решение:** Используйте глобальную систему координат (World Frame) или

учитывайте текущий угол Yaw.

14.2. Проблемы с подключением и Wi-Fi

14.2.1. Connection Refused / Timeout

- **Причина 1:** Компьютер подключился к другой Wi-Fi сети (с интернетом).
- **Решение:** Проверьте, что вы подключены к точке доступа Pioneer_XXXX.
- **Причина 2:** Брандмауэр Windows блокирует порт 8000/8888.
- **Решение:** Разрешите Python доступ в частные сети.

14.3. Ошибки в коде Python

14.3.1. IndentationError

Python очень чувствителен к отступам. Не смешивайте табы и пробелы. Используйте 4 пробела для отступа.

14.3.2. OpenCV Error: Assertion failed

Обычно означает, что функция получила пустое изображение (None).

- Проверьте, что `camera.get_frame()` не вернул None.
- Проверьте, что файл изображения существует (если читаете с диска).

14.4. Аварийные ситуации

14.4.1. Дрон не реагирует на Land

Если программная посадка не срабатывает:

1. Не паникуйте.
2. Нажмите кнопку Disarm в Ground Station (если есть).
3. Аккуратно поймите дрон снизу за аккумулятор (берегите пальцы от пропеллеров!) и переверните его вверх ногами. Моторы отключатся автоматически.

⚠ Внимание

Внимание! Никогда не пытайтесь схватить дрон сверху или сбоку — пропеллеры вращаются со скоростью 10 000 об/мин и могут нанести серьезные травмы.

А. Методика тестирования и безопасность

А.1. Протокол безопасности полетов

А.1.1. 1. Подготовка зоны (Pre-flight)

- Очистить зону 3×3 метра.
- Закрыть окна (ветер).
- Убрать животных и детей.
- Предупредить окружающих о запуске.
- Надеть защитные очки (для оператора).

А.1.2. 2. Проверка дрона (Checklist)

- **Пропеллеры:** Целые, затянуты, вращаются свободно.
- **Аккумулятор:** Заряжен $> 3.8V$ на банку, закреплен надежно.
- **Камера:** Линза чистая, шлейф не болтается.
- **Wi-Fi:** Сигнал стабильный ($RSS > -60 \text{ dBm}$).

А.1.3. 3. Экстренная ситуация

Если дрон ведет себя неадекватно:

1. Громко крикнуть «ПОСАДКА!».
2. Нажать кнопку LAND (если есть).
3. Если нет реакции — аккуратно поймать снизу (за батарею) и перевернуть.
4. Отключить аккумулятор.

А.2. Тестирование в разных условиях

А.2.1. Погодные условия (Ветер)

Дрон Pioneer очень легкий (около 100 г). Он не предназначен для полетов на улице при ветре $> 3 \text{ м/с}$. Для теста в помещении используйте вентилятор:

- Включите вентилятор на минимальную скорость с расстояния 2 метра.
- Попробуйте удерживать точку (режим Optical Flow).
- Если дрон сносит $> 50 \text{ см}$ — условия непригодны для точной навигации.

А.2.2. Освещение (Lux)

Камера Pioneer имеет низкий динамический диапазон.

- **Яркое солнце:** Пересвет (белое пятно). Маркеры не видны.
- **Сумерки:** Сильный шум матрицы. Маркеры «дрожат».
- **Мерцание ламп:** Люминесцентные лампы (50 Гц) могут давать полосы на видео.

В. Готовые конфигурации (Templates)

В.1. Файл конфигурации camera_params.yaml

 camera_params.yaml

```
%YAML:1.0
image_width: 640
image_height: 480
camera_matrix: !!opencv-matrix
  rows: 3
  cols: 3
  dt: d
  data: [ 600.0, 0., 320.0, 0., 600.0, 240.0, 0., 0., 1. ]
distortion_coefficients: !!opencv-matrix
  rows: 1
  cols: 5
  dt: d
  data: [ 0.1, -0.2, 0., 0., 0. ]
```

В.2. Шаблон скрипта полета (main.py)

 Базовый шаблон

```
import time
from pioneer_sdk import Pioneer

def main():
    drone = Pioneer()
    try:
        drone.arm()
        drone.takeoff()
        time.sleep(3)

        # ... Ваш код ...

    drone.land()
```

```
except Exception as e:
    print(f"Error: {e}")
    drone.land()
finally:
    drone.disarm()
    drone.close_connection()

if __name__ == "__main__":
    main()
```

Список литературы

1. Camera calibration with OpenCV. — 2026. — URL: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html (visited on 03/04/2026).
2. MediaPipe Pose. — 2026. — URL: https://developers.google.com/mediapipe/solutions/vision/pose_landmarker (visited on 03/04/2026).
3. *Slatkin B.* Effective Python : 90 Specific Ways to Write Better Python. — 2nd ed. — Addison-Wesley Professional, 2019.